Formal Verification Report: Lido Lend Markets

- Date: September 1st, 2026
- Author: AlexZoid (Cyfrin private formal verification engagement)
- Certora Prover version: 8.16.2

Table of Contents

1. Verification Scope
2. Methodology
    - Setup Scenes
    - Types of Properties
    - Types of Assumptions
3. Properties
    - GuardianSignals
    - LidoLendMarketController
    - LidoLend
    - LidoLendERC4626
    - LidoLend_LinkedController
    - LidoLendERC4626_LinkedCore
    - Erc4626ExitBook_LinkedVaultCore
    - MorphoExitBook_LinkedCore
4. Real Issues Found
5. Mutation Testing
6. Reproducing the Results
    - Prerequisites
    - Remote Execution
    - Local Execution
    - Running the Suites
7. Resources

Verification Scope

Lido Lend is a Morpho Blue fork for isolated, permissionless loop markets. What separates it from the original is a per-market policy layer: a controller hook every core operation must pass, a guardian committee that can slow or block a borrower, and a par-value exit that lets a supplier leave against borrower debt when free liquidity cannot cover the withdrawal. The scope spans two codebases, `lido-lend/` with the forked core and `lido-lend-markets/` with the policy and periphery layer built around it, each read at the commit the audit reviewed and re-read at the commit its fixes landed on:

Repository	Audit Commit	Mitigation Commit
lido-lend-markets	`0e3e16af06c632fb3eeda965bcde2f183cc8d854`	`2e5852e45a7cd1ab6734de9f7c6a4eadf5b90075`
lido-lend	`ec7171654bdb7211e07bf696a813f2e7a290a1e2`	`490f9d7b3aba2edde9403f90ffa6a69628fdd44f`

The prover read those trees through the audit repository, audit-2026-07-lido-lend, which carries both of them as subtrees: every verdict below was taken at `0f0033ee22b91b762d344060c82e94802a01c1e3` and every mitigation at `d575790b48d022a63f89f1c7495fa9cf945952ef`.

The contracts in scope, grouped by source tree:

The forked core, in `lido-lend/src/`:

1. **LidoLend** (`LidoLend.sol`): the forked Morpho Blue core, which adds to the original a per-market controller bound at creation whose hook fires on every state-moving entry point, and a par-value `settle` exit that burns a supplier's shares to cancel borrower debt and seize the matching collateral.

The markets layer, in `lido-lend-markets/src/`:

2. **GuardianSignals** (`GuardianSignals.sol`): the guardian registry, a committee shared across markets whose members flag accounts to block their borrowing and suspect them to raise their collateral activation delay.
3. **LidoLendMarketController** (`controllers/LidoLendMarketController.sol`): the per-market policy controller, called by the core on every supply, withdraw, borrow and settle. It caps what suppliers may add, holds new collateral until it matures and turns away a banned borrower, and keeps borrowers ordered by risk so a supplier settling against debt meets the riskiest one first.
4. **LidoLendERC4626** (`LidoLendERC4626.sol`): the tokenized lender vault, a share token bound to one core market that supplies deposits into its core position, takes its deposit and withdrawal limits from the controller, and settles against borrower debt when free liquidity cannot cover an exit.
5. **Erc4626ExitBook** (`periphery/Erc4626ExitBook.sol`): the ERC-4626 exit book, where a lender posts an exit request backed by its vault shares and a filler redeems those shares to pay it, over the vaults governance allows.
6. **MorphoExitBook** (`periphery/MorphoExitBook.sol`): the Morpho exit book, the same order book backed by a lender's supply position instead of vault shares, filled by withdrawing from it on the markets governance allows.

These files, in scope for the audit, enter no scene here, so nothing below states anything about them:

- the oracle adapters, `WstEthRateOracle.sol` and `UsdyStableOracle.sol`
- the governance facades, `LidoLendOwner.sol` and `RiskStewardDashboard.sol`
- the upgrade shell, `OssifiableProxy.sol`
- the core's read helpers, `MorphoLib.sol`, `MorphoBalancesLib.sol` and `MorphoStorageLib.sol`, which no contract in scope calls
- the rate models in `lido-lend-irm/`, the third codebase of the audit, which this edition does not carry

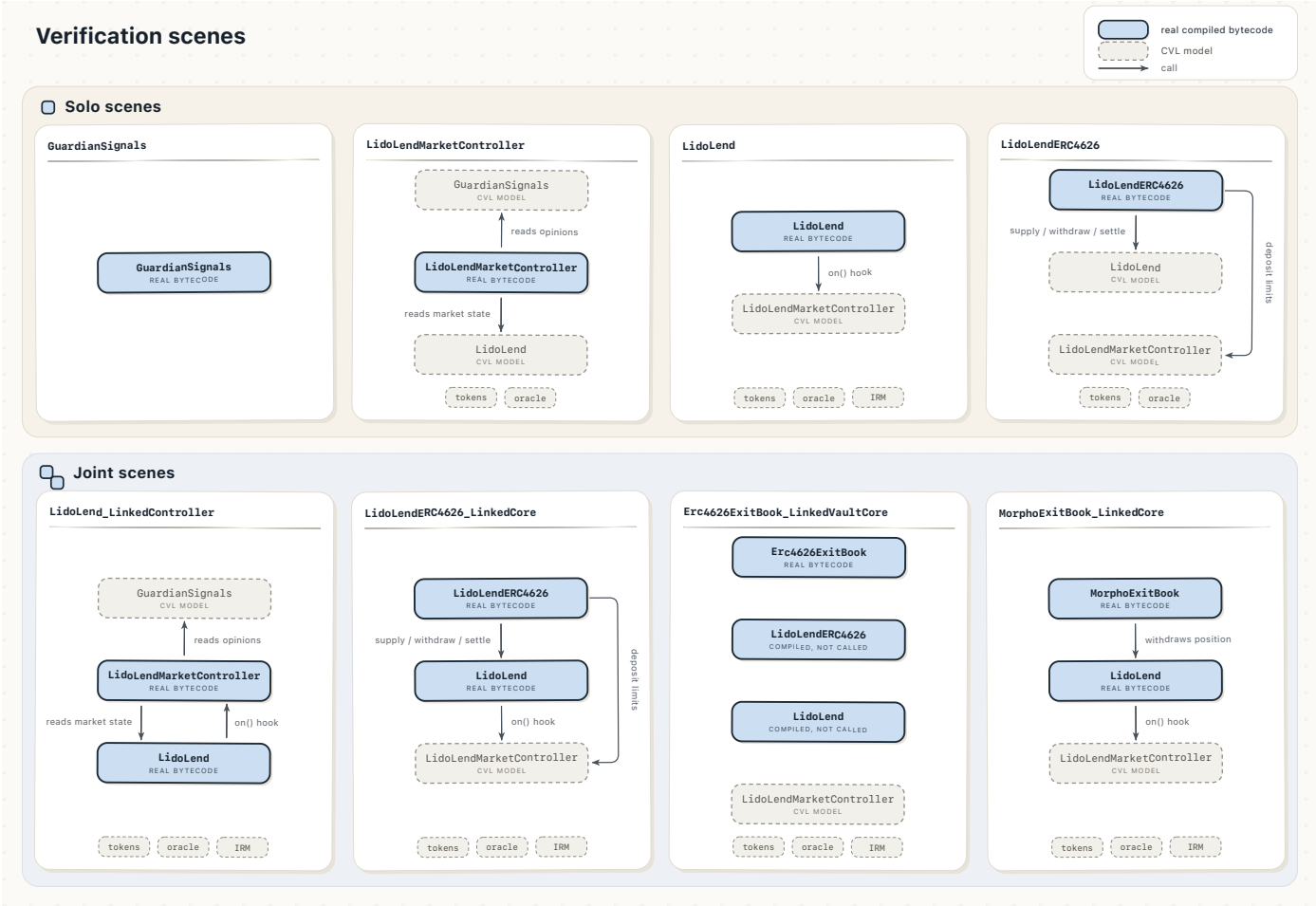
Methodology

Certora Formal Verification (FV) provides mathematical proofs of smart contract correctness by verifying code against a formal specification. Unlike testing and fuzzing which examine specific execution paths, Certora FV examines all possible states and execution paths.

The process involves crafting properties in CVL (Certora Verification Language) and submitting them alongside compiled Solidity smart contracts to the prover. The prover transforms the contract bytecode and rules into a mathematical model and determines the validity of rules.

Setup Scenes

Each contract is verified over its **real** bytecode and stands in as a **CVL model** inside another contract's scene.



Each target's **valid state** is a pack of invariants proven over that scene and then assumed by every rule there through `requireInvariant`, so a rule reasons only about states the contract can actually reach and its counterexamples are real ones.

A property spanning two contracts runs on a joint scene named `A_LinkedB`, with its own section below. How much of the neighbour such a scene brings differs, and the difference is worth reading first. `LidoLend_LinkedController` and `LidoLendERC4626_LinkedCore` run both contracts' real bytecode and assume both valid states, so the seam is checked rather than modeled. `MorphoExitBook_LinkedCore` executes the seam too, its `MORPHO` address linked to the compiled core, but assumes none of the core's invariants, which widens the states it reasons over. `Erc4626ExitBook_LinkedVaultCore` compiles the vault and the core in for their state alone: the book reaches a vault through an unlinked `vaultOf` address, so that call resolves into the shared ERC4626 model and their code is never entered.

Types of Properties

The five categories of the official Certora methodology, extended with Accrue Interest, ERC4626, Reachability, Reverts and Access Control. A **parametric** category is checked against every external function of the scene, including ones added after the specification; the rest name the call they are about.

Valid State (parametric): system-wide invariants that hold in every reachable state; once proven, other properties reuse them as preconditions through `requireInvariant`.

Variable Transitions (parametric): one storage variable at a time: snapshot it, run any external call, then assert it moved only as its own law allows, or not at all.

State Transitions (parametric): the moves between valid states, so a state change is authorized and follows the order the protocol defines.

High Level: a whole operation or a pair of them: what a call must leave untouched, which effects travel together, and which of them reverse. Frame laws among them are stated parametrically.

Unit Tests: the exact effect of one named call: the cells a setter writes, the entry an operation leaves behind, the value a view derives.

Accrue Interest: every accruing entry point behaves identically whether or not `accrueInterest` ran first in the same block.

ERC4626: EIP-4626 conformance: previews bound their live counterparts, the max functions are total and achievable, and conversions round toward the vault.

Reachability: `satisfy()` witnesses that each workflow can execute and take effect, so no property above is proven over a scene where the mutators never fire.

Reverts: exactly when a call must fail and when it must not, so a caller is neither denied a legitimate operation nor slipped past a guard.

Access Control (parametric): for each piece of state, the callers that may move it.

Whatever its category, a rule that proves nothing reads the same as one that proves something, so each is given a mutant: a focused break planted in the source that the rule must reject. The Mutations column names it, and its entry, under Mutation Testing, carries the diff and the command to reproduce the kill.

Types of Assumptions

The prover explores every input and every storage state, so a `require` in a rule body or in the scene setup narrows that space: to drop unreachable states that would answer with a false counterexample, to keep the solver inside its budget, or to aim the rule at one execution path. The tag on each says which.

SAFE: excludes no reachable state. It holds in any realistic deployment, follows from EVM semantics, or restates a narrowing imposed elsewhere and is kept for solver tractability (`msg.sender != 0`).

SCOPE: a regime pin fixing which flow the rule reasons about; the excluded region is covered by a sibling rule or is immaterial, so no reachable, uncovered state is dropped (a bystander cell, when the cell acted on has its own rule).

UNSAFE: does drop reachable, uncovered states. Narrowed deliberately for tractability or a tool limit, with nothing else covering what it excludes (the guardian committee bounded to three members).

TRUSTED: guaranteed outside the scene by a trusted party (governance, an admin, an un-modeled ratifier); the excluded paths are misconfiguration that party prevents (the initialized proxy state, established at deployment).

PROVED: not an assumption. A fact discharged elsewhere and reused as a precondition: a verified invariant injected through `requireInvariant`, or a proven rule cited by its ID (`frontierNotInFuture`).

ASSERT: not an assumption. Inside a CVL summary it stands in for a revert the summarized code performs (the `AccessControlUnauthorizedAccount` check inside the role model).

Properties

Every verdict below comes from a locally built prover, the version named in the header.

- **Properties**: every invariant and rule the suite states
- **Proved**: verified by the prover
- **Timeout**: still undecided after the solver budget the confs carry, and such a rule whose mutant is killed is taken as working (the prover finds a violation far sooner than it proves there is none)
- **Mutations**: rules with a mutant the prover killed, meaning the rule holds on the real code and breaks on a deliberately faulted one
- **Violations**: rules that fail, each tied to a fault in the audited code

Contract	Properties	Proved ✅	Timeout ⏱	Mutations 🎯	Violations ❌	Issues
GuardianSignals	227	224	0	207	3	#8 #54
LidoLendMarketController	199	186	9	195	4	#11 #68 #73
LidoLend	187	169	16	185	2	#12 #42
LidoLendERC4626	125	93	32	125	0	
LidoLend_LinkedController	41	33	8	41	0	
LidoLendERC4626_LinkedCore	16	15	0	15	1	#71
Erc4626ExitBook_LinkedVaultCore	128	128	0	128	0	
MorphoExitBook_LinkedCore	152	151	1	152	0	
Total	**1075**	**999**	**66**	**1048**	**10**	**8**

Every property is expected to carry a mutant that breaks it. A row without one is either a valid state invariant about an external library's own storage, such as the OpenZeppelin set GuardianSignals uses, which no fault planted in the audited contract reaches, or a rule a real issue already violates, where the finding stands in for the planted fault.

- **Properties**: ✅ Verified · ❌ Violated · ⏱ Timeout · ⏳ written, its run not done yet
- **Mutations**: 🎯 the mutant this rule kills · ⏳ the mutant is written and waiting for its run · ⏱ the mutant ran and the prover did not decide · · no legal mutant exists · 🔻 after a 🎯: that kill was taken in a narrowed scene, not in the one the rule's own conf builds
- **Mitigations**: ❌ → ✅ Violated, then proved on the mitigation commit · ❌ → ✖ Violated, and the mitigation commit no longer carries the rule (its finding says why)

Narrowed scenes

A rule that does not decide over the scene its own conf builds still has to show it has teeth, and that is what its mutant does, so a few mutants run in a lighter conf beside the parent one: it folds a part of the scene the rule does not speak about, an arithmetic helper answered by a CVL summary, a sweep bounded to one iteration, a neighbour driven as a model rather than as linked bytecode. The rule text there is the parent's, name and body alike, and what the lane folds is stated at the head of its spec, which the mutant's own entry links.

GuardianSignals

- Sets holding up to 3 of the contract's 32

- Loops running up to 3 iterations

Valid State

Identification	Name	Description	Properties	Mutations
GS-VS-01 … GS-VS-17	`committee*`, `flaggedBy*`, `suspectedBy*`	The three guardian address sets are well formed, stated on purpose so that a scene running the real set library starts from a coherent state. No legal mutant exists for the family: the set layout is maintained by the OZ library code, outside the mutation target.	✅	.
GS-VS-18	`suspectStampImpliesMembership`	No suspect stamp survives the report it belongs to. `suspectedAt[id][a][g] != 0 => suspectedBy[id][a].contains(g)`	✅	🎯
GS-VS-19	`heartbeatNotInFuture`	A guardian can never report having scanned into the future. `heartbeats[g] <= block.timestamp`	✅	🎯
GS-VS-20	`suspectedAtNotInFuture`	No suspect report is backdated into the future. `suspectedAt[id][a][g] <= block.timestamp`	✅	🎯
GS-VS-21	`quorumBoundedByCommittee`	The quorum never demands more votes than there are guardians to cast them. `quorum <= guardians.length`	✅	🎯
GS-VS-22	`suspectWindowBounded`	Suspicion can never be made to last longer than the contract's own cap. `suspectWindow <= MAX_SUSPECT_WINDOW`	✅	🎯
GS-VS-23	`suspectSnapshotNotInFuture`	A borrower's suspicion snapshot is never dated in the future. `suspectReportedAt[id][a] <= block.timestamp`	✅	🎯
GS-VS-24	`suspectCountBoundedByCommittee`	An account's suspect tally never exceeds the modeled committee cap. `suspectCount[id][a] <= MAX_COMMITTEE_GS_CVL (model committee cap 3)`	✅	🎯
GS-VS-25	`suspectCountAnchoredBySnapshot`	A live suspect count always carries the date it was taken. `suspectCount[id][a] > 0 => suspectReportedAt[id][a] > 0`	✅	🎯
GS-VS-26	`emptySnapshotImpliesEmptySuspectSet`	An account that was never snapshotted holds no suspect reports. `suspectReportedAt[id][a] == 0 => suspectedBy[id][a].length == 0`	✅	🎯
GS-VS-27	`suspectTimestampBoundedBySnapshot`	The account's snapshot is at least as recent as any report behind it. `suspectedAt[id][a][g] <= suspectReportedAt[id][a]`	✅	🎯
GS-VS-28	`membershipImpliesSuspectStamp`	Every live suspect report carries the time it was filed. `suspectedBy[id][a].contains(g) => suspectedAt[id][a][g] != 0`	✅	🎯

GS-VS-29	`frontierNotInFuture`	The frontier never runs ahead of the chain. `_quorumFrontierTimestamp <= block.timestamp`	✅	🎯
GS-VS-30	`quorumAtLeastOne`	The ban quorum never reads zero, so nobody is banned by default. `_quorum >= 1`	✅	🎯

Variable Transitions

Identification	Name	Description	Properties	Mutations
GS-VT-01	`vtGsHeartbeatNeverRegresses`	A guardian heartbeat never regresses. `heartbeats[g]' >= heartbeats[g]`	✅	🎯🎯
GS-VT-02	`vtGsCommitteeSizeStepsByOne`	The committee size moves by at most one per call. `guardians.length' >= guardians.length - 1 AND guardians.length' <= guardians.length + 1`	✅	🎯
GS-VT-03	`vtGsSnapshotClockMonotone`	The suspect snapshot clock never rewinds. `signals[id][a].suspectReportedAt' >= signals[id][a].suspectReportedAt`	✅	🎯
GS-VT-04	`vtGsSuspectStampRisesOrClears`	A suspect stamp only refreshes forward or clears. `signals[id][a].suspectedAt[g]' == 0 OR signals[id][a].suspectedAt[g]' >= signals[id][a].suspectedAt[g]`	✅	🎯
GS-VT-05	`vtGsFlaggerSetGrowsByOne`	The flagger set gains at most one member per call. `signals[id][a].flaggedBy.length' <= signals[id][a].flaggedBy.length + 1`	✅	🎯
GS-VT-06	`vtGsSuspectSetGrowsByOne`	The suspect set gains at most one member per call. `signals[id][a].suspectedBy.length' <= signals[id][a].suspectedBy.length + 1`	✅	🎯
GS-VT-07	`quorumFrontierNeverRegresses`	The quorum-scan frontier never moves backwards. `_quorumFrontierTimestamp' >= _quorumFrontierTimestamp`	✅	🎯
GS-VT-08	`quorumFrontierMovesOnlyInSync`	Nothing but the quorum-scan sync writes the frontier. `_quorumFrontierTimestamp' != _quorumFrontierTimestamp => f == syncQuorumBlock()`	✅	🎯

State Transitions

Identification	Name	Description	Properties	Mutations
GS-ST-01	`stGsSnapshotRestamped`	A suspect-count change re-stamps the snapshot clock. `signals[id][a].suspectCount' != signals[id][a].suspectCount => signals[id][a].suspectReportedAt' == block.timestamp`	✅	🎯🎯
GS-ST-02	`stGsFlagLatchNeedsQuorum`	A flag-driven latch fires only at quorum. `(!signals[id][a].banned AND signals[id][a].banned' AND signals[id][a].flaggedBy.length' > signals[id][a].flaggedBy.length) => signals[id][a].flaggedBy.length' >= quorum`	✅	🎯🎯
GS-ST-03	`stGsHeartbeatCallerOnly`	A heartbeat moves only for the caller. `heartbeats[g]' != heartbeats[g] => g == msg.sender`	✅	🎯
GS-ST-04	`stGsSuspectAddStampsCaller`	A suspect-set growth stamps the caller's report at now. `signals[id][a].suspectedBy.length' > signals[id][a].suspectedBy.length => signals[id][a].suspectedAt[msg.sender]' == block.timestamp`	✅	🎯

High Level

Identification	Name	Description	Properties	Mutations
GS-HL-01	`hlGsFlagLeavesOtherAccounts`	Flag touches no other flagger set. `flag(id, [a]): (id2, b) != (id, a) => flaggedByLen[id2][b]' == flaggedByLen[id2][b]`	✅	🎯
GS-HL-02	`hlGsUnbanLeavesOtherAccounts`	Unban touches no other ban latch. `unban(id, a): (id2, b) != (id, a) => banned[id2][b]' == banned[id2][b]`	✅	🎯
GS-HL-03	`hlGsPruneKeepsBan`	The permissionless cleanup call never changes an account's ban. `prune(id, a): banned[id][a]' == banned[id][a]`	✅	🎯
GS-HL-04	`hlGsUnflagKeepsBan`	A guardian retracting its strike never changes the ban on the account. `unflag(id, [a]): banned[id][a]' == banned[id][a]`	✅	🎯
GS-HL-05	`hlGsBanKeepsFlaggers`	Banning an account leaves the guardians' strike record on it untouched. `ban(id, a): flaggedBy[id][a].length' == flaggedBy[id][a].length`	✅	🎯
GS-HL-06	`hlGsPruneEvictsExGuardianFlag`	Prune evicts a non-guardian's flag. `prune(id, a): !isGuardian(g) => g not in flaggedBy[id][a]'`	✅	🎯
GS-HL-07	`hlGsPruneEvictsExGuardianSuspect`	Prune evicts a non-guardian's suspect report. `prune(id, a): !isGuardian(g) => g not in suspectedBy[id][a]'`	✅	🎯🎯
		Flag then unflag restores the strike count, latched or not. `msg.sender not in flaggedBy[id][a]:`		

GS-HL-08	`hlGsFlagUnflagRoundTrip`	`flag(id, [a]); unflag(id, [a]) => flaggedByLen[id][a]' == flaggedByLen[id][a]`	✅	🎯
GS-HL-09	`hlGsRemoveGuardianLeavesOtherMembers`	removeGuardian never flips another member's membership. `removeGuardian(g): h != g => isGuardian(h)' == isGuardian(h)`	✅	🎯
GS-HL-10	`hlGsAddGuardianKeepsStrikes`	Adding a guardian retallies no strikes and moves no strike threshold. `addGuardian(g): flaggedBy[id][a].length' == flaggedBy[id][a].length AND quorum' == quorum`	✅	🎯
GS-HL-11	`hlGsRemoveGuardianKeepsStrikes`	Removing a guardian retallies no strikes and moves no strike threshold. `removeGuardian(g): flaggedBy[id][a].length' == flaggedBy[id][a].length AND quorum' == quorum`	✅	🎯
GS-HL-12	`hlGsRotationKeepsBan`	Rotation moves neither the ban latch nor the effective ban. `addGuardian(g): banned[id][a]' == banned[id][a] AND isBanned(id, a)' == isBanned(id, a)`	✅	🎯
GS-HL-13	`hlGsBanLiftedOnlyByUnban`	A latched ban is cleared only by unban, never by a guardian retracting its flag. `banned[id][a] AND !banned[id][a]' => f == unban(id, a)`	✅	🎯
GS-HL-14	`hlGsPruneNeverWorsensOpinion`	Prune is relief-only: strikes may fall, an unset ban stays unset, suspect and frontier hold. `prune(id, a): strikeCount' <= strikeCount AND (!banned => !banned') AND suspectCount' == suspectCount AND suspectExpiryTimestamp' == suspectExpiryTimestamp AND quorumFrontierTimestamp' == quorumFrontierTimestamp`	✅	🎯
GS-HL-15	`hlGsRepeatFlagLatchesNoBan`	A repeat flag by an existing flagger latches no ban. `msg.sender in flaggedBy[id][a]: flag(id, [a]) => banned[id][a]' == banned[id][a]`	✅	🎯
GS-HL-16	`hlGsFirstSuspectReportCountsOne`	The first suspect report snapshots exactly one live suspicion. `suspectedBy[id][a] empty: suspect(id, [a]) => suspectedBy[id][a].length' == 1 AND suspectCount[id][a]' == 1 AND suspectedAt[id][a][msg.sender]' == now AND suspectReportedAt[id][a]' == now`	✅	🎯🎯
GS-HL-17	`hlGsSoleReporterDrainsToZero`	Retracting the sole live report drains the snapshot to zero. `suspectedBy[id][a] == {msg.sender}, suspectedAt[id][a][msg.sender] + suspectWindow >= now: unsuspect(id, [a]) => suspectedByLen[id][a]' == 0 AND suspectCount[id][a]' == 0 AND suspectedAt[id][a][msg.sender]' == 0 AND suspectReportedAt[id][a]' == now`	✅	🎯
		A lapsed report is swept out and never reaches the tally. `suspectedBy[id][a] == {g}, g != msg.sender, suspectedAt[id][a][g] +`		

GS-HL-18	`hlGsExpiredReportNotCounted`	`suspectWindow < now: suspect(id, [a]) => g not in suspectedBy[id][a]' AND suspectedAt[id][a][g]' == 0 AND suspectedByLen[id][a]' == 1 AND suspectCount[id][a]' == 1`	✅	🎯
GS-HL-19	`hlGsBoundaryReportStillCounted`	A report exactly at the window edge survives the sweep and counts. `suspectedBy[id][a] == {g}, g != msg.sender, suspectedAt[id][a][g] + suspectWindow == now: suspect(id, [a]) => suspectedByLen[id][a]' == 2 AND g in suspectedBy[id][a]' AND suspectCount[id][a]' == 2`	✅	🎯🎯🎯
GS-HL-20	`hlGsRepeatSuspectRefreshesStamp`	A repeat report refreshes the caller's stamp without double-counting the suspicion. `suspectedBy[id][a] == {msg.sender}, suspectedAt[id][a][msg.sender] + suspectWindow >= now: suspect(id, [a]) => suspectedByLen[id][a]' == 1 AND suspectCount[id][a]' == 1 AND suspectedAt[id][a][msg.sender]' == now`	✅	🎯
GS-HL-21	`hlGsSetQuorumBanCharacterization`	setQuorum freezes every per-signal cell and re-scores the effective ban against the new quorum. `setQuorum(q): banned[id][a], flaggedByLen[id][a], suspectCount[id][a], suspectReportedAt[id][a] all unchanged AND isBanned(id, a)' == (banned[id][a] OR flaggedByLen[id][a] >= q)`	✅	🎯🎯
GS-HL-22	`hlGsFlagKeepsSoftChannel`	A strike never touches the softer suspect reports standing against the same account. `flag(id, [a]): suspectedBy[id][a].length, suspectCount[id][a], suspectReportedAt[id][a], suspectedAt[id][a][g] all unchanged`	✅	🎯
GS-HL-23	`hlGsUnflagKeepsSoftChannel`	Retracting a strike never touches the softer suspect reports standing against the same account. `unflag(id, [a]): suspectedBy[id][a].length, suspectCount[id][a], suspectReportedAt[id][a], suspectedAt[id][a][g] all unchanged`	✅	🎯
GS-HL-24	`hlGsSuspectKeepsHardChannel`	A suspect report never adds a strike and never bans the account it names. `suspect(id, [a]): banned[id][a], flaggedBy[id][a].length AND (g in flaggedBy[id][a]) all unchanged`	✅	🎯
GS-HL-25	`hlGsUnsuspectKeepsHardChannel`	Retracting a suspect report never changes the strikes or the ban on that account. `unsuspect(id, [a]): banned[id][a], flaggedBy[id][a].length AND (g in flaggedBy[id][a]) all unchanged`	✅	🎯
GS-HL-26	`hlGsUnbanEndsEffectiveBan`	An unban ends the effective ban outright. `unban(id, a) => isBanned(id, a)' == false`	✅	🎯
GS-HL-27	`hlGsBatchFlagFlagsBothTargets`	One batch call lands the caller's strike on every listed account. `flag(id, [a, b]), a != b => msg.sender in flaggedBy[id][a]' AND msg.sender in flaggedBy[id][b]'`	✅	🎯
		A duplicated batch entry lands a single strike.		

GS-HL-28	`hlGsBatchDuplicateFlagCountsOnce`	`flag(id, [a, a]) by a guardian not yet flagging a => flaggedBy[id][a].length' == flaggedBy[id][a].length + 1`	✅	🎯
GS-HL-29	`hlGsSetSuspectWindowExpiryCharacterization`	Changing the suspect window rewrites no account's signals yet re-dates every report's expiry to the new window. `setSuspectWindow(w): banned[id][a], flaggedBy[id][a].length, suspectCount[id][a], suspectReportedAt[id][a] all unchanged AND suspectExpiryTimestamp(id, a)' == suspectReportedAt[id][a] + w`	✅	🎯
GS-HL-30	`hlGsRemoveGuardianKeepsEffectiveBan`	Removing a guardian never moves the effective ban; the removed member's cast flags keep counting. `removeGuardian(g): isBanned(id, a)' == isBanned(id, a)`	✅	🎯
GS-HL-31	`hlGsStaleSuspectCountServedAfterExpiry`	An individually lapsed report stays charged at full weight until the next re-snapshot. `suspect@t1 by g1; suspect@t2 by g2, t1+W >= t2; read@t3, t1+W < t3 <= t2+W => getOpinion(id, a).suspectCount == 2 AND suspectedAt[id][a][g1] + W < t3 AND suspectExpiryTimestamp >= t3`	✅	🎯
GS-HL-32	`hlGsRetractionExtendsSurvivorExpiry`	A retraction leaving a live survivor re-anchors the account expiry to now. `suspectedBy[id][a] == {g, msg.sender}, both live, suspectReportedAt[id][a] < now: unsuspect(id, [a]) => suspectedByLen[id][a]' == 1 AND g in suspectedBy[id][a]' AND suspectCount[id][a]' == 1 AND suspectedAt[id][a][g]' == suspectedAt[id][a][g] AND suspectReportedAt[id][a]' == now > suspectReportedAt[id][a] AND suspectReportedAt[id][a]' + suspectWindow > suspectedAt[id][a][g] + suspectWindow`	✅	🎯
GS-HL-33	`hlGsFreshReportExtendsSurvivorExpiry`	Another guardian's fresh report extends the account expiry an older live report is scored by. `suspectedBy[id][a] == {g}, g != msg.sender, suspectedAt[id][a][g] live AND < now: suspect(id, [a]) => suspectedByLen[id][a]' == 2 AND suspectCount[id][a]' == 2 AND suspectedAt[id][a][g]' == suspectedAt[id][a][g] AND suspectReportedAt[id][a]' == now AND suspectReportedAt[id][a]' + suspectWindow > suspectedAt[id][a][g] + suspectWindow`	✅	🎯
GS-HL-34	`quorumRaiseKeepsStaleFrontier`	Raising the quorum never tightens the ratcheted frontier. `setQuorum(q'); syncQuorumBlock(): q' > q AND count(g in guardians: _heartbeats[g] >= _quorumFrontierTimestamp) < q' => _quorumFrontierTimestamp'' == _quorumFrontierTimestamp`	✅	🎯
GS-HL-35	`hlGsPruneKeepsEffectiveBan`	The permissionless prune never ends an effective ban. `suspectedBy[id][a] == {}, isBanned(id, a): prune(id, a) => isBanned(id, a)'`	❌→✅	Issue #8 (LOW)
		A retraction never pushes the account anchor forward.		

GS-HL-36	`hlGsRetractionNeverExtendsExpiry`	`suspectedBy[id][a] == {g, msg.sender}, both live, suspectReportedAt[id][a] < now: unsuspect(id, [a]) => suspectReportedAt[id][a]' <= suspectReportedAt[id][a]`	❌→✅	Issue #54 (LOW)

Unit Tests

Identification	Name	Description	Properties	Mutations
GS-UT-01	`utGsSetQuorumStoresValue`	Setting the quorum stores exactly the requested value. `setQuorum(q): quorum' == q`	✅	🎯
GS-UT-02	`utGsSetQuorumKeepsCommitteeSize`	Setting the quorum leaves the committee size untouched. `setQuorum(q): committeeLen' == committeeLen`	✅	🎯
GS-UT-03	`utGsSetSuspectWindowStoresValue`	Setting the suspect window stores exactly the requested value. `setSuspectWindow(w): suspectWindow' == w`	✅	🎯
GS-UT-04	`utGsAddGuardianMakesMember`	Adding a guardian makes it a committee member. `addGuardian(g): g in guardians'`	✅	🎯
GS-UT-05	`utGsAddGuardianGrowsCommitteeByOne`	Adding a fresh guardian grows the committee by exactly one. `g not in guardians => addGuardian(g): committeeLen' == committeeLen + 1`	✅	🎯
GS-UT-06	`utGsRemoveGuardianDropsMember`	Removing a guardian drops it from the committee. `removeGuardian(g): g not in guardians'`	✅	🎯
GS-UT-07	`utGsRemoveGuardianShrinksCommitteeByOne`	Removing a member shrinks the committee by exactly one. `g in guardians => removeGuardian(g): committeeLen' == committeeLen - 1`	✅	🎯
GS-UT-08	`utGsHeartbeatStoresReportedTimestamp`	A heartbeat stores exactly the caller-reported scanned-block timestamp, not block.timestamp. `heartbeat(blk): heartbeats[msg.sender]' == blk.timestamp`	✅	🎯
GS-UT-09	`utGsBanSetsLatch`	Ban latches the account's ban for the named market. `ban(id, a): banned[id][a]'`	✅	🎯
GS-UT-10	`utGsBanLeavesOtherAccounts`	Banning one account leaves every other account's ban, in this market and any other, exactly as it was. `(id2, b) != (id, a) => ban(id, a): banned[id2][b]' == banned[id2][b]`	✅	🎯
GS-UT-11	`utGsUnbanClearsLatch`	Unban clears the account's ban latch for the named market. `unban(id, a): banned[id][a]' == false`	✅	🎯
GS-UT-12	`utGsUnbanEmptiesFlaggerSet`	Unban empties the account's flagger set, banned before or not. `unban(id, a): flaggedByLen[id][a]' == 0`	✅	🎯🎯
GS-UT-13	`utGsOpinionBannedIsEffectiveBan`	Opinion.banned is the effective ban (stored latch or strikes at quorum). `getOpinion(id, a).banned == (banned[id][a] OR flaggedByLen[id][a] >= quorum)`	✅	🎯🎯
GS-UT-14	`utGsOpinionStrikeCountIsFlaggerSetSize`	opinion.strikeCount is the live flagger-set size. `getOpinion(id, a).strikeCount == flaggedByLen[id][a]`	✅	🎯🎯
GS-UT-15	`utGsOpinionSuspectCountIsSnapshot`	opinion.suspectCount is the stored snapshot count. `getOpinion(id, a).suspectCount == suspectCount[id][a]`	✅	🎯🎯
		opinion.suspectExpiryTimestamp is the snapshot time plus the window current at read time.		

GS-UT-16	`utGsOpinionExpiryIsSnapshotPlusWindow`	`getOpinion(id, a).suspectExpiryTimestamp == suspectReportedAt[id][a] + suspectWindow`	✅	🎯
GS-UT-17	`utGsOpinionFrontierIsStored`	opinion.quorumFrontierTimestamp is the stored frontier. `getOpinion(id, a).quorumFrontierTimestamp == quorumFrontier`	✅	🎯
GS-UT-18	`utGsRemoveGuardianKeepsFlagMembership`	Flag membership moves only through flag, unflag, prune or unban, never a rotation. `membership of w in flaggedBy[id][a] changed => f in {flag, unflag, prune, unban}`	✅	🎯
GS-UT-19	`utGsRemoveGuardianKeepsSuspectMembership`	Suspect membership moves only through suspect, unsuspect or prune, never a rotation. `membership of w in suspectedBy[id][a] changed => f in {suspect, unsuspect, prune}`	✅	🎯
GS-UT-20	`utGsRemoveGuardianKeepsBanLatch`	Removing a guardian moves neither an account's stored ban nor the quorum those bans are scored against. `removeGuardian(g): banned[id][a]' == banned[id][a] AND quorum' == quorum`	✅	🎯
GS-UT-21	`utGsRemoveGuardianKeepsHeartbeats`	Removing a guardian keeps every heartbeat record: an ex-guardian's stamp survives. `removeGuardian(g): heartbeats[w]' == heartbeats[w]`	✅	🎯
GS-UT-22	`utGsRemoveGuardianKeepsFrontier`	Removing a guardian never moves the quorum frontier (removal does not sync). `removeGuardian(g): quorumFrontier' == quorumFrontier`	✅	🎯
GS-UT-23	`utGsAddGuardianKeepsSuspectSnapshot`	Only a suspect report or its retraction re-snapshots the suspect count and its timestamp; no committee change or prune does. `(suspectCount[id][a], suspectReportedAt[id][a]) changed => f in {suspect, unsuspect}`	✅	🎯
GS-UT-24	`utGsAddGuardianKeepsHeartbeats`	Adding a guardian keeps every heartbeat record: a re-added guardian inherits its old stamp. `addGuardian(g): heartbeats[w]' == heartbeats[w]`	✅	🎯
GS-UT-25	`utGsAddGuardianKeepsFrontier`	Adding a guardian never moves the quorum frontier (joining does not sync). `addGuardian(g): quorumFrontier' == quorumFrontier`	✅	🎯
GS-UT-26	`utGsFlagInsertsCaller`	Flag inserts exactly the caller (msg.sender, not the flagged account) into the flagger set. `flag(id, [a]): msg.sender in flaggedBy[id][a]'`	✅	🎯
GS-UT-27	`utGsUnflagRemovesCallerFlag`	Unflag removes the caller's flag on the named account. `unflag(id, [a]): msg.sender not in flaggedBy[id][a]'`	✅	🎯
GS-UT-28	`utGsUnflagKeepsOtherFlaggers`	Unflag moves no other guardian's flag on the same account. `w != msg.sender => unflag(id, [a]): (w in flaggedBy[id][a]') == (w in flaggedBy[id][a])`	✅	🎯
GS-UT-29	`utGsUnflagDropsExactlyCallerStrike`	Unflag drops the strike count by exactly the retracted flag, never more, never less. `unflag(id, [a]): flaggedByLen[id][a]' == flaggedByLen[id][a] - (msg.sender in flaggedBy[id][a] ? 1 : 0)`	✅	🎯
GS-UT-30	`utGsSuspectInsertsCaller`	Suspect inserts exactly the caller into the reported account's suspect set. `suspect(id, [a]): msg.sender in suspectedBy[id][a]'`	✅	🎯

GS-UT-31	`utGsSuspectStampsCallerNow`	Suspect stamps the caller's report at now, refreshing even a re-report. `suspect(id, [a]): suspectedAt[id][a][msg.sender]' == block.timestamp`	✅	🎯
GS-UT-32	`utGsSuspectLeavesSiblingSets`	Suspecting one account leaves the reporters standing against every other account and market untouched. `(id2, b) != (id, a) => suspect(id, [a]): suspectedByLen[id2][b]' == suspectedByLen[id2][b]`	✅	🎯
GS-UT-33	`utGsUnsuspectRemovesCallerReport`	Unsuspect removes the caller's report on the named account, live, expired or never made. `unsuspect(id, [a]): msg.sender not in suspectedBy[id][a]'`	✅	🎯
GS-UT-34	`utGsUnsuspectClearsCallerStamp`	Unsuspect zeroes the caller's suspect stamp; a caller with no report starts at zero. `unsuspect(id, [a]): suspectedAt[id][a][msg.sender]' == 0`	✅	🎯
GS-UT-35	`utGsPruneKeepsLiveGuardianFlag`	Prune evicts only ex-guardians: a live guardian's flag survives. `g in guardians AND g in flaggedBy[id][a] => prune(id, a): g in flaggedBy[id][a]'`	✅	🎯
GS-UT-36	`utGsPruneKeepsLiveGuardianSuspect`	Prune keeps a live guardian's suspect report, expired or not; it goes by membership only. `g in guardians AND g in suspectedBy[id][a] => prune(id, a): g in suspectedBy[id][a]'`	✅	🎯
GS-UT-37	`utGsHeartbeatsRowPerMember`	The heartbeat listing returns one aligned row per committee member. `getHeartbeats(): guardians.length == committeeLen AND timestamps.length == guardians.length`	✅	🎯
GS-UT-38	`utGsHeartbeatsValueMatchesRecord`	Each returned timestamp is exactly the matching guardian's recorded heartbeat. `getHeartbeats(): i < guardians.length => timestamps[i] == heartbeats[guardians[i]]`	✅	🎯
GS-UT-39	`utGsFlagNeverShrinksFlaggers`	Flag never shrinks any flagger set; strikes only accumulate through it. `flag(id, accounts): flaggedByLen[id2][b]' >= flaggedByLen[id2][b]`	✅	🎯
GS-UT-40	`utGsUnflagNeverGrowsFlaggers`	Unflag never grows any flagger set; a retraction cannot add strikes. `unflag(id, accounts): flaggedByLen[id2][b]' <= flaggedByLen[id2][b]`	✅	🎯
GS-UT-41	`utGsUnsuspectNeverGrowsSuspectSet`	Unsuspect never grows any suspect set; it only retracts and sweeps. `unsuspect(id, accounts): suspectedByLen[id2][b]' <= suspectedByLen[id2][b]`	✅	🎯
GS-UT-42	`utGsPruneShrinkOnly`	Prune only ever shrinks a set of flags or suspect reports, on any account in any market. `prune(id, a): flaggedByLen[id2][b]' <= flaggedByLen[id2][b] AND suspectedByLen[id2][b]' <= suspectedByLen[id2][b]`	✅	🎯
GS-UT-43	`utGsFlagStampsHeartbeat`	Flag stamps the caller's implicit heartbeat, exactly as heartbeat() would. `flag(id, accounts, blk): heartbeats[msg.sender]' == blk.timestamp`	✅	🎯
GS-UT-44	`utGsUnflagStampsHeartbeat`	Unflag stamps the caller's implicit heartbeat, exactly as heartbeat() would. `unflag(id, accounts, blk): heartbeats[msg.sender]' == blk.timestamp`	✅	🎯
		Suspect stamps the caller's implicit heartbeat, exactly as heartbeat() would.		

GS-UT-45	`utGsSuspectStampsHeartbeat`	`suspect(id, accounts, blk): heartbeats[msg.sender]' == blk.timestamp`	✅	🎯
GS-UT-46	`utGsUnsuspectStampsHeartbeat`	Unsuspect stamps the caller's implicit heartbeat, exactly as heartbeat() would. `unsuspect(id, accounts, blk): heartbeats[msg.sender]' == blk.timestamp`	✅	🎯
GS-UT-47	`utGsGetQuorumIsStored`	The quorum getter returns the stored ban quorum, not a field packed beside it. `getQuorum() == quorum`	✅	🎯
GS-UT-48	`utGsGetSuspectWindowIsStored`	The suspect-window getter returns the stored window. `getSuspectWindow() == suspectWindow`	✅	🎯
GS-UT-49	`utGsGetQuorumFrontierIsStored`	The quorum-frontier getter returns the stored frontier. `getQuorumFrontier() == quorumFrontier`	✅	🎯
GS-UT-50	`utGsGetGuardianCountIsCommitteeSize`	The guardian-count getter returns the size of the committee. `getGuardianCount() == committeeLen`	✅	🎯
GS-UT-51	`utGsGetHeartbeatIsGuardianStamp`	The heartbeat getter answers for the named guardian, not the caller. `getHeartbeat(g) == heartbeats[g]`	✅	🎯
GS-UT-52	`utGsIsGuardianIsMembership`	The guardian check answers for the named account, not the caller. `isGuardian(a) == (a in guardians)`	✅	🎯
GS-UT-53	`utGsIsBannedIsEffectiveBan`	The ban check is the effective ban: the stored latch, or a flag count that has reached quorum. `isBanned(id, a) == (banned[id][a] OR flaggedByLen[id][a] >= quorum)`	✅	🎯🎯🎯
GS-UT-54	`utGsFlaggedByRowPerFlagger`	The flagger listing returns one row per live flagger of the named account. `getFlaggedBy(id, a).length == flaggedByLen[id][a]`	✅	🎯
GS-UT-55	`utGsSuspectedByRowPerReporter`	The suspect listing returns one aligned row per live reporter of the named account. `getSuspectedBy(id, a): guardians.length == suspectedByLen[id][a] AND reportedAt.length == guardians.length`	✅	🎯
GS-UT-56	`utGsSuspectedByStampMatchesRecord`	Each returned report time is exactly the matching guardian's recorded suspect stamp. `getSuspectedBy(id, a): i < guardians.length => reportedAt[i] == suspectedAt[id][a][guardians[i]]`	✅	🎯
GS-UT-57	`syncBelowQuorumLeavesFrontierUntouched`	A committee that cannot make quorum leaves the frontier where it was. `syncQuorumBlock(): _quorum == 0 OR guardians.length < _quorum => _quorumFrontierTimestamp' == _quorumFrontierTimestamp`	✅	🎯
GS-UT-58	`frontierAdvanceIsBackedByQuorum`	An advance of the frontier is backed by a quorum that has scanned up to the new value. `syncQuorumBlock(): _quorumFrontierTimestamp' > _quorumFrontierTimestamp => count(g in guardians: _heartbeats[g] >= _quorumFrontierTimestamp') >= _quorum`	✅	🎯
GS-UT-59	`frontierAdvanceIsNotAboveQuorumthHighest`	No quorum is strictly ahead of a freshly set frontier, so the sync picks the quorum-th highest. `syncQuorumBlock(): _quorumFrontierTimestamp' > _quorumFrontierTimestamp => count(g in guardians: _heartbeats[g] > _quorumFrontierTimestamp') < _quorum`	✅	🎯
GS-UT-60	`syncKeepsFrontierOnlyWhenItAlreadyDominatesTheCommittee`	A sync that leaves the frontier put found no quorum past it: the leg GS-UT-59 misses. `syncQuorumBlock(): _quorum > 0 AND guardians.length >= _quorum AND`	✅	🎯

		`_quorumFrontierTimestamp' == _quorumFrontierTimestamp => count(g in guardians: _heartbeats[g] > _quorumFrontierTimestamp) < _quorum`		
GS-UT-61	`utGsSetQuorumKeepsFrontier`	Setting the quorum never moves the quorum frontier (a config change does not sync). `setQuorum(q): quorumFrontier' == quorumFrontier`	✅	🎯

Reachability

Identification	Name	Description	Properties	Mutations
GS-RC-01	`rcGsAddGuardianIsReachable`	The manager adds a fresh guardian to the committee. `satisfy(guardianCount' > guardianCount AND isGuardian(guardian)')`	✅	🎯
GS-RC-02	`rcGsRemoveGuardianIsReachable`	The manager removes a guardian while staying at or above quorum. `satisfy(guardianCount' < guardianCount AND !isGuardian(guardian)')`	✅	🎯
GS-RC-03	`rcGsSetQuorumIsReachable`	The manager moves the ban quorum to a fresh value. `satisfy(quorum' == newQuorum AND quorum' != quorum)`	✅	🎯
GS-RC-04	`rcGsSetSuspectWindowIsReachable`	The manager moves the suspect window to a fresh value. `satisfy(suspectWindow' == newWindow AND suspectWindow' != suspectWindow)`	✅	🎯
GS-RC-05	`rcGsHeartbeatIsReachable`	A committee guardian reports a heartbeat. `satisfy(heartbeats[msg.sender]' == blk.timestamp AND heartbeats[msg.sender]' > heartbeats[msg.sender])`	✅	🎯
GS-RC-06	`rcGsFlagIsReachable`	A guardian flags an account, joining its flagger set. `satisfy(flaggedByLen[id][a]' > flaggedByLen[id][a] AND msg.sender in flaggedBy[id][a]')`	✅	🎯
GS-RC-07	`rcGsUnflagIsReachable`	A guardian retracts its flag, leaving the flagger set. `satisfy(flaggedByLen[id][a]' < flaggedByLen[id][a] AND !(msg.sender in flaggedBy[id][a]'))`	✅	🎯
GS-RC-08	`rcGsUnsuspectIsReachable`	A guardian retracts its suspect report, clearing its stamp. `satisfy(suspectedByLen[id][a]' < suspectedByLen[id][a] AND suspectedAt[id][a][msg.sender]' == 0)`	✅	🎯
GS-RC-09	`rcGsPruneIsReachable`	Permissionless prune drops an ex-guardian's stale flag. `satisfy(!(g in flaggedBy[id][a]') AND flaggedByLen[id][a]' < flaggedByLen[id][a])`	✅	🎯
GS-RC-10	`rcGsBanIsReachable`	An access limiter force-latches a ban. `satisfy(!banned[id][a] AND banned[id][a]')`	✅	🎯
GS-RC-11	`rcGsRemoveGuardianDownToQuorumIsReachable`	A guardian can be removed right down to the quorum floor. `satisfy(guardianCount' == quorum AND !isGuardian(guardian)') (pre: guardianCount == quorum + 1)`	✅	🎯
GS-RC-12	`rcGsSetSuspectWindowToMaxIsReachable`	The suspect window reaches its maximal allowed value. `satisfy(suspectWindow' == MAX_SUSPECT_WINDOW)`	✅	🎯
GS-RC-13	`rcGsFlagLatchesBanAtQuorumIsReachable`	A quorum-crossing flag latches the ban. `satisfy(banned[id][a]' AND flaggedByLen[id][a]' == quorum) (pre: !banned, flaggedByLen == quorum - 1)`	✅	🎯
GS-RC-14	`rcGsUnbanClearsFlaggersIsReachable`	An unban lifts the latch and clears the flagger set together. `satisfy(!banned[id][a]' AND flaggedByLen[id][a]' == 0)`	✅	🎯
GS-RC-15	`rcGsUnflagEmptiesSetIsReachable`	The last flag retraction empties the flagger set. `satisfy(flaggedByLen[id][a]' == 0) (pre: msg.sender is the sole flagger)`	✅	🎯
GS-RC-16	`rcGsPruneDropsStaleSuspectIsReachable`	Permissionless prune drops an ex-guardian's stale suspect report. `satisfy(!(g in suspectedBy[id][a]') AND suspectedAt[id][a][g]' == 0)`	✅	🎯
GS-RC-17	`rcGsSecondDistinctFlaggerIsReachable`	A second distinct guardian joins the flagger set. `satisfy(flaggedByLen[id][a]' == flaggedByLen[id][a] + 1 AND msg.sender in flaggedBy[id][a]')`	✅	🎯

GS-RC-18	`rcGsSecondDistinctSuspecterIsReachable`	A second distinct guardian joins the suspect set. `satisfy(suspectedByLen[id][a]' == suspectedByLen[id][a] + 1 AND suspectedAt[id][a][msg.sender]' == block.timestamp)`	✅	🎯
GS-RC-19	`rcGsAddThenIsGuardianIsReachable`	A freshly added guardian reads back as a committee member. `addGuardian(g): satisfy(isGuardian(g))`	✅	🎯
GS-RC-20	`rcGsAddThenHeartbeatIsReachable`	A freshly added guardian immediately reports a heartbeat. `addGuardian(g); heartbeat(blk) from g: satisfy(heartbeats[g]' == blk.timestamp AND heartbeats[g]' > 0)`	✅	🎯
GS-RC-21	`rcGsFlagThenUnflagRoundTripIsReachable`	A flag followed by its retraction returns the flagger set to baseline. `flag(id,[a]); unflag(id,[a]): satisfy(len_mid == len + 1 AND len' == len), len = flaggedByLen`	✅	🎯
GS-RC-22	`rcGsQuorumLoweringRetroBansIsReachable`	A quorum lowering retro-latches an effective ban with no new flag. `setQuorum(q): satisfy(!isBanned(id, a) AND isBanned(id, a)')`	✅	🎯
GS-RC-23	`rcGsQuorumRaiseUnbansUnlatchedIsReachable`	A quorum raise silently drops an effective-but-never-latched ban. `setQuorum(q): satisfy(isBanned(id, a) AND !isBanned(id, a)')`	✅	🎯
GS-RC-24	`rcGsUnflagDropsEffectiveBanIsReachable`	A plain unflag ends a never-latched effective ban without the restorer role. `unflag(id, [a]): satisfy(isBanned(id, a) AND !isBanned(id, a)')`	✅	🎯
GS-RC-25	`rcGsOpinionPopulatedIsReachable`	A populated opinion with live unexpired suspects is observable. `getOpinion(id,a): satisfy(strikeCount > 0 AND suspectCount > 0 AND suspectExpiryTimestamp > block.timestamp)`	✅	🎯
GS-RC-26	`rcGsDynamicBanWithoutLatchIsReachable`	An effective ban holds from quorum flags alone with no stored latch. `satisfy(isBanned(id, a) AND !banned[id][a])`	✅	🎯
GS-RC-27	`rcGsSuspectThenUnsuspectRoundTripIsReachable`	A suspect report followed by its retraction returns the suspect set to baseline. `suspect(id,[a]); unsuspect(id,[a]): satisfy(len_mid == len + 1 AND len' == len), len = suspectedByLen`	✅	🎯
GS-RC-28	`rcGsRelatchAfterUnbanIsReachable`	After an unban the committee can re-flag the account straight back into a latched ban. `unban(id,a); flag(id,[a]): satisfy(banned[id][a]')`	✅	🎯
GS-RC-29	`rcGsWindowCutRetroExpiresIsReachable`	A window cut alone retro-expires a live suspect snapshot, no suspect call needed. `setSuspectWindow(w): satisfy(suspectCount > 0 AND suspectCount' == suspectCount AND suspectExpiryTimestamp > block.timestamp AND suspectExpiryTimestamp' <= block.timestamp)`	✅	🎯
GS-RC-30	`quorumRaiseStaleFrontierWitness`	The raised quorum can inherit a frontier only the old quorum backed. `setQuorum(q'); syncQuorumBlock(): satisfy(q' > q AND frontier > 0 AND frontier'' == frontier AND count(g in guardians: _heartbeats[g] >= frontier) < q')`	✅	🎯
GS-RC-31	`setQuorumStaleFrontierWitness`	Right after setQuorum a full quorum can already sit strictly past the stored frontier. `setQuorum(q'): satisfy(count(g in guardians: _heartbeats[g] > _quorumFrontierTimestamp) >= _quorum)`	✅	🎯
GS-RC-32	`opinionFrontierStaleUntilSync`	A same-block permissionless sync can strictly advance the frontier getOpinion just reported. `getOpinion(id, a); syncQuorumBlock(); getOpinion(id, a): satisfy(opinion''.quorumFrontierTimestamp > opinion.quorumFrontierTimestamp)`	✅	🎯
GS-RC-33	`staleHeartbeatFeedsFrontierAfterReAdd`	A removed-then-re-added guardian's retained heartbeat can feed the next frontier sync. `removeGuardian(g); addGuardian(g); syncQuorumBlock(): satisfy(frontier''' == heartbeats[g] AND heartbeats[g] > frontier AND heartbeats[h] < heartbeats[g])`	✅	🎯

GS-RC-34	`rcGsStaleSuspectCountIsReachable`	getOpinion serves a stale suspect over-count; two reports charged, only one individually live. `suspect(id,[a]) by g1 at t1; suspect(id,[a]) by g2 at t2; getOpinion(id,a) at t3: satisfy(suspectCount == 2 AND suspectedAt[id][a][g1] + suspectWindow < t3 AND suspectExpiryTimestamp >= t3)`	✅	🎯
GS-RC-35	`rcGsStaleFlagCompletesQuorumIsReachable`	A single live flag latches the ban with an ex-guardian's stale flag counted inside the quorum. `g in flaggedBy[id][a], !isGuardian(g), !banned[id][a], flaggedByLen[id][a] == quorum - 1: flag(id, [a]): satisfy(banned[id][a]' AND g in flaggedBy[id][a]' AND !isGuardian(g)')`	✅	🎯
GS-RC-36	`rcGsBanQuorumMetByExGuardiansIsReachable`	An effective ban rests entirely on votes from guardians who have all left the committee. `satisfy(isBanned(id, a)) with !banned[id][a], flaggedBy[id][a] == {g1, g2}, quorum == 2, !isGuardian(g1), !isGuardian(g2)`	✅	🎯
GS-RC-37	`rcGsHeartbeatUnboundTimestampIsReachable`	A heartbeat pairing an aged attested block with a claimed now-timestamp is accepted verbatim. `heartbeat(blk): satisfy(blk.number + 100 <= block.number <= blk.number + 256 AND blk.timestamp == block.timestamp AND heartbeats[msg.sender]' == blk.timestamp)`	✅	🎯
GS-RC-38	`rcGsReAddResumesStaleHeartbeatIsReachable`	A guardian removed and later re-added resumes with its nonzero pre-removal heartbeat. `removeGuardian(g); addGuardian(g): satisfy(isGuardian(g)' AND heartbeats[g]' == heartbeats[g] > 0)`	✅	🎯
GS-RC-39	`rcGsPruneLeavesStaleSuspectCountIsReachable`	Prune leaves the cached suspect count charging after the raw report is gone. `suspectedBy[id][a] == {g}, !isGuardian(g), suspectCount[id][a] == 1: prune(id, a): satisfy(suspectedByLen[id][a]' == 0 AND opinion.suspectCount' == 1 AND opinion.suspectExpiryTimestamp' > block.timestamp)`	✅	🎯
GS-RC-40	`rcGsRetractionExtendsExpiryIsReachable`	Retracting one of two live suspect reports strictly extends the survivor's surcharge expiry. `suspectedBy[id][a] == {msg.sender, g}: unsuspect(id,[a]): satisfy(suspectedByLen[id][a]' < suspectedByLen[id][a] AND suspectCount[id][a]' > 0 AND suspectReportedAt[id][a]' + suspectWindow > suspectReportedAt[id][a] + suspectWindow)`	✅	🎯

Reverts

Identification	Name	Description	Properties	Mutations
GS-RV-01	`rvGsAddGuardianRevertsIff`	addGuardian reverts iff the guardian is zero or already a member. `hasRole(MANAGER_ROLE, msg.sender) AND committeeLen < MAX_GUARDIANS_COUNT: addGuardian(guardian) REVERTS <=> (guardian == 0 OR isGuardian(guardian))`	✅	🎯🎯
GS-RV-02	`rvGsRemoveGuardianRevertsIff`	removeGuardian reverts iff the target is absent or removal breaks the quorum. `hasRole(MANAGER_ROLE, msg.sender): removeGuardian(guardian) REVERTS <=> (!isGuardian(guardian) OR committeeLen - 1 < quorum)`	✅	🎯
GS-RV-03	`rvGsSetQuorumRevertsIff`	setQuorum reverts iff the new quorum is zero or above the committee size. `hasRole(MANAGER_ROLE, msg.sender): setQuorum(newQuorum) REVERTS <=> (newQuorum == 0 OR newQuorum > committeeLen)`	✅	🎯🎯
GS-RV-04	`rvGsSetSuspectWindowRevertsIff`	setSuspectWindow reverts iff the window exceeds the 30-day cap. `hasRole(MANAGER_ROLE, msg.sender):`	✅	🎯

		`setSuspectWindow(suspectWindow) REVERTS <=> suspectWindow > MAX_SUSPECT_WINDOW`		
GS-RV-05	`rvGsHeartbeatRevertsWhenNotGuardian`	Only a guardian can heartbeat. `!isGuardian(msg.sender) => heartbeat(blk) REVERTS`	✅	🎯
GS-RV-06	`rvGsHeartbeatRevertsOnFutureBlockNumber`	A heartbeat reporting a block that does not exist yet reverts. `blk.number > block.number => REVERTS`	✅	🎯
GS-RV-07	`rvGsHeartbeatRevertsOnFutureTimestamp`	A heartbeat reporting a timestamp past the current block reverts. `blk.timestamp > block.timestamp => REVERTS`	✅	🎯
GS-RV-08	`rvGsHeartbeatRevertsOnRegressedTimestamp`	A heartbeat backdating the guardian's own last-scanned time reverts. `blk.timestamp < heartbeats[msg.sender] => REVERTS`	✅	🎯
GS-RV-09	`rvGsHeartbeatRevertsOnZeroBlockHash`	A heartbeat reporting a zero block hash reverts. `blk.hash == 0 => REVERTS`	✅	🎯
GS-RV-10	`rvGsFlagRevertsWhenNotGuardian`	Only a guardian can flag. `!isGuardian(msg.sender) => flag(id, accounts, blk) REVERTS`	✅	🎯
GS-RV-11	`rvGsFlagRevertsOnZeroAccount`	Flagging the zero address reverts. `accounts.length == 1 AND accounts[0] == 0 => REVERTS`	✅	🎯
GS-RV-12	`rvGsUnflagRevertsWhenNotGuardian`	Only a guardian can unflag. `!isGuardian(msg.sender) => unflag(id, accounts, blk) REVERTS`	✅	🎯
GS-RV-13	`rvGsSuspectRevertsWhenNotGuardian`	Only a guardian can suspect. `!isGuardian(msg.sender) => suspect(id, accounts, blk) REVERTS`	✅	🎯
GS-RV-14	`rvGsSuspectRevertsOnZeroAccount`	Suspecting the zero address reverts. `accounts.length == 1 AND accounts[0] == 0 => REVERTS`	✅	🎯
GS-RV-15	`rvGsUnsuspectRevertsWhenNotGuardian`	Only a guardian can unsuspect. `!isGuardian(msg.sender) => unsuspect(id, accounts, blk) REVERTS`	✅	🎯
GS-RV-16	`rvGsPruneNeverReverts`	Prune never reverts: permissionless cleanup is a no-op when nothing is stale. `prune(id, account): never REVERTS`	✅	🎯
GS-RV-17	`rvGsBanRevertsIff`	Ban reverts iff the account is the zero address. `hasRole(ACCESS_LIMITER_ROLE, msg.sender): ban(id, account) REVERTS <=> account == 0`	✅	🎯
GS-RV-18	`rvGsUnbanNeverReverts`	Unban never reverts on the authorized path (no parameter or state guard). `hasRole(ACCESS_RESTORER_ROLE, msg.sender): unban(id, account) never REVERTS`	✅	🎯
GS-RV-19	`rvGsGetQuorumNeverReverts`	getQuorum never reverts. `getQuorum(): never REVERTS`	✅	🎯
GS-RV-20	`rvGsGetSuspectWindowNeverReverts`	getSuspectWindow never reverts. `getSuspectWindow(): never REVERTS`	✅	🎯
GS-RV-21	`rvGsIsGuardianNeverReverts`	isGuardian never reverts. `isGuardian(account): never REVERTS`	✅	🎯
GS-RV-22	`rvGsGetGuardianCountNeverReverts`	getGuardianCount never reverts. `getGuardianCount(): never REVERTS`	✅	🎯
GS-RV-23	`rvGsGetGuardiansNeverReverts`	getGuardians never reverts. `getGuardians(): never REVERTS`	✅	🎯

GS-RV-24	`rvGsGetHeartbeatNeverReverts`	getHeartbeat never reverts. `getHeartbeat(guardian): never REVERTS`	✅	🎯
GS-RV-25	`rvGsGetHeartbeatsNeverReverts`	getHeartbeats never reverts. `getHeartbeats(): never REVERTS`	✅	🎯
GS-RV-26	`rvGsGetQuorumFrontierNeverReverts`	getQuorumFrontier never reverts. `getQuorumFrontier(): never REVERTS`	✅	🎯
GS-RV-27	`rvGsGetOpinionNeverReverts`	getOpinion never reverts. `getOpinion(id, account): never REVERTS`	✅	🎯
GS-RV-28	`rvGsIsBannedNeverReverts`	isBanned never reverts. `isBanned(id, account): never REVERTS`	✅	🎯
GS-RV-29	`rvGsGetFlaggedByNeverReverts`	getFlaggedBy never reverts. `getFlaggedBy(id, account): never REVERTS`	✅	🎯
GS-RV-30	`rvGsGetSuspectedByNeverReverts`	getSuspectedBy never reverts. `getSuspectedBy(id, account): never REVERTS`	✅	🎯
GS-RV-31	`rvGsHeartbeatRevertsOnAgedOutBlock`	A heartbeat reporting a block outside the 256-block blockhash window reverts. `(blk.number + 256 < block.number OR blk.number >= block.number) => REVERTS`	✅	🎯

Access Control

Identification	Name	Description	Properties	Mutations
GS-AC-01	`acGsOnlyGuardianGrowsFlaggerSet`	A flagger set grows only for a committee-guardian caller. `flaggedByLen'[id][a] > flaggedByLen[id][a] => committeePositions[msg.sender] != 0`	✅	🎯🎯
GS-AC-02	`acGsOnlyGuardianGrowsSuspectSet`	A suspect set grows only for a committee-guardian caller. `suspectedByLen'[id][a] > suspectedByLen[id][a] => committeePositions[msg.sender] != 0`	✅	🎯🎯
GS-AC-03	`acGsOnlyManagerMovesQuorum`	Only a manager can change the quorum. `quorum' != quorum => hasRole(MANAGER_ROLE, msg.sender)`	✅	🎯
GS-AC-04	`acGsOnlyManagerMovesSuspectWindow`	Only a manager can change how long a suspect report stays in force. `suspectWindow' != suspectWindow => hasRole(MANAGER_ROLE, msg.sender)`	✅	🎯
GS-AC-05	`acGsBanLatchedOnlyByLimiterOrFlagger`	An account is banned only by the access limiter or by a guardian that has flagged it. `!banned[id][a] AND banned[id][a]' => hasRole(ACCESS_LIMITER_ROLE, msg.sender) OR flaggedBy[id][a].contains(msg.sender)'`	✅	🎯
GS-AC-06	`acGsBanLiftedOnlyByRestorer`	Only the access restorer can lift a ban. `banned[id][a] AND !banned[id][a]' => hasRole(ACCESS_RESTORER_ROLE, msg.sender)`	✅	🎯
GS-AC-07	`acGsOnlyManagerMovesCommitteeMembership`	Only a manager can change who sits on the guardian committee. `guardians.position[g]' != guardians.position[g] => hasRole(MANAGER_ROLE, msg.sender)`	✅	🎯
GS-AC-08	`acGsOnlyGuardianMovesHeartbeat`	A guardian's last-scanned timestamp moves only when a sitting guardian calls in. `heartbeats[g]' != heartbeats[g] => isGuardian(msg.sender)`	✅	🎯🎯
		Only a sitting guardian can restamp an account's suspicion snapshot.		

GS-AC-09	`acGsOnlyGuardianRestampsSnapshotClock`	`suspectReportedAt[id][a]' != suspectReportedAt[id][a] => isGuardian(msg.sender)`	✅	🎯
GS-AC-10	`acGsOnlyGuardianMovesSuspectCount`	Only a sitting guardian can change the suspect count recorded against an account. `suspectCount[id][a]' != suspectCount[id][a] => isGuardian(msg.sender)`	✅	🎯
GS-AC-11	`acGsFlaggerEntryIsCallerOnly`	A guardian can raise a flag only under its own name, never another guardian's. `!flaggedBy[id][a].contains(g) AND flaggedBy[id][a].contains(g)' => g == msg.sender`	✅	🎯
GS-AC-12	`acGsSuspectEntryIsCallerOnly`	A guardian can file a suspect report only under its own name, never another guardian's. `!suspectedBy[id][a].contains(g) AND suspectedBy[id][a].contains(g)' => g == msg.sender`	✅	🎯
GS-AC-13	`acGsSuspectStampWrittenOnlyBySelf`	A suspect report's timestamp is set only by the guardian it belongs to; clearing it is open to anyone. `suspectedAt[id][a][g]' != suspectedAt[id][a][g] AND suspectedAt[id][a][g]' != 0 => g == msg.sender`	✅	🎯
GS-AC-14	`acGsOnlyGuardianWritesSuspectStamp`	A suspect report's timestamp is set only by a sitting guardian. `suspectedAt[id][a][g]' != suspectedAt[id][a][g] AND suspectedAt[id][a][g]' != 0 => isGuardian(msg.sender)`	✅	🎯
GS-AC-15	`roleGrantedOnlyByItsAdmin`	A role is handed out only by the holder of that role's admin role. `!hasRole[role][a] AND hasRole[role][a]' => hasRole[roleAdmin[role]][msg.sender]'`	✅	🎯
GS-AC-16	`roleRevokedOnlyByItsAdminOrItsHolder`	A role is taken away only by the holder of that role's admin role, or given up by its own holder. `hasRole[role][a] AND !hasRole[role][a]' => hasRole[roleAdmin[role]][msg.sender]' OR a == msg.sender`	✅	🎯
GS-AC-17	`acGsNoPermissionlessEffectiveUnban`	No unprivileged caller ends an effective ban. `caller unprivileged: isBanned(id, a) AND f(...) => isBanned(id, a)'`	❌ → ✅	Issue #8 (LOW)

LidoLendMarketController

- GuardianSignals linked as a CVL model
- Core totals read without pending interest, the CVL model of the core not accruing
- Settlement queue holding up to 7 borrowers
- Transient inflow counter zeroed at the start of every rule
- Loops running up to 3 iterations

Valid State

Identification	Name	Description	Properties	Mutations
MC-VS-01	`heapIndexToSlot`	Every borrower in the settlement queue is indexed at exactly the place they occupy. `1 <= i < nodes.length => slot[nodes[i].account] == i`	✅	🎯

MC-VS-02	`heapSlotToAccount`	The place recorded for a queued borrower really holds that borrower. `slot[a] != 0 => nodes[slot[a]].account == a`	✅	🎯
MC-VS-03	`heapSlotBounded`	A queued borrower's recorded place always lies inside the queue. `slot[a] != 0 => slot[a] < nodes.length`	✅	🎯
MC-VS-04	`heapTailKeyZero`	Nothing outside the live part of the settlement queue carries a leftover risk key. `i == 0 OR i >= nodes.length => nodes[i].key == 0`	✅	🎯
MC-VS-05	`heapParentKeyDominatesChild`	The settlement queue is correctly ordered by the risk key it stores. `1 <= p AND c < nodes.length AND c in {2p, 2p+1} => nodes[p].key >= nodes[c].key`	✅	🎯🎯
MC-VS-06	`activatingRequiresTimerStart`	Timed collateral always has a start time recorded. `activating[b] != 0 => activatingStart[b] != 0`	✅	🎯🎯
MC-VS-07	`clearedTimerClearsDelay`	A cleared activation timer leaves no stale wait behind. `activatingStart[b] == 0 => activatingDelay[b] == 0`	✅	🎯
MC-VS-08	`timerStartNotInFuture`	An activation timer never starts in the future. `activatingStart[b] <= block.timestamp`	✅	🎯
MC-VS-09	`inflowSnapshotNotInFuture`	The inflow throttle is never stamped in the future. `inflowUpdatedAt <= block.timestamp`	✅	🎯
MC-VS-10	`activatingDelayCapped`	The activation wait set at deposit time never exceeds seven days. `activatingDelay[b] <= MAX_DELAY_DURATION`	✅	🎯
MC-VS-11	`minDelayLeMaxDelay`	The activation wait floor never exceeds its cap. `minActivationDelay <= maxActivationDelay`	✅	🎯
MC-VS-12	`maxActivationDelayCapped`	The activation wait cap never exceeds seven days. `maxActivationDelay <= MAX_DELAY_DURATION`	✅	🎯
MC-VS-13	`activatingLockDurationCapped`	The configured deposit lock window never exceeds seven days. `activatingLockDuration <= MAX_DELAY_DURATION`	✅	🎯

MC-VS-14	`quorumDelayZeroOrAboveMaxDelay`	A nonzero guardian-quorum fallback always exceeds the activation wait cap. `maxGuardiansQuorumDelay == 0 OR maxGuardiansQuorumDelay > maxActivationDelay`	✅	🎯
MC-VS-15	`delayPerStrikeCapped`	A single guardian strike never adds more than seven days to a borrower's wait. `delayPerStrike <= MAX_DELAY_DURATION`	✅	🎯
MC-VS-16	`delayPerSuspectCapped`	A single suspect report never adds more than seven days to a borrower's wait. `delayPerSuspect <= MAX_DELAY_DURATION`	✅	🎯
MC-VS-17	`sliceFloorBpsCapped`	The settlement policy's floor share of a debt never exceeds 100%. `sliceFloorBps <= BPS`	✅	🎯
MC-VS-18	`ltvStepBpsCapped`	The settlement policy's LTV step never exceeds 100%. `ltvStepBps <= BPS`	✅	🎯
MC-VS-19	`settlementLtvBelowWad`	A borrower's settle-down LTV target always sits strictly below 100%. `settlementLtv[b] < WAD`	✅	🎯
MC-VS-20	`timerStartRequiresActivating`	A running activation timer always has collateral in it. `activatingStart[b] != 0 => activating[b] != 0`	✅	🎯
MC-VS-21	`timerStartRequiresDelay`	A running activation timer always carries a nonzero wait. `activatingStart[b] != 0 => activatingDelay[b] != 0 (minActivationDelay != 0)`	✅	🎯

Variable Transitions

Identification	Name	Description	Properties	Mutations
MC-VT-01	`vtMcMarketIdFrozen`	Once initialized, the controller can never be repointed to a different market. `marketId' == marketId`	✅	🎯
MC-VT-02	`vtMcGuardianRegistryFrozen`	Once initialized, the guardian registry can never be swapped out. `guardianSignals' == guardianSignals`	✅	🎯
MC-VT-03	`vtMcHeapLengthStepsByOne`	The settlement queue gains or loses at most one borrower per call, and never drops its sentinel. `|nodes.length' - nodes.length| <= 1 AND (nodes.length >= 1 => nodes.length' >= 1)`	✅	🎯🎯
MC-VT-04	`vtMcHeapSentinelUntouched`	No call writes the settlement queue's reserved first slot. `nodes[0].account' == nodes[0].account AND nodes[0].key' == nodes[0].key`	✅	🎯
MC-VT-05	`vtMcInflowClockLandsAtNow`	The inflow throttle's decay clock is only ever stamped at now. `inflowUpdatedAt' == inflowUpdatedAt OR inflowUpdatedAt' == block.timestamp`	✅	🎯
MC-VT-06	`vtMcTimerStartClearsOrLandsAtNow`	A borrower's activation timer is only ever cleared or stamped at now. `activatingStart'[b] in {activatingStart[b], 0, block.timestamp}`	✅	🎯🎯
MC-VT-07	`vtMcResumeStampNeverPast`	The market's resume timestamp never moves into the past. `resumeSince' == resumeSince OR resumeSince' >= block.timestamp`	✅	🎯
MC-VT-08	`vtMcMinActivationDelayNeverZeroed`	No call zeroes the activation wait floor. `minActivationDelay != 0 => minActivationDelay' != 0`	✅	🎯
MC-VT-09	`vtMcMinActivationAmountNeverZeroed`	No call zeroes the activation threshold. `minActivationAmount != 0 => minActivationAmount' != 0`	✅	🎯
MC-VT-10	`vtMcMaxDelayInflowNeverZeroed`	No call zeroes the inflow reference the wait scales against. `maxDelayInflow != 0 => maxDelayInflow' != 0`	✅	🎯

State Transitions

Identification	Name	Description	Properties	Mutations
MC-ST-01	`stMcActivatingDeadlineMonotone`	Topping up collateral never moves a borrower's base unlock deadline earlier. `activating[b]' > activating[b] => activatingStart[b]' + activatingDelay[b]' >= activatingStart[b] + activatingDelay[b]`	✅	🎯
MC-ST-02	`stMcEnqueueGrowsHeap`	A borrower joining the settlement queue adds exactly one entry. `slot[a] == 0 AND slot[a]' != 0 => nodes.length' == nodes.length + 1`	✅	🎯
MC-ST-03	`stMcDequeueShrinksHeap`	A borrower leaving the settlement queue removes exactly one entry. `slot[a] != 0 AND slot[a]' == 0 => nodes.length' == nodes.length - 1`	✅	🎯
MC-ST-04	`stMcInflowStampsSnapshot`	A rise in the inflow throttle restarts its decay clock. `inflow' > inflow => inflowUpdatedAt' == block.timestamp`	✅	🎯
MC-ST-05	`stMcSettlementLtvSelfService`	Only the borrower can move their own settle-down LTV target. `settlementLtv'[b] != settlementLtv[b] => b == msg.sender`	✅	🎯
MC-ST-06	`stMcTimerEntryFeedsThrottle`	Collateral entering the activation timer feeds the inflow throttle, up to its ceiling. `d := activating[b]' - activating[b]; d > 0 => inflow' >= min(d, maxDelayInflow)`	✅	🎯

High Level

Identification	Name	Description	Properties	Mutations
MC-HL-01	`hlMcOnLeavesMinDelay`	Market traffic never moves the activation wait floor. `on(id, ev, data): minActivationDelay' == minActivationDelay`	✅	🎯
MC-HL-02	`hlMcOnLeavesMaxDelay`	Market traffic never moves the activation wait cap. `on(id, ev, data): maxActivationDelay' == maxActivationDelay`	✅	🎯
MC-HL-03	`hlMcOnLeavesSupplyCap`	Market traffic never moves the supply cap. `on(id, ev, data): supplyCap' == supplyCap`	✅	🎯
MC-HL-04	`hlMcOnLeavesSettlementLtv`	Market traffic never moves any borrower's settle-down LTV target. `on(id, ev, data): settlementLtv[b]' == settlementLtv[b]`	✅	🎯

MC-HL-05	`hlMcOnLeavesInstantActivationList`	Market traffic never grants or revokes instant activation. `on(id, ev, data): instantActivationCallers[c]' == instantActivationCallers[c]`	✅	🎯
MC-HL-06	`hlMcFreezeUnfreezeRoundTrip`	An unfreeze always undoes a freeze. `freeze(); unfreeze() => !isFrozen`	✅	🎯
MC-HL-07	`hlMcActivationDelayMonotoneBanded`	The base wait never falls as the deposit grows, and stays between the floor and the cap. `x <= y => minActivationDelay <= currentActivationDelay(x) <= currentActivationDelay(y) <= maxActivationDelay`	⏱️	🎯
MC-HL-08	`hlMcSupplyCollateralTouchesOneBorrower`	A collateral deposit moves at most one borrower's activation timer and dust. `on(id, SupplyCollateral, data), b1 != b2 => (dust, activating, activatingStart, activatingDelay) unchanged for b1, OR unchanged for b2`	✅	🎯
MC-HL-09	`hlMcSetActivationConfigLeavesPositions`	A new activation config never touches a borrower's un-matured collateral or its timer. `setCollateralActivationConfig(cfg): dust[b], activating[b], activatingStart[b], activatingDelay[b] all unchanged`	✅	🎯
MC-HL-10	`hlMcPauseResumeRoundTrip`	Resuming reopens the market at once and leaves the risk parameters alone. `pauseFor(d) at t1; resume() at t2: resumeSinceTimestamp' == t2 AND isFrozen, supplyCap, minActivationDelay, maxActivationDelay unchanged`	✅	🎯
MC-HL-11	`hlMcResumeLeavesFrozen`	Resuming a paused market never clears the freeze. `resume(): isFrozen' == isFrozen`	✅	🎯
MC-HL-12	`hlMcPauseForLeavesFrozen`	Pausing a market never sets or clears the freeze. `pauseFor(d): isFrozen' == isFrozen`	✅	🎯
MC-HL-13	`hlMcFreezeLeavesResumeTimestamp`	Freezing a market never changes when a pause ends. `freeze(): resumeSinceTimestamp' == resumeSinceTimestamp`	✅	🎯
MC-HL-14	`hlMcUnfreezeLeavesResumeTimestamp`	Unfreezing a market never changes when a pause ends. `unfreeze(): resumeSinceTimestamp' == resumeSinceTimestamp`	✅	🎯
MC-HL-15	`hlMcZeroPriceSizesNoSettlement`	A zero oracle price sizes every settlement to nothing. `price == 0 => settlementFor(b) == (0, 0, 0)`	✅	🎯
MC-HL-16	`hlMcMaxDepositOracleIndependent`	The remaining deposit room never moves with the oracle price. `maxDeposit() is unchanged by any change to price[oracle]`	✅	🎯
MC-HL-17	`hlMcMaxWithdrawableOracleIndependent`	A lender's withdrawable amount never moves with the oracle price. `maxWithdrawableAssets(s) is unchanged by any change to price[oracle]`	✅	🎯
MC-HL-18	`hlMcSettlementViewsAgreeSupply`	The next-to-settle and per-borrower views agree on the supply shares. `nextSettlementBorrower() == (b, s, bs, r) => settlementFor(b).supplyShares == s`	✅	🎯
MC-HL-19	`hlMcSettlementViewsAgreeBorrow`	The next-to-settle and per-borrower views agree on the borrow shares. `nextSettlementBorrower() == (b, s, bs, r) => settlementFor(b).borrowShares == bs`	✅	🎯
MC-HL-20	`hlMcSettlementViewsAgreeRemainder`	The next-to-settle and per-borrower views agree on the debt left after the cut. `nextSettlementBorrower() == (b, s, bs, r) => settlementFor(b).remainingBorrowShares == r`	✅	🎯
MC-HL-21	`hlMcInflowCapBreaksOnlyViaConfig`	Only a governance config change can leave the inflow throttle above its ceiling. `inflow <= maxDelayInflow AND inflow' > maxDelayInflow' => f == setCollateralActivationConfig`	✅	🎯

MC-HL-22	`hlMcBorrowMaturationGateExact`	Maturation on a borrow needs the full guardian-surcharged wait and an open quorum gate. `unlock = activatingStart[b] + activatingDelay[b] + strikeCount[b] * delayPerStrike + (activatingStart[b] < suspectExpiry[b] ? suspectCount[b] * delayPerSuspect : 0); on(id, Borrow, data): activating[b]' != activating[b] => activating[b]' == 0 AND now >= unlock AND (unlock <= quorumFrontier OR now >= activatingStart[b] + maxGuardiansQuorumDelay)`	✅	🎯🎯🎯
MC-HL-23	`hlMcFrozenOrBannedDefersMaturation`	A freeze or a ban defers maturation, leaving the deposit on its timer. `isFrozen OR banned[b]; on(id, SupplyCollateral, data): activating[b]' >= activating[b] AND (activatingStart[b] != 0 => activatingStart[b]' != 0)`	✅	🎯
MC-HL-24	`hlMcInstantActivationSkipsThrottle`	An instant-activation deposit clearing the threshold lands active, off the throttle. `instantActivationCallers[caller] AND activatingStart[b] == 0 AND assets + dust[b] >= minActivationAmount; on(id, SupplyCollateral, d): activating[b]' == activating[b] AND inflow' == inflow AND dust[b]' == 0`	✅	🎯
MC-HL-25	`hlMcNonGrantedDepositEntersActivating`	An ordinary caller's deposit clearing the threshold enters the timer, dust and all. `!instantActivationCallers[caller] AND activatingStart[b] == 0 AND assets + dust[b] >= minActivationAmount; on(id, SupplyCollateral, d): activating[b]' == assets + dust[b] AND dust[b]' == 0`	✅	🎯
MC-HL-26	`hlMcInflowDecayLaws`	With no new deposits the inflow throttle only decays, and the wait it quotes never grows. `t2 >= t1: currentInflow(t1) <= inflow AND currentInflow(t2) <= currentInflow(t1) AND currentActivationDelay(t2, x) <= currentActivationDelay(t1, x) AND (t2 - inflowUpdatedAt >= maxActivationDelay AND inflow <= maxDelayInflow => currentInflow(t2) == 0)`	⏱	🎯
MC-HL-27	`hlMcTopUpDeadlineIsMaxOfOldAndQuote`	A top-up unlocks at the later of the standing deadline and the one its own size earns. `start[b] == 0 OR now < start[b] + activatingDelay[b], on(id, SupplyCollateral, data) books inc for b: start[b]' + activatingDelay[b]' == max(start[b] + activatingDelay[b], now + currentActivationDelay(inc))`	✅	🎯
MC-HL-28	`hlMcWithdrawUnderLockKeepsFloor`	While the deposit lock holds, a withdrawal cannot take the locked portion or grow dust. `start[b] != 0, activatingLockAmount != 0, now < start[b] + activatingLockDuration, lot not yet due: on(id, WithdrawCollateral, data): activating[b]' >= min(activating[b], activatingLockAmount) AND dust[b]' <= dust[b]`	✅	🎯🎯
MC-HL-29	`hlMcSettlementSliceConservesDebt`	The allowed cut never exceeds the debt, and any cut plus what is left equals that debt. `settlementFor(b) == (s, bs, r): bs <= borrowShares[b] AND (bs == 0 => s == 0 AND r == 0) AND (bs != 0 => bs + r == borrowShares[b]) AND (collateral[b] == 0 => bs == 0)`	✅	🎯
MC-HL-30	`hlMcTimedPauseHoldsThroughWindow`	A timed pause holds for its whole window. `pauseFor(d) at t, d != max_uint256 OR t2 < MAX_BLOCK_TIMESTAMP: t <= t2 < t + d => isPaused(t2)`	✅	🎯
MC-HL-31	`hlMcTimedPauseExpiresWithoutResume`	A timed pause expires on its own, with no resume call. `pauseFor(d) at t: t2 >= t + d => !isPaused(t2)`	✅	🎯
		Once the lock has elapsed, a covering withdrawal clears the whole un-matured balance.		

MC-HL-32	`hlMcLockLiftsAfterDeadline`	`now >= start[b] + activatingLockDuration, assets >= activating[b] + dust[b]: on(id, WithdrawCollateral, d): activating[b]' == 0 AND dust[b]' == 0`	✅	🎯🎯
MC-HL-33	`hlMcVoluntaryDrainFollowsWaterfall`	A debt-free borrower's withdrawal takes dust first, then timed collateral above the lock. `debt-free b, lot not due, lock = in-force activatingLockAmount else 0, fromDust = min(dust[b], assets), rest = activating[b] - min(assets - fromDust, max(activating[b] - lock, 0)), folds = (lock == 0 AND 0 < rest < minActivationAmount): (dust[b]', activating[b]') == (folds ? (dust[b] - fromDust + rest, 0) : (dust[b] - fromDust, rest)) AND (activating[b]' == 0 => start[b]' == 0 AND activatingDelay[b]' == 0)`	✅	🎯
MC-HL-34	`hlMcRootSliceHonorsPolicyFloors`	The riskiest queued borrower's cut meets both policy floors and never exceeds their debt. `b at the queue root, byBps = debtAssets * sliceFloorBps / BPS, sizeable = debtAssets != 0 AND collateralValue != 0: slice <= debtAssets AND (sizeable => slice >= min(byBps, debtAssets) AND slice >= min(sliceFloorAssets, debtAssets)) AND (sliceFloorBps == 0 AND sliceFloorAssets == 0 AND ltvStepBps == 0 AND settlementLtv[b] == 0 => slice == 0)`	⏱	🎯
MC-HL-35	`hlMcSlicePartitionsByEligibility`	Other than the riskiest borrower, only an opt-in allows a cut, to that target or in full. `b != heap.nodes[1].account, (ltv, ltvSlice) = settlementLtv(b), (debt, slice) = positionOf(b), capped = min(ltvSlice, debt): (ltv == 0 => slice == 0) AND (ltv != 0 AND fullCutBelowCollateral == 0 => slice == capped) AND (slice == capped OR slice == debt)`	✅	🎯🎯
MC-HL-36	`hlMcSameTxSupplyWithdrawUndoesInflow`	A deposit fully withdrawn in the same transaction leaves the inflow throttle untouched. `ungranted caller, debt-free b with no standing lot AND no dust, assets >= minActivationAmount, activatingLockAmount == 0, inflow <= maxDelayInflow: on(SupplyCollateral, assets); on(WithdrawCollateral, assets) => inflow' == currentInflow() AND transientInflow[b] == 0`	✅	🎯
MC-HL-37	`hlMcDustDepositLeavesThrottle`	A deposit under the activation threshold never feeds the inflow throttle. `assets + dust[b] < minActivationAmount: on(id, SupplyCollateral, d): inflow' == inflow AND inflowUpdatedAt' == inflowUpdatedAt AND transientInflow[b] == 0`	✅	🎯
MC-HL-38	`hlMcSupplyGateIgnoresTheClock`	Outside a pause, a lender's deposit fares the same at any time and enters no timer. `one caller, t1 AND t2 both >= resumeSince: on(t1, Supply, d) REVERTS <=> on(t2, Supply, d) REVERTS, AND dust[b]' == dust[b] AND activating[b]' == activating[b]`	✅	🎯
MC-HL-39	`hlMcNoRestrictionIsTerminal`	An unfreeze and a resume always reopen the market, whatever call came before. `any f(...); unfreeze(); resume(): !isFrozen' AND now >= resumeSince'`	✅	🎯
MC-HL-40	`hlMcRootSliceHasMonotoneLtvFloor`	While collateral covers the debt, the LTV-step cut's minimum rises with the LTV. `debt <= col => (allowedSlice(root) + 1) * (WAD - ltv + step) > debt * step`	✅	🎯
MC-HL-41	`hlMcFallbackAloneMaturesSurchargedLot`	A deposit whose surcharged wait runs past the quorum window matures with no quorum scan. `start[b] > 0 AND maxGuardiansQuorumDelay != 0 AND unlock >= start[b] + maxGuardiansQuorumDelay AND quorumFrontier < unlock AND now >= unlock; on(id, Borrow, d): activating[b]' == 0 AND start[b]' == 0`	✅	🎯

MC-HL-42	`hlMcInstantSubThresholdDepositStaysDust`	An instant-activation deposit below the threshold lands wholly as dust, never active. `instantActivationCallers[caller] AND activatingStart[b] == 0 AND assets + dust[b] < minActivationAmount; on(id, SupplyCollateral, d): dust[b]' == assets + dust[b] AND activating[b]' == 0 AND activatingStart[b]' == 0`	✅	🎯
MC-HL-43	`hlMcSameTxDemotingWithdrawUndoesInflow`	A same-transaction withdrawal clears the inflow charge even on a demoted remainder. `ungranted caller, debt-free b, no lot, no dust, assets >= minActivationAmount, activatingLockAmount == 0, 0 < assets - w < minActivationAmount, currentInflow() + assets <= maxDelayInflow: on(SupplyCollateral, assets); on(WithdrawCollateral, w): inflow' == currentInflow() AND transientInflow[b] == 0`	❌→✅	Issue #11 (LOW)
MC-HL-44	`hlMcLiquidateSeizeFoldsSubThresholdRemainder`	A seizure leaves no activating remainder below the activation floor. `on(id, Liquidate, d), b = d.borrower: activating[b]' != 0 => activating[b]' >= minActivationAmount`	❌→✅	Issue #68 (LOW)
MC-HL-45	`hlMcBadDebtVerdictIsNotRounding`	The settle gate calls bad debt only on a root that is underwater in exact arithmetic. `root = nextSettlementBorrower(), shares[root] * (totalBorrowAssets + 1) * SCALE <= collateral[root] * price * (totalBorrowShares + VIRTUAL_SHARES) => !hasBadDebt()`	❌→✖	Issue #73 (MEDIUM)
MC-HL-46	`hlMcNextToSettleIsAlwaysCuttable`	The borrower the queue names as next to settle is one a lender can cut. `root = nextSettlementBorrower(), root != 0 AND shares[root] != 0 AND collateral[root] != 0 AND sliceFloorAssets != 0 => nextSettlementBorrower().borrowShares != 0`	❌→✖	Issue #73 (MEDIUM)
MC-HL-47	`hlMcCrossTxWithdrawLeavesInflowCharged`	A deposit withdrawn in a later transaction leaves the inflow meter charged; only the same-tx counter can undo an addition, and a fresh transaction starts it at zero. `ungranted caller, debt-free b with no standing lot AND no dust, assets >= minActivationAmount, activatingLockAmount == 0, inflow <= maxDelayInflow, t2 >= t1: on(t1, SupplyCollateral, assets); transientInflow[b] := 0; on(t2, WithdrawCollateral, assets) => inflow' == inflow AND inflowUpdatedAt' == inflowUpdatedAt`	✅	🎯

Unit Tests

Identification	Name	Description	Properties	Mutations
MC-UT-01	`utMcMaxDepositMatchesCapHeadroom`	maxDeposit is the remaining supply-cap headroom, zero while paused. `paused => room == 0; !paused AND supplyCap == 0 => room == max_uint256; !paused AND supplyCap != 0 => room == (supplyCap > totalSupplyAssets ? supplyCap - totalSupplyAssets : 0)`	✅	🎯
MC-UT-02	`utMcMaxWithdrawableMatchesFormula`	maxWithdrawableAssets is a supplier's supply capped by idle liquidity, zero while paused. `paused => maxWithdrawableAssets(s) == 0; else == min(supplyShares[s] * (totalSupplyAssets + 1) / (totalSupplyShares + VIRTUAL_SHARES), max(totalSupplyAssets - totalBorrowAssets, 0))`	✅	🎯
MC-UT-03	`utMcHasBadDebtMatchesRootPredicate`	hasBadDebt is true exactly when the queue root is underwater. `nodes.length <= 1 => !bad; nodes.length > 1 => (bad <=> borrowShares[root] != 0 AND toAssetsUp(borrowShares[root]) > collateral[root] * price / ORACLE_PRICE_SCALE)`	✅	🎯🎯
MC-UT-04	`utMcNextSettlementBorrowerIsHeapRoot`	nextSettlementBorrower hands back the queue root, and nothing on an empty queue. `heap.nodes.length > 1 => nextSettlementBorrower().borrower == heap.nodes[1].account; heap.nodes.length <= 1 => nextSettlementBorrower() == (0, 0, 0, 0)`	✅	🎯
MC-UT-05	`utMcPositionOfUnlockTimeFormula`	positionOf's unlock time carries the guardian surcharges, and a ban means no unlock. `start[b] == 0 => unlockTime == 0; start[b] != 0 AND banned => unlockTime == max_uint256; start[b] != 0 AND !banned => unlockTime == start[b] + activatingDelay[b] + strikeCount * delayPerStrike + (start[b] < suspectExpiryTimestamp ? suspectCount * delayPerSuspect : 0); activating, dust, banned, flagCount reported verbatim; active == collateral[b] - activating - dust; debtAssets == toAssetsUp(borrowShares[b]); settlementLtv == settlementLtv[b]; isNextToSettle <=> queue root is b`	✅	🎯🎯🎯
MC-UT-06	`utMcSetSupplyCapStoresValue`	setSupplyCap installs exactly the cap the manager asked for. `setSupplyCap(cap) => supplyCap() == cap`	✅	🎯
MC-UT-07	`utMcFreezeLatchesFlag`	The market reports itself frozen as soon as the freezer acts. `freeze() => isFrozen()`	✅	🎯
MC-UT-08	`utMcRootSliceHonorsLtvStep`	The LTV-step cut lowers the riskiest borrower's LTV by one step, capped at their debt. `step = ltvStepBps*WAD/BPS, ltv = debt*WAD/col: allowedSlice(root) == min(sliceToLtv(debt, col, max(ltv - step, 0)), debt)`	✅	🎯
MC-UT-09	`utMcRootSliceSaturatesToFullDebt`	With LTV at or under one step, the riskiest borrower's allowed cut is their whole debt. `sliceToLtv(debt, col, 0) == debt AND (ltv <= step => allowedSlice(root) == debt)`	✅	🎯

Reachability

Identification	Name	Description	Properties	Mutations
MC-RC-01	`rcMcSetSupplyCapIsReachable`	The manager can move the supply cap. `setSupplyCap(cap): satisfy(supplyCap' !=`	✅	🎯

		`supplyCap)`		
MC-RC-02	`rcMcSetCollateralActivationConfigIsReachable`	The manager can move the activation wait floor or cap. `setCollateralActivationConfig(cfg): satisfy(minActivationDelay' != minActivationDelay OR maxActivationDelay' != maxActivationDelay)`	✅	🎯
MC-RC-03	`rcMcSetSettlementConfigIsReachable`	The manager can change how deeply a settlement may cut a borrower. `setSettlementConfig(config): satisfy(sliceFloorBps' != sliceFloorBps OR ltvStepBps' != ltvStepBps)`	✅	🎯
MC-RC-04	`rcMcSetSettlementLtvIsReachable`	A borrower can opt in to a settle-down LTV target. `setSettlementLtv(ltv): satisfy(settlementLtv[msg.sender]' == ltv AND ltv > 0)`	✅	🎯
MC-RC-05	`rcMcGrantInstantActivationIsReachable`	The manager can grant another address instant activation. `grantInstantActivation(c): satisfy(!instantActivationCallers[c] AND instantActivationCallers[c]' AND c != msg.sender)`	✅	🎯
MC-RC-06	`rcMcRevokeInstantActivationIsReachable`	The manager can revoke a live instant activation grant. `revokeInstantActivation(c): satisfy(instantActivationCallers[c] AND !instantActivationCallers[c]')`	✅	🎯
MC-RC-07	`rcMcOnBorrowGrowsHeapIsReachable`	A borrow can add a new borrower to the settlement queue. `on(Borrow): satisfy(nodes.length' == nodes.length + 1)`	✅	🎯
MC-RC-08	`rcMcOnSupplyCollateralCreditsActivatingIsReachable`	A collateral deposit can land in the activation timer. `on(SupplyCollateral): satisfy(activating[b]' > activating[b])`	✅	🎯
MC-RC-09	`rcMcOnWithdrawCollateralDebitsBucketsIsReachable`	A borrower can withdraw dust or collateral still in the timer. `on(WithdrawCollateral): satisfy(dust[b]' < dust[b] OR 0 < activating[b]' < activating[b])`	✅	🎯
MC-RC-10	`rcMcOnRepayShrinksHeapIsReachable`	A borrower repaying in full can drop out of the settlement queue. `on(Repay): heapLen' == heapLen - 1 AND slot[b] != 0 AND slot'[b] == 0 AND collateral'[b] > 0`	✅	🎯
MC-RC-11	`rcMcOnSettleShrinksHeapIsReachable`	A settled borrower can drop out of the settlement queue. `on(Settle): satisfy(slot[b] != 0 AND slot[b]' == 0 AND nodes.length' == nodes.length - 1)`	✅	🎯
MC-RC-12	`rcMcOnLiquidateSeizesBucketsIsReachable`	A liquidation can seize a borrower's dust or collateral still in the timer. `on(Liquidate): satisfy(dust[b]' + activating[b]' < dust[b] + activating[b])`	✅	🎯
MC-RC-13	`rcMcOnSupplyPassesLiveCapIsReachable`	A zero-asset supply can pass the cap check with the market exactly at the cap. `on(Supply): satisfy(supplyCap > 0 AND totalSupplyAssets == supplyCap)`	✅	🎯
MC-RC-14	`rcMcOnNoOpEventsSucceedWithEmptyPayloadIsReachable`	A lender withdrawal and an interest accrual can pass the policy hook carrying no data. `on(Withdraw, d1); on(AccrueInterest, d2): satisfy(d1.length == 0 AND d2.length == 0)`	✅	🎯
MC-RC-15	`rcMcOnFirstEnqueueFromEmptyHeapIsReachable`	A borrower can enter an empty settlement queue. `on(...): satisfy(nodes.length == 1 AND nodes.length' == 2)`	✅	🎯
		A queued borrower's risk ranking can change while the queue size stays the same.		

MC-RC-16	`rcMcOnRekeysNodeAtFullHeapIsReachable`	`on(...): satisfy(nodes.length == MAX_HEAP_NODES + 1 AND nodes.length' == nodes.length AND 1 <= i < nodes.length AND nodes[i].key' != nodes[i].key)`	✅	🎯
MC-RC-17	`rcMcFreezeThenUnfreezeIsReachable`	The market can be frozen and then unfrozen again. `freeze(); unfreeze(): satisfy(!isFrozen AND isFrozen(mid) AND !isFrozen')`	✅	🎯🎯
MC-RC-18	`rcMcPauseThenResumeIsReachable`	The market can be paused and then resumed early. `pauseFor(d); resume(): satisfy(!isPaused AND isPaused(mid) AND !isPaused')`	✅	🎯🎯
MC-RC-19	`rcMcGrantThenHasInstantActivationIsReachable`	A fresh instant-activation grant can be read back at once. `grantInstantActivation(c); hasInstantActivation(c): satisfy(!instantActivationCallers[c] AND has)`	✅	🎯
MC-RC-20	`rcMcSetThenReadSettlementLtvIsReachable`	A borrower's fresh settle-down LTV target can read back at once. `setSettlementLtv(ltv); settlementLtv(msg.sender): satisfy(readLtv == ltv AND readLtv > 0)`	✅	🎯
MC-RC-21	`rcMcResumeWhileFrozenIsReachable`	The pause can be lifted while the market stays frozen. `resume(): isFrozen AND resumeSince > now AND resumeSince' <= now`	✅	🎯
MC-RC-22	`rcMcNextSettlementBorrowerRealIsReachable`	The settlement queue can name a real borrower to settle next. `nextSettlementBorrower(): satisfy(borrower != 0)`	✅	🎯
MC-RC-23	`rcMcHasBadDebtIsReachable`	The bad-debt check can report true when the queue root is underwater. `satisfy(hasBadDebt())`	✅	🎯
MC-RC-24	`rcMcSettlementForSizesPositiveCutIsReachable`	A borrower can be eligible for a positive settlement cut. `settlementFor(b): satisfy(borrowShares > 0)`	✅	🎯
MC-RC-25	`rcMcMaxDepositPositiveUnderCapIsReachable`	The market can still report room to deposit while a supply cap is in force. `satisfy(maxDeposit() > 0 AND supplyCap > 0)`	✅	🎯
MC-RC-26	`rcMcMaxWithdrawableAssetsPositiveIsReachable`	A lender can be shown a positive withdrawable amount. `satisfy(maxWithdrawableAssets(s) > 0)`	✅	🎯
MC-RC-27	`rcMcActivationDelayStrictlyGrowsIsReachable`	A bigger deposit can face a strictly longer activation wait. `satisfy(x < y AND currentActivationDelay(x) < currentActivationDelay(y))`	✅	🎯
MC-RC-28	`rcMcOnLastDequeueEmptiesHeapIsReachable`	The last borrower leaving can return the settlement queue to empty. `on(...): heapLen == 2 AND heapLen' == 1`	✅	🎯
MC-RC-29	`rcMcOnSupplyCollateralInstantActivationIsReachable`	An instant-activation deposit can release held dust straight into borrowable collateral. `on(SupplyCollateral): satisfy(dust[b] > 0 AND dust[b]' == 0 AND activating[b]' == 0)`	✅	🎯
MC-RC-30	`rcMcOnSupplyCollateralDustStaysIsReachable`	A sub-threshold deposit can stay dust with the timed lot untouched. `on(SupplyCollateral): satisfy(dust[b]' > dust[b] AND activating[b]' == activating[b])`	✅	🎯
MC-RC-31	`rcMcOnBorrowMaturesLotIsReachable`	An unflagged borrower's collateral can mature the moment its wait elapses. `on(Borrow): satisfy(activating[b] > 0 AND activating[b]' == 0 AND dust[b]' == dust[b] AND block.timestamp == activatingStart[b] + activatingDelay[b] AND strikeCount[b] == 0 AND suspectCount[b] == 0)`	✅	🎯

MC-RC-32	`rcMcOnSupplyWithCapHeadroomIsReachable`	A supply can pass the cap check with the market total below the cap. `on(Supply) succeeds with totalSupplyAssets < supplyCap`	✅	🎯
MC-RC-33	`rcMcOnWithdrawCollateralDemotesRemainderIsReachable`	A withdrawal can leave a sub-threshold remainder as untimed dust. `on(WithdrawCollateral): dust'[b] > dust[b] AND activating'[b] == 0 AND activatingStart'[b] == 0`	✅	🎯
MC-RC-34	`rcMcOnWithdrawCollateralFoldsDustPastThresholdIsReachable`	A withdrawal can leave untimed dust at or above the activation threshold. `on(id, WithdrawCollateral, d): satisfy(dust[b]' > dust[b] AND dust[b]' >= minActivationAmount)`	✅	🎯
MC-RC-35	`rcMcOnBorrowFallbackMaturesSurchargedLotIsReachable`	A flagged deposit can still mature on the quorum fallback with the guardian scan behind. `on(Borrow): satisfy(activating[b] > 0 AND activating[b]' == 0 AND surcharge > 0 AND maxGuardiansQuorumDelay != 0 AND unlock >= start[b] + maxGuardiansQuorumDelay AND quorumFrontier < unlock AND now >= unlock)`	✅	🎯
MC-RC-36	`rcMcOnSupplyCollateralInstantDustStaysIsReachable`	A sub-threshold deposit can land as dust even with instant activation. `on(SupplyCollateral): satisfy(instantActivationCallers[caller] AND dust[b] == 0 AND d.assets > 0 AND dust[b]' > 0 AND activating[b]' == activating[b])`	✅	🎯
MC-RC-37	`rcMcMaturedLotReportedActivatingIsReachable`	Collateral past its unlock can still read as waiting until a market action matures it. `unlock = start[b] + activatingDelay[b] + strikeCount[b] * delayPerStrike + (start[b] < suspectExpiry[b] ? suspectCount[b] * delayPerSuspect : 0); satisfy(start[b] != 0 AND !banned[b] AND now >= unlock AND (unlock <= quorumFrontier OR now >= start[b] + maxGuardiansQuorumDelay) AND positionOf(b).activating == activating[b] AND activating[b] > 0)`	✅	🎯
MC-RC-38	`rcMcOnRepayNotBlockedByFreeze`	A borrower can still repay while the market is frozen. `satisfy(isFrozen AND on(id, Repay, d) completes)`	✅	🎯
MC-RC-39	`rcMcOnRepayNotBlockedByBan`	A banned borrower can still repay their debt. `satisfy(isBanned(id, d.caller) AND isBanned(id, d.onBehalf) AND on(id, Repay, d) completes)`	✅	🎯
MC-RC-40	`rcMcOnWithdrawCollateralDebtFreeExitNotBlocked`	A debt-free borrower can still pull their collateral out under both a freeze and a ban. `satisfy(borrowShares[b] == 0 AND assets != 0 AND isFrozen AND isBanned(id, d.caller) AND isBanned(id, d.onBehalf) AND on(id, WithdrawCollateral, d) completes)`	✅	🎯
MC-RC-41	`rcMcOnSupplyCollateralNotBlockedByBan`	A banned address can still post collateral. `satisfy(isBanned(id, d.caller) AND isBanned(id, d.onBehalf) AND on(id, SupplyCollateral, d) completes)`	✅	🎯

Reverts

Identification	Name	Description	Properties	Mutations
MC-RV-01	`rvMcOnRevertsWhenCallerNotCore`	Only the core may drive the market hook. `msg.sender != LIDO_LEND => on(id, ev, data) REVERTS`	✅	🎯
MC-RV-02	`rvMcOnRevertsOnForeignMarketId`	A hook targeting any id but the bound market reverts. `id != marketId => on(id, ev, data)`	✅	🎯

		`REVERTS`		
MC-RV-03	`rvMcOnRevertsWhenPaused`	A pause stops every core action on this market. `now < resumeSinceTimestamp => on(id, ev, data) REVERTS`	✅	🎯
MC-RV-04	`rvMcOnPassiveEventRevertsIff`	Only the caller, market and pause gates can stop a lender withdrawal or interest accrual. `ev in {Withdraw, AccrueInterest}: on(id, ev, data) REVERTS <=> (msg.sender != LIDO_LEND OR id != marketId OR now < resumeSinceTimestamp)`	✅	🎯
MC-RV-05	`rvMcOnBorrowRevertsWhenFrozen`	A frozen market reverts every new borrow. `isFrozen => on(id, Borrow, data) REVERTS`	✅	🎯
MC-RV-06	`rvMcOnSettleRevertsOnRootBadDebt`	No settle proceeds while the riskiest queued borrower is underwater. `hasBadDebt() => on(id, Settle, data) REVERTS`	✅	🎯
MC-RV-07	`rvMcSetActivationConfigRevertsIff`	The manager can install an activation config exactly when every parameter is in range. `authorized: setCollateralActivationConfig(cfg) REVERTS <=> cfg fails any of the nine field guards`	✅	🎯
MC-RV-08	`rvMcGrantInstantActivationNeverReverts`	The manager can always grant instant activation. `authorized: grantInstantActivation(caller) never REVERTS`	✅	🎯
MC-RV-09	`rvMcRevokeInstantActivationNeverReverts`	The manager can always revoke instant activation. `authorized: revokeInstantActivation(caller) never REVERTS`	✅	🎯
MC-RV-10	`rvMcSetSupplyCapNeverReverts`	The manager can always set the supply cap to any value. `authorized: setSupplyCap(cap) never REVERTS`	✅	🎯
MC-RV-11	`rvMcSetSettlementConfigRevertsIff`	The manager can install a settlement policy exactly when its percentages are within 100%. `authorized: setSettlementConfig(c) REVERTS <=> c.sliceFloorBps > BPS OR c.ltvStepBps > BPS`	✅	🎯
MC-RV-12	`rvMcSetSettlementLtvRevertsIff`	Any borrower can set their own settle-down LTV target, up to just below 100%. `setSettlementLtv(ltv) REVERTS <=> ltv >= WAD`	✅	🎯
MC-RV-13	`rvMcFreezeNeverReverts`	The freezer can always freeze the market. `authorized: freeze() never REVERTS`	✅	🎯
MC-RV-14	`rvMcUnfreezeNeverReverts`	The unfreezer can always lift a freeze. `authorized: unfreeze() never REVERTS`	✅	🎯
MC-RV-15	`rvMcPauseForRevertsIff`	The pauser can pause a running market for any nonzero duration that does not overflow. `authorized: pauseFor(d) REVERTS <=> paused OR d == 0 OR (d != max_uint256 AND now + d > max_uint256)`	✅	🎯
MC-RV-16	`rvMcResumeRevertsIff`	The resumer can always lift a live pause. `authorized: resume() REVERTS <=> !paused`	✅	🎯
MC-RV-17	`rvMcIsPausedNeverReverts`	Anyone can always read whether the market is paused. `isPaused() never REVERTS`	✅	🎯
MC-RV-18	`rvMcGetResumeSinceTimestampNeverReverts`	Anyone can always read when the pause ends. `getResumeSinceTimestamp() never REVERTS`	✅	🎯
MC-RV-19	`rvMcCollateralActivationConfigNeverReverts`	Anyone can always read the activation config. `collateralActivationConfig() never REVERTS`	✅	🎯
MC-RV-20	`rvMcCurrentInflowNeverReverts`	Anyone can always read the inflow throttle's current level. `currentInflow() never REVERTS`	✅	🎯
		Anyone can always read the wait a deposit of any size would face.		

MC-RV-21	`rvMcCurrentActivationDelayNeverReverts`	`currentActivationDelay(inflowIncrement)` never REVERTS	✅	🎯
MC-RV-22	`rvMcHasInstantActivationNeverReverts`	Anyone can always check who has instant activation. `hasInstantActivation(caller)` never REVERTS	✅	🎯
MC-RV-23	`rvMcMaxDepositNeverReverts`	Anyone can always read the remaining deposit room. `maxDeposit()` never REVERTS	✅	🎯
MC-RV-24	`rvMcMaxWithdrawableAssetsNeverReverts`	Anyone can always read how much a lender can pull right now. `maxWithdrawableAssets(supplier)` never REVERTS `(supplyShares[supplier] <= totalSupplyShares)`	✅	🎯
MC-RV-25	`rvMcSettlementConfigNeverReverts`	Anyone can always read the settlement policy. `settlementConfig()` never REVERTS	✅	🎯
MC-RV-26	`rvMcSettlementLtvViewNeverReverts`	Anyone can always read a borrower's settle-down LTV target. `settlementLtv(borrower)` never REVERTS	✅	🎯
MC-RV-27	`rvMcSettlementBorrowersCountNeverReverts`	Anyone can always read the size of the settlement queue. `settlementBorrowersCount()` never REVERTS	✅	🎯
MC-RV-28	`rvMcHasBadDebtNeverReverts`	Anyone can always read whether the market carries bad debt. `hasBadDebt()` never REVERTS	✅	🎯
MC-RV-29	`rvMcNextSettlementBorrowerNeverReverts`	Anyone can always read who is next to settle and how much may be cut. `nextSettlementBorrower()` never REVERTS	⏱	🎯
MC-RV-30	`rvMcSettlementForNeverReverts`	Anyone can always read the amounts a cut against a borrower may take. `settlementFor(borrower)` never REVERTS	⏱	🎯🎯
MC-RV-31	`rvMcSettlementBorrowersNeverReverts`	Anyone can always page the settlement queue, even past its end, in windows of up to three. `settlementBorrowers(offset, maxCount)` never REVERTS `(maxCount <= 3)`	✅	🎯
MC-RV-32	`rvMcPositionOfNeverReverts`	Anyone can always read a borrower's whole position. `positionOf(borrower)` never REVERTS `(activating + dust <= collateral)`	✅	🎯
MC-RV-33	`rvMcOnSupplyRevertsAboveCapExact`	A lender deposit that would push the market past a live supply cap is rejected. `supplyCap != 0 AND totalSupplyAssets + d.assets > supplyCap => on(id, Supply, d)` REVERTS	✅	🎯🎯
MC-RV-34	`rvMcOnWithdrawRevertsWhenFrozenWithDebt`	A frozen market reverts a collateral withdraw while the position still carries debt. `isFrozen AND borrowShares[d.onBehalf] != 0 => on(id, WithdrawCollateral, d)` REVERTS	✅	🎯
MC-RV-35	`rvMcOnBorrowRevertsWhenActorBanned`	A ban on the caller, the receiver, or the borrower reverts the borrow. `isBanned(id, d.caller) OR isBanned(id, d.receiver) OR isBanned(id, d.onBehalf) => on(id, Borrow, d)` REVERTS	✅	🎯🎯
MC-RV-36	`rvMcOnBorrowRevertsBelowDustMarketMinimum`	A market under its borrowing floor rejects smaller borrows, and any zero-asset borrow. `(totalBorrowAssets < DUST_MARKET_MIN_BORROW AND d.assets < DUST_MARKET_MIN_BORROW) OR d.assets == 0 => on(id, Borrow, d)` REVERTS	✅	🎯
MC-RV-37	`rvMcOnBorrowRevertsAboveActiveCapacity`	Only active collateral above the threshold backs a borrow, whatever the timer does. `unfrozen: debtAssets + d.assets > maxBorrow(collateral - dust) OR (collateral - dust < minActivationAmount AND d.assets > 0) => on(id, Borrow, d)` REVERTS	⏱	🎯🎯
		A borrower cannot pull collateral their debt still needs, whatever the timer does.		

MC-RV-38	`rvMcOnWithdrawRevertsWhenRemainingActiveInsufficient`	`unfrozen: borrowShares[d.onBehalf] != 0 AND debtAssets > maxBorrow(collateral - d.assets - max(dust - d.assets, 0)) => on(id, WithdrawCollateral, d) REVERTS`	⏱	🎯
MC-RV-39	`rvMcOnSettleIneligibleBorrowerReverts`	A settle can only cut the riskiest queued borrower, or one who opted in. `settlementLtv[d.borrower] == 0 AND d.borrower != queue root AND d.borrowSharesCanceled > 0 => on(id, Settle, d) REVERTS`	✅	🎯
MC-RV-40	`rvMcOnSettleRevertsWhenExceedsAllowedSlice`	A settle cannot cancel more debt than the policy allows. `d.borrowSharesCanceled > toSharesUp(positionOf(d.borrower).slice) => on(id, Settle, d) REVERTS`	✅	🎯
MC-RV-41	`rvMcOnFullRepayRemovesFromQueue`	A full repay drops the borrower out of the settlement queue. `d.shares == borrowShares[d.onBehalf]: on(id, Repay, d) => heap.slot[d.onBehalf] == 0`	✅	🎯
MC-RV-42	`rvMcOnSupplyCollateralInstantSurvivesCallerBan`	A banned instant-activation caller can still land a deposit active at once. `satisfy(isBanned(id, d.caller) AND instantActivationCallers[d.caller] AND NOT isBanned(id, d.onBehalf) AND d.assets + dust[b] >= minActivationAmount AND on(id, SupplyCollateral, d) completes AND dust[b]' == 0 AND activating[b]' == activating[b] AND activatingStart[b]' == 0)`	✅	🎯
MC-RV-43	`rvMcOnSupplyCollateralInstantSurvivesFreeze`	An instant-activation caller can still land a deposit active in a frozen market. `satisfy(isFrozen AND instantActivationCallers[d.caller] AND d.assets + dust[b] >= minActivationAmount AND on(id, SupplyCollateral, d) completes AND dust[b]' == 0 AND activating[b]' == activating[b] AND activatingStart[b]' == 0)`	✅	🎯
MC-RV-44	`rvMcOnPartialRepayKeepsQueued`	A partial repay keeps the borrower queued exactly while collateral remains. `d.shares < borrowShares[d.onBehalf]: on(id, Repay, d) => (heap.slot[d.onBehalf] != 0 <=> collateral[d.onBehalf] > 0)`	✅	🎯
MC-RV-45	`rvMcOnRepayDustRootStaysNext`	A repay down to one borrow share can leave the borrower as the next settlement target. `satisfy(d.shares + 1 == borrowShares[d.onBehalf] AND on(id, Repay, d) completes AND nextSettlementBorrower().borrower == d.onBehalf)`	✅	🎯
MC-RV-46	`rvMcOnWithdrawCollateralDebtFreeFullExitRevertsUnderLock`	A debt-free full collateral exit reverts while the deposit lock holds. `borrowShares[b] == 0 AND start[b] != 0 AND now < start[b] + activatingLockDuration AND now < start[b] + activatingDelay[b] AND activatingLockAmount != 0 AND assets == collateral[b] => on(id, WithdrawCollateral, d) REVERTS`	✅	🎯🎯
MC-RV-47	`rvMcOnSupplyRevertsIff`	Only the caller, market, pause and cap gates can stop a lender deposit. `on(id, Supply, d) REVERTS <=> (msg.sender != LIDO_LEND OR id != marketId OR now < resumeSinceTimestamp OR (supplyCap != 0 AND totalSupplyAssets + d.assets > supplyCap))`	✅	🎯
MC-RV-48	`rvMcOnRepayRevertsIff`	Only the caller, market and pause gates and an over-sized cut can stop a repayment. `on(id, Repay, d) REVERTS <=> (msg.sender != LIDO_LEND OR id != marketId OR now <`	✅	🎯

		`resumeSinceTimestamp OR d.shares > borrowShares[d.onBehalf])`		
MC-RV-49	`rvMcOnLiquidateRevertsIff`	Only the caller, market and pause gates and a payload past the position can stop a liquidation. `on(id, Liquidate, d) REVERTS <=> (msg.sender != LIDO_LEND OR id != marketId OR now < resumeSinceTimestamp OR d.repaidShares > borrowShares[d.borrower] OR d.seizedAssets > collateral[d.borrower])`	✅	🎯
MC-RV-50	`rvMcOnBorrowRevertsIff`	Only its own gates and the active-capacity check can stop a borrow. `on(id, Borrow, d) REVERTS <=> (msg.sender != LIDO_LEND OR id != marketId OR now < resumeSinceTimestamp OR isFrozen OR d.assets == 0 OR (totalBorrowAssets < DUST_MARKET_MIN_BORROW AND d.assets < DUST_MARKET_MIN_BORROW) OR banned(d.caller) OR banned(d.receiver) OR banned(d.onBehalf) OR debtAssets + d.assets > maxBorrow(active))`	⏱	🎯🎯
MC-RV-51	`rvMcOnSupplyCollateralRevertsIff`	A well-formed collateral deposit reverts only on its gates or a bucket that cannot hold it. `on(id, SupplyCollateral, d) REVERTS <=> (msg.sender != LIDO_LEND OR id != marketId OR now < resumeSinceTimestamp OR (d.assets + dust[d.onBehalf] >= minActivationAmount AND !instantActivationCallers[d.caller] AND activatingAfterSync + d.assets + dust[d.onBehalf] > max_uint128))`	✅	🎯
MC-RV-52	`rvMcOnWithdrawCollateralRevertsIff`	A well-formed collateral withdrawal reverts only on its gates, the position bound, the locked remainder or the debt it would strand. `on(id, WithdrawCollateral, d) REVERTS <=> (msg.sender != LIDO_LEND OR id != marketId OR now < resumeSinceTimestamp OR (isFrozen AND borrowShares != 0) OR d.assets > collateral OR inactiveAfterDrain > collateral - d.assets OR (borrowShares != 0 AND debtAssets > maxBorrow(activeAfter)))`	⏱	🎯🎯

Access Control

Identification	Name	Description	Properties	Mutations
MC-AC-01	`acMcOnlyManagerMovesActivationConfig`	Only the manager can change the activation config. `any activation-config field changes => hasRole(MANAGER_ROLE, msg.sender)`	✅	🎯🎯
MC-AC-02	`acMcOnlyManagerMovesSettlementConfig`	Only the manager can change the settlement policy. `any settlement-config field changes => hasRole(MANAGER_ROLE, msg.sender)`	✅	🎯
MC-AC-03	`acMcOnlyManagerMovesSupplyCap`	Only the manager can move the supply cap. `supplyCap' != supplyCap => hasRole(MANAGER_ROLE, msg.sender)`	✅	🎯
MC-AC-04	`acMcOnlyManagerFlipsInstantActivation`	Only the manager can grant or revoke instant activation. `instantActivationCallers'[c] != instantActivationCallers[c] => hasRole(MANAGER_ROLE, msg.sender)`	✅	🎯
MC-AC-05	`acMcOnlyFreezerFreezes`	Only the freezer can freeze the market. `!isFrozen AND isFrozen' => hasRole(FREEZER_ROLE, msg.sender)`	✅	🎯
MC-AC-06	`acMcOnlyUnfreezerUnfreezes`	Only the unfreezer can lift a freeze. `isFrozen AND !isFrozen' => hasRole(UNFREEZER_ROLE, msg.sender)`	✅	🎯
MC-AC-07	`acMcOnlyPauserPauses`	Only the pauser can pause the market. `!isPaused() AND isPaused()' => hasRole(PAUSER_ROLE, msg.sender)`	✅	🎯
MC-AC-08	`acMcOnlyResumerResumes`	Only the resumer can end a pause early. `isPaused() AND !isPaused()' => hasRole(RESUMER_ROLE, msg.sender)`	✅	🎯
MC-AC-09	`acMcOnlyCoreMovesActivationPosition`	Only the core can move a borrower's un-matured collateral or its timer. `any of positions[b].{dust, activating, activatingStart, activatingDelay} changes => msg.sender == LIDO_LEND`	✅	🎯
MC-AC-10	`acMcOnlyCoreMovesInflow`	Only the core can move the inflow throttle. `inflow OR inflowUpdatedAt changes => msg.sender == LIDO_LEND`	✅	🎯
MC-AC-11	`acMcOnlyCoreMovesSettlementHeap`	Only the core can change the settlement queue. `nodes.length, slot[a], nodes[i].account OR nodes[i].key changes => msg.sender == LIDO_LEND`	✅	🎯
MC-AC-12	`acMcOnlyDefaultAdminMovesControllerRoles`	A role is granted or revoked only by the default admin, or given up by its own holder. `role in {MANAGER, FREEZER, UNFREEZER, PAUSER, RESUMER, DEFAULT_ADMIN}, roleAdmin[role] == DEFAULT_ADMIN_ROLE: hasRole[role][account]' != hasRole[role][account] => hasRole[DEFAULT_ADMIN_ROLE][msg.sender] OR (account == msg.sender AND !hasRole[role][account]')`	✅	🎯
MC-AC-13	`acMcRoleAdminNeverMoves`	No entry point re-points a role's admin. `getRoleAdmin(role)' == getRoleAdmin(role)`	✅	🎯

LidoLend

- Controller, oracle, interest model and tokens linked as CVL models
- Borrower callbacks summarized away, so reentrancy is unanalyzed

- One symbolic market, the per-market mirrors carrying no market key
- Position holders up to 7 modeled accounts plus the zero slot
- Loops running up to 2 iterations

Valid State

Identification	Name	Description	Properties	Mutations
LL-VS-01	`feeCapped`	The protocol's cut of accrued interest can never be set past the contract's own cap. `fee <= MAX_FEE (0.25e18)`	✅	🎯
LL-VS-02	`lastUpdateNotInFuture`	A market's accrual clock never runs ahead of the chain. `lastUpdate <= block.timestamp`	✅	🎯
LL-VS-03	`lastUpdateMinBound`	A live market's accrual clock was stamped by a real block, never a dust value. `lastUpdate != 0 => lastUpdate >= MIN_BLOCK_TIMESTAMP (65535)`	✅	🎯
LL-VS-04	`borrowLeSupplyAssets`	A market never lends out more than its suppliers put in; utilization cannot pass 100%. `totalBorrowAssets <= totalSupplyAssets`	✅	🎯
LL-VS-05	`supplySharesSumCorrect`	The supply-share ledger balances, the total being the sum of every lender's claim. `totalSupplyShares == sum of supplyShares[u] over the tracked accounts AND the zero slot`	✅	🎯
LL-VS-06	`borrowSharesSumCorrect`	The borrow-share ledger balances, the total being the sum of every borrower's debt. `totalBorrowShares == sum of borrowShares[u] over the tracked accounts AND the zero slot`	✅	🎯
LL-VS-07	`borrowerCollateralizedUnlessSettlement`	Outside settlement markets, debt requires collateral; settle can leave it uncollateralized. `!isSettlementEnabled AND borrowShares[u] != 0 => collateral[u] != 0`	✅	🎯
LL-VS-08	`zeroAddressNeverAuthorizes`	Nobody is ever authorized to act on the zero address's behalf, outside the signature-based grant path. `!isAuthorized[0][a]`	✅	🎯
LL-VS-09	`tokenBackingSolvency`	LidoLend always holds enough of the market's loan and collateral tokens to cover the liquidity it has not lent out plus the collateral entrusted to it. `t in {loanToken, collateralToken} => balance[t][LidoLend] >= (t == loanToken ? totalSupplyAssets - totalBorrowAssets : 0) + (t == collateralToken ? sum of collateral[u] over the loan token's tracked accounts AND the zero slot : 0)`	✅	🎯

Variable Transitions

Identification	Name	Description	Properties	Mutations
LL-VT-01	`vtLlLastUpdateMonotonic`	The market's accrual clock never moves backwards. `lastUpdate' >= lastUpdate`	✅	🎯
LL-VT-02	`vtLlInitializedLatchesOn`	The initializer latch is one-way. `isInitialized => isInitialized'`	✅	🎯
LL-VT-03	`vtLlNonceMonotonic`	A signer's nonce never rewinds and never jumps: each call leaves it in place or advances it one step. `nonce[account]' == nonce[account] OR nonce[account]' == nonce[account] + 1`	✅	🎯
LL-VT-04	`vtLlCoreBalanceOutflowOnlyViaExits`	Only the six exit entry points may debit the pool; no other call shrinks its token balance. `f not in {withdraw, borrow, withdrawCollateral, liquidate, settle, flashLoan}: balance[token][LidoLend]' >= balance[token][LidoLend]`	✅	🎯
LL-VT-05	`vtLlMarketWiringImmutable`	A market's wiring never moves after creation: controller binding, settlement flag, whitelists and terms are fixed. `{marketController, isSettlementEnabled, isIrmEnabled[i], isLltvEnabled[l], loanToken, collateralToken, oracle, irm, lltv}' unchanged`	✅	🎯

State Transitions

Identification	Name	Description	Properties	Mutations
LL-ST-01	`stLlBorrowGrowthIsFresh`	Total debt grows only through a path that first brings interest current. `totalBorrowAssets' > totalBorrowAssets => lastUpdate' == now`	✅	🎯
LL-ST-02	`stLlNewDebtNeedsCollateral`	New debt requires posted collateral. `borrowShares[u]' > borrowShares[u] => collateral[u]' != 0`	✅	🎯🎯
LL-ST-03	`stLlFeeChangeIsFresh`	A fee change rides a fresh accrual. `fee' != fee => lastUpdate' == now`	✅	🎯
LL-ST-04	`stLlSupplyShrinkIsAccounted`	The suppliers' pot shrinks only by burning claims against it or by writing debt off the books. `totalSupplyAssets' < totalSupplyAssets =>`	⏱	🎯

		`totalSupplyShares' < totalSupplyShares OR totalBorrowAssets' < totalBorrowAssets`		
LL-ST-05	`stLlSupplyShareMintAddsAssets`	Total supply shares grow only together with total supply assets. `lastUpdate == now: totalSupplyShares' > totalSupplyShares => totalSupplyAssets' > totalSupplyAssets`	✅	🎯
LL-ST-06	`stLlSupplyShareBurnRemovesAssets`	When total supply shares fall, total supply assets fall or stay (rounding can zero the cut). `lastUpdate == now != 0: totalSupplyShares' < totalSupplyShares => totalSupplyAssets' <= totalSupplyAssets`	✅	🎯
LL-ST-07	`stLlBorrowShareMintAddsDebt`	When total borrow shares grow, total borrow assets grow or stay (rounding can zero the draw). `lastUpdate == now: totalBorrowShares' > totalBorrowShares => totalBorrowAssets' >= totalBorrowAssets`	✅	🎯
LL-ST-08	`stLlAccrualFavorsSuppliers`	A pure accrual credits suppliers at least the full debt growth. `f == accrueInterest, lastUpdate != 0: (totalBorrowAssets' > totalBorrowAssets AND totalBorrowShares' == totalBorrowShares) => totalSupplyAssets' - totalSupplyAssets >= totalBorrowAssets' - totalBorrowAssets`	✅	🎯
LL-ST-09	`stLlAccountingRefreshesClock`	Any totals change after time has passed restamps lastUpdate to the current block. `any of totalSupplyAssets, totalSupplyShares, totalBorrowAssets, totalBorrowShares changed AND now > lastUpdate => lastUpdate' == now`	✅	🎯
LL-ST-10	`stLlSupplyShareMintIsFunded`	When supply() grows total supply shares, the pool's loan token balance grows with them. `supply(...), lastUpdate == now: totalSupplyShares' > totalSupplyShares => balance[loanToken][LidoLend]' >`	✅	🎯

		`balance[loanToken][LidoLend]`		
LL-ST-11	`stLlUserSupplyMintGrowsTotal`	A user's supply-share credit always registers in the total. `lastUpdate == now: supplyShares[u]' > supplyShares[u] => totalSupplyShares' > totalSupplyShares`	✅	🎯
LL-ST-12	`stLlUserSupplyBurnShrinksTotal`	A user's supply-share burn always registers in the total. `lastUpdate == now: supplyShares[u]' < supplyShares[u] => totalSupplyShares' < totalSupplyShares`	✅	🎯
LL-ST-13	`stLlUserDebtMintGrowsTotal`	A borrower's debt-share credit always registers in the total. `borrowShares[u]' > borrowShares[u] => totalBorrowShares' > totalBorrowShares`	✅	🎯
LL-ST-14	`stLlUserDebtBurnShrinksTotal`	A borrower's debt-share burn always registers in the total. `borrowShares[u]' < borrowShares[u] => totalBorrowShares' < totalBorrowShares`	✅	🎯

High Level

Identification	Name	Description	Properties	Mutations
LL-HL-01	`hlLlSupplyCreditsExactShares`	Supply credits exactly the returned shares. `onBehalf != feeRecipient: supply() -> retShares: supplyShares[onBehalf]' == supplyShares[onBehalf] + retShares`	✅	🎯
LL-HL-02	`hlLlWithdrawDebitsExactShares`	Withdraw debits exactly the returned shares. `onBehalf != feeRecipient: withdraw() -> retShares: supplyShares[onBehalf]' == supplyShares[onBehalf] - retShares`	✅	🎯
LL-HL-03	`hlLlBorrowCreditsExactDebt`	Borrow credits exactly the returned debt shares. `borrow() -> retShares: borrowShares[onBehalf]' == borrowShares[onBehalf] + retShares`	✅	🎯
LL-HL-04	`hlLlRepayDebitsExactDebt`	Repay debits exactly the returned debt shares. `repay() -> retShares: borrowShares[onBehalf]' == borrowShares[onBehalf] - retShares`	✅	🎯
LL-HL-05	`hlLlSupplyCollateralLeavesLoanTotals`	Posting collateral never moves the market's supply or borrow totals. `supplyCollateral(): totalSupplyAssets' == totalSupplyAssets AND totalSupplyShares' == totalSupplyShares AND totalBorrowAssets' == totalBorrowAssets AND totalBorrowShares' == totalBorrowShares`	✅	🎯
LL-HL-06	`hlLlWithdrawCollateralLeavesBorrowShares`	Taking collateral back never changes the market's total debt shares. `withdrawCollateral(): totalBorrowShares' == totalBorrowShares`	✅	🎯
		flashLoan's own transfers round-trip both token balances and leave the market's supply assets untouched. `callback token moves excluded:`		

LL-HL-07	`hlLlFlashLoanLeavesMarkets`	`totalSupplyAssets' == totalSupplyAssets AND bal[token][LidoLend]' == bal[token][LidoLend] AND bal[token][msg.sender]' == bal[token][msg.sender]`	✅	🎯
LL-HL-08	`hlLlAccrueKeepsBorrowShares`	Accruing interest never mints or burns debt shares. `accrueInterest(): totalBorrowShares' == totalBorrowShares`	✅	🎯
LL-HL-09	`hlLlSupplyLeavesOtherPositions`	Supply leaves every other position untouched, apart from the fee recipient's interest credit. `other != onBehalf AND other != feeRecipient => position[other]' == position[other]`	✅	🎯
LL-HL-10	`hlLlSettleReturnsExactDeltas`	Settle reports exactly what it moved; every amount it returns is the change it made to the books. `lastUpdate == now: (seized, dAssets, dShares, burned) = settle(): burned cuts supplyShares[onBehalf] AND totalSupplyShares, dAssets cuts totalSupplyAssets AND totalBorrowAssets, dShares cuts borrowShares[borrower] AND totalBorrowShares, seized cuts collateral[borrower], AND seized > 0`	✅	🎯🎯🎯🎯
LL-HL-11	`hlLlSettleSeizesAtPar`	Settle seizes collateral at exact par: the floor of canceled debt at the oracle price. `settle(): seizedAssets == floor(borrowAssetsCanceled * ORACLE_PRICE_SCALE / price)`	✅	🎯
LL-HL-12	`hlLlBorrowLeavesPositionHealthy`	Borrow leaves the touched position healthy at the frozen oracle price. `borrow(): healthy(onBehalf)' [healthy(u): (collateral[u]*price/ORACLE_PRICE_SCALE)*lltv/WAD >= ceil(borrowShares[u]* (totalBorrowAssets+1)/(totalBorrowShares+1e6))]`	✅	🎯🎯
LL-HL-13	`hlLlWithdrawCollateralLeavesPositionHealthy`	Pulling collateral out can never leave the borrower's own position under water. `withdrawCollateral(): healthy(onBehalf)' [healthy(u): (collateral[u]*price/ORACLE_PRICE_SCALE)*lltv/WAD >= ceil(borrowShares[u]* (totalBorrowAssets+1)/(totalBorrowShares+1e6))]`	✅	🎯🎯
LL-HL-14	`hlLlLiquidateRevertsOnHealthyBorrower`	With interest brought current, a healthy borrower can never be liquidated. `lastUpdate == now AND healthy(borrower) => liquidate(borrower) REVERTS`	⏱	🎯
LL-HL-15	`hlLlLiquidateDebitsSharesLockstep`	Liquidate debits borrower and market borrow shares in lockstep and seizes the returned assets. `totalBorrowShares - totalBorrowShares' == borrowShares[borrower] - borrowShares[borrower]' AND collateral[borrower]' == collateral[borrower] - retSeized`	✅	🎯🎯🎯🎯
LL-HL-16	`hlLlLiquidateLeavesOtherPositions`	With interest already up to date, liquidate leaves every other position untouched. `lastUpdate == now AND other != borrower => position[other]' == position[other]`	✅	🎯
LL-HL-17	`hlLlAccrueGrowsSupplyAssets`	Interest accrual never shrinks the suppliers' pot. `accrueInterest(): totalSupplyAssets' >= totalSupplyAssets`	✅	🎯🎯
LL-HL-18	`hlLlAccrueAddsEqualValue`	Interest accrual credits suppliers exactly what it charges borrowers. `accrueInterest(): totalSupplyAssets' - totalSupplyAssets == totalBorrowAssets' - totalBorrowAssets`	✅	🎯🎯
LL-HL-19	`hlLlSupplyWithdrawNoProfit`	A supply-then-withdraw round trip never pays out more than went in. `msg.sender == onBehalf == receiver: (_, s) = supply(assets, 0); withdraw(0, s): bal[loanToken][onBehalf]' <= bal[loanToken][onBehalf]`	✅	🎯
LL-HL-20	`hlLlBorrowRepayNoProfit`	A borrow-then-repay round trip always costs at least what was borrowed. `msg.sender == onBehalf == receiver: (_, s) = borrow(assets, 0); repay(0, s): bal[loanToken][onBehalf]' <= bal[loanToken][onBehalf]`	⏱	🎯

LL-HL-21	`hlLlSupplyPullsExactAssets`	Supply moves exactly the credited assets between payer and pool, verbatim in assets mode. `supply() -> retAssets: bal[loanToken][LidoLend]' == bal[loanToken][LidoLend] + retAssets AND bal[loanToken][msg.sender]' == bal[loanToken][msg.sender] - retAssets AND (assets != 0 => retAssets == assets)`	✅	🎯
LL-HL-22	`hlLlWithdrawLeavesOtherPositions`	Withdraw leaves every other position untouched, apart from the fee recipient's interest credit. `other != onBehalf AND other != feeRecipient => position[other]' == position[other]`	✅	🎯
LL-HL-23	`hlLlBorrowLeavesOtherPositions`	Borrow leaves every other position untouched, apart from the fee recipient's interest credit. `other != onBehalf AND other != feeRecipient => position[other]' == position[other]`	✅	🎯
LL-HL-24	`hlLlRepayLeavesOtherPositions`	Repay leaves every other position untouched, apart from the fee recipient's interest credit. `other != onBehalf AND other != feeRecipient => position[other]' == position[other]`	✅	🎯
LL-HL-25	`hlLlSupplyCollateralLeavesOtherPositions`	Posting collateral leaves other users' positions untouched, the fee recipient included. `other != onBehalf => position[other]' == position[other]`	✅	🎯
LL-HL-26	`hlLlWithdrawCollateralLeavesOtherPositions`	Taking collateral back leaves every other position untouched, apart from the fee recipient's interest credit. `other != onBehalf AND other != feeRecipient => position[other]' == position[other]`	✅	🎯
LL-HL-27	`hlLlAccrueRaisesSupplySharePrice`	Accruing interest never cheapens a supply share. `accrueInterest(): (totalSupplyAssets+1)*(totalSupplyShares'+1e6) <= (totalSupplyAssets'+1)*(totalSupplyShares+1e6)`	✅	🎯🎯
LL-HL-28	`hlLlSupplySharePriceOnlyFallsOnSeizure`	With interest accrued, no method other than liquidate cheapens a supply share (LL-HL-29). `f != liquidate, lastUpdate == now (accrual no-op): (totalSupplyAssets + 1) * (totalSupplyShares' + 1e6) <= (totalSupplyAssets' + 1) * (totalSupplyShares + 1e6)`	✅	🎯🎯
LL-HL-29	`hlLlLiquidateNoBadDebtRaisesSupplySharePrice`	A liquidation that leaves the borrower collateral never cheapens a supply share. `lastUpdate == now, collateral[borrower]' != 0: (totalSupplyAssets+1)*(totalSupplyShares'+1e6) <= (totalSupplyAssets'+1)*(totalSupplyShares+1e6)`	✅	🎯
LL-HL-30	`hlLlBorrowSharePriceOnlyRisesOnAccrual`	Once interest is accrued, no operation makes an existing debt share more expensive to hold. `f not in {repay, liquidate, settle}, lastUpdate == now (accrual no-op): (totalBorrowAssets + 1) * (totalBorrowShares' + 1e6) >= (totalBorrowAssets' + 1) * (totalBorrowShares + 1e6)`	✅	🎯
LL-HL-31	`hlLlRepayNeverRaisesDebtSharePrice`	A repayment within the market's whole debt never makes remaining debt shares more expensive. `lastUpdate == now, retAssets <= totalBorrowAssets: (totalBorrowAssets+1)*(totalBorrowShares'+1e6) >= (totalBorrowAssets'+1)*(totalBorrowShares+1e6)`	⏱	🎯
LL-HL-32	`hlLlLiquidateNeverRaisesDebtSharePrice`	With no liquidator callback, a liquidation that leaves the market carrying debt never raises what a debt share owes. `no callback, lastUpdate == now, totalBorrowAssets' != 0: (totalBorrowAssets+1)*(totalBorrowShares'+1e6) >= (totalBorrowAssets'+1)*(totalBorrowShares+1e6)`	⏱	🎯
LL-HL-33	`hlLlLiquidateSeizureBoundedByIncentive`	With no liquidator callback, liquidating a position whose debt sits below its incentive-discounted collateral leaves it no worse off. `lastUpdate == now, no callback, collateral[b] covers debt(b) at the liquidation bonus: collateral[b]' != 0 AND (borrowShares[b]-borrowShares[b]')*collateral[b] >= retSeized*borrowShares[b] AND borrowShares[b]*collateral[b]' >= borrowShares[b]'*collateral[b] AND (totalBorrowAssets' > 0 => what a debt share owes`	⏱	🎯

		`does not grow)`		
LL-HL-34	`hlLlOthersStayHealthy`	Once interest is accrued, no operation pushes an untouched, healthy borrower under water. `f not in {liquidate, settle}, lastUpdate == now, healthy(user), borrowShares[user]' == borrowShares[user] AND collateral[user]' == collateral[user]: healthy'(user) [healthy(u) := (collateral[u]*price/SCALE)*lltv/WAD >= debtUp(u)]`	⏱	🎯
LL-HL-35	`hlLlLiquidateLeavesOthersHealthy`	A seizure-anchored liquidation that leaves the borrower collateral cannot push a healthy bystander under water. `lastUpdate == now, liquidate(seizedAssets, repaidShares = 0), healthy(user), collateral[borrower]' > 0, totalBorrowAssets' > 0: user != borrower AND borrowShares[user]' == borrowShares[user] AND debtUp'(user) <= debtUp(user) AND healthy'(user)`	⏱	🎯
LL-HL-36	`hlLlLastBorrowLiquidationLeavesOthersHealthy`	Even the seizure-anchored liquidation that clears the market's last debt assets leaves a bystander healthy. `lastUpdate == now, liquidate(seizedAssets, repaidShares = 0), healthy(user), totalBorrowAssets' == 0: healthy'(user)`	⏱	🎯
LL-HL-37	`hlLlSupplyBookValueBounded`	Supplying never buys more book value than it pays for, with interest already accrued. `lastUpdate == now: ownedDown(onBehalf)' <= ownedDown(onBehalf) + suppliedAssets [ownedDown(u) = supplyShares[u] valued down at the market rate]`	⏱	🎯
LL-HL-38	`hlLlWithdrawBookValueBounded`	Withdrawing never pays out more than the position was worth before the call. `lastUpdate == now: withdrawnAssets <= ownedDown(onBehalf) [ownedDown(u) = supplyShares[u] valued down at the market rate]`	✅	🎯
LL-HL-39	`hlLlBorrowDebtValueBounded`	With interest accrued, borrowing always books at least as much debt as it hands out. `lastUpdate == now, borrow(0, shares): debtUp(onBehalf)' >= debtUp(onBehalf) + borrowedAssets [debtUp(u) = borrowShares[u] valued up at the market rate]`	⏱	🎯
LL-HL-40	`hlLlRepayCostBounded`	A repayment that clears the position pays at least what its debt shares were worth. `lastUpdate == now, borrowShares[onBehalf]' == 0: repaidAssets >= debtUp(onBehalf) [debtUp(u) = borrowShares[u] valued up at the market rate]`	⏱	🎯
LL-HL-41	`hlLlSupplyWithdrawAnyModeNoProfit`	A supply-then-withdraw round trip in one block never turns a profit, in either input mode. `supplyShares[onBehalf] == 0 AND onBehalf != feeRecipient: suppliedAssets* (totalSupplyShares'+1e6) >= suppliedShares* (totalSupplyAssets'+1) (also net of the mint itself) AND withdrawnAssets <= suppliedAssets`	⏱	🎯🎯
LL-HL-42	`hlLlBorrowRepayAnyModeNoProfit`	A borrow-then-repay round trip in one block never turns a profit, in either borrow input mode, with the draw repaid in shares. `borrowShares[onBehalf] == 0: borrowedAssets* (totalBorrowShares'+1e6) <= borrowedShares* (totalBorrowAssets'+1) (also net of the draw itself) AND repaidAssets >= borrowedAssets`	✅	🎯
LL-HL-43	`hlLlSettleBorrowAnchoredNeverRaisesDebtSharePrice`	A borrow-anchored settle never makes the remaining debt shares more expensive. `lastUpdate == now, settle(0, borrowShares): (totalBorrowAssets+1)*(totalBorrowShares'+1e6) >= (totalBorrowAssets'+1)*(totalBorrowShares+1e6)`	✅	🎯
LL-HL-44	`hlLlSettleBorrowAnchoredLeavesOthersHealthy`	A borrow-anchored settle cannot push a healthy bystander under water. `lastUpdate == now, settle(0, borrowShares), healthy(user), position[user]' == position[user] => healthy(user)'`	⏱	🎯
		A borrow in assets mode draws exactly the assets named and books no less debt than it hands out. `lastUpdate == now, borrow(assets, 0):`		

LL-HL-45	`hlLlBorrowDebtValueBoundedAssetsMode`	`borrowedAssets == assets AND debtUp(onBehalf)' >= debtUp(onBehalf) + assets [debtUp(u) = borrowShares[u] valued up at the market rate]`	⏱	🎯
LL-HL-46	`hlLlFreeLiquidityFallsOnlyOnExitOrDraw`	Free liquidity (supply assets minus borrow assets) never falls except on withdraw or borrow. `f not in {withdraw, borrow}: totalSupplyAssets' - totalBorrowAssets' >= totalSupplyAssets - totalBorrowAssets`	✅	🎯
LL-HL-47	`hlLlWithdrawPaysAtMostFreeLiquidity`	A withdraw pays out at most the market's free liquidity, and takes exactly that out of it. `free = totalSupplyAssets - totalBorrowAssets: retAssets <= free AND free' == free - retAssets`	✅	🎯
LL-HL-48	`hlLlBorrowDrawsAtMostFreeLiquidity`	A borrow draws at most the market's free liquidity, and takes exactly that out of it. `free = totalSupplyAssets - totalBorrowAssets: retAssets <= free AND free' == free - retAssets`	✅	🎯
LL-HL-49	`hlLlSettleOnlyReachesUnservableSupply`	With interest already up to date, a settle cancels no more than the settler's holding net of the market's free liquidity. `lastUpdate == now: borrowAssetsCanceled + free <= ownedDown(onBehalf) AND free < ownedDown(onBehalf) [free := totalSupplyAssets - totalBorrowAssets]`	✅	🎯🎯🎯
LL-HL-50	`hlLlLiquidateBadDebtTailClearsDebt`	A liquidation that empties the collateral leaves no residual debt behind. `no callback: collateral[borrower]' == 0 => borrowShares[borrower]' == 0`	✅	🎯
LL-HL-51	`hlLlLiquidateSocializesAtMostBorrowerDebt`	With interest already up to date, a liquidation never charges suppliers more than the borrower owed. `lastUpdate == now: totalSupplyAssets - totalSupplyAssets' <= debtUp(borrower)`	⏱	🎯
LL-HL-52	`hlLlFeeRecipientRotationKeepsClosedWindow`	Rotating the fee recipient never credits the incoming address with a fee window that closed before it took over. `irm != 0 AND totalBorrowAssets > 0 AND borrowRate > 0 AND fee > 0 AND lastUpdate < now: setFeeRecipient(r); accrueInterest(mp): supplyShares[r]' == supplyShares[r]`	❌→✖	Issue #12 (LOW)

Unit Tests

Identification	Name	Description	Properties	Mutations
LL-UT-01	`utLlSetOwnerSetsNewOwner`	A successful ownership transfer installs exactly the named new owner. `setOwner(newOwner): owner' == newOwner`	✅	🎯🎯
LL-UT-02	`utLlSetFeeRecipientSetsNewRecipient`	The owner installs exactly the named new fee recipient. `setFeeRecipient(newFeeRecipient): feeRecipient' == newFeeRecipient`	✅	🎯🎯
LL-UT-03	`utLlSetFeeSetsNewFee`	The owner sets the market fee to exactly the requested value. `setFee(mp, newFee): fee' == newFee`	✅	🎯
LL-UT-04	`utLlSetAuthorizationSetsFlag`	An authorization change lands exactly the requested flag on the caller's row for the grantee. `setAuthorization(authorized, newIsAuthorized): isAuthorized[msg.sender][authorized]' == newIsAuthorized`	✅	🎯
LL-UT-05	`utLlSupplyCollateralCreditsExactCollateral`	A collateral deposit credits the beneficiary with exactly the assets supplied. `supplyCollateral(mp, assets, onBehalf, data): collateral[onBehalf]' == collateral[onBehalf] + assets`	✅	🎯🎯
LL-UT-06	`utLlWithdrawCollateralDebitsExactCollateral`	A collateral withdrawal debits the position by exactly the requested assets. `withdrawCollateral(mp, assets, onBehalf, receiver): collateral[onBehalf]' == collateral[onBehalf] - assets`	✅	🎯
		Liquidation never grows the borrower's debt shares,		

LL-UT-07	`utLlLiquidateNeverGrowsBorrowerDebt`	whoever calls it. `liquidate(mp, borrower, ...): borrowShares[borrower]' <= borrowShares[borrower]`	✅	🎯
LL-UT-08	`utLlSetOwnerLeavesFeeRecipient`	An ownership transfer leaves the fee recipient untouched. `setOwner(newOwner): feeRecipient' == feeRecipient`	✅	🎯
LL-UT-09	`utLlSetFeeRecipientLeavesOwner`	Changing the fee recipient leaves the owner untouched. `setFeeRecipient(newFeeRecipient): owner' == owner`	✅	🎯
LL-UT-10	`utLlSetAuthorizationLeavesOtherRows`	An authorization change writes only the caller's own row for the named grantee. `setAuthorization(authorized, flag): (otherOwner != msg.sender OR otherAuth != authorized) => isAuthorized[otherOwner][otherAuth]' == isAuthorized[otherOwner][otherAuth]`	✅	🎯
LL-UT-11	`utLlSupplyGrowsTotalsExactly`	With interest up to date, a supply grows both supply totals by exactly the shares and assets it returns. `lastUpdate == now: totalSupplyShares += retShares, totalSupplyAssets += retAssets`	✅	🎯
LL-UT-12	`utLlWithdrawShrinksTotalsExactly`	With interest up to date, a withdrawal shrinks both supply totals by exactly the shares and assets it returns. `lastUpdate == now: totalSupplyShares -= retShares, totalSupplyAssets -= retAssets`	✅	🎯
LL-UT-13	`utLlBorrowGrowsDebtTotalsExactly`	With interest up to date, a borrow grows both debt totals by exactly the shares and assets it returns. `lastUpdate == now: totalBorrowShares += retShares, totalBorrowAssets += retAssets`	✅	🎯
LL-UT-14	`utLlRepayShrinksDebtTotalsExactly`	Repay shrinks the borrower's and the market's debt by exactly the returned amounts, with the assets total floored at zero. `lastUpdate == now (accrual no-op): borrowShares[onBehalf]' == borrowShares[onBehalf] - retShares AND totalBorrowShares' == totalBorrowShares - retShares AND totalBorrowAssets' == max(totalBorrowAssets - retAssets, 0)`	✅	🎯
LL-UT-15	`utLlWithdrawMovesExactTokens`	A withdrawal pays the receiver exactly the assets it returns out of the pool, and an assets-denominated request is paid in full. `receiver != pool: loanBal[receiver] += retAssets, loanBal[pool] -= retAssets, assets != 0 => retAssets == assets`	✅	🎯
LL-UT-16	`utLlBorrowMovesExactTokens`	A borrow hands the receiver exactly the assets it returns out of the pool, and an assets-denominated request is paid in full. `receiver != pool: loanBal[receiver] += retAssets, loanBal[pool] -= retAssets, assets != 0 => retAssets == assets`	✅	🎯
LL-UT-17	`utLlRepayMovesExactTokens`	Repay pulls exactly the repaid assets from the payer into the pool, an assets request verbatim. `no callback token moves: loanBal[pool]' == loanBal[pool] + retAssets AND loanBal[msg.sender]' == loanBal[msg.sender] - retAssets AND (assets != 0 => retAssets == assets)`	✅	🎯
LL-UT-18	`utLlSupplyCollateralMovesExactTokens`	A collateral deposit pulls exactly the deposited collateral from the payer into the pool. `no callback token moves: collBal[pool]' == collBal[pool] + assets AND collBal[msg.sender]' == collBal[msg.sender] - assets`	✅	🎯
LL-UT-19	`utLlWithdrawCollateralMovesExactTokens`	A collateral withdrawal pays the receiver exactly the requested collateral out of the pool. `receiver != pool: collBal[receiver]' == collBal[receiver] + assets AND collBal[pool]' == collBal[pool] - assets`	✅	🎯

LL-UT-20	`utLlLiquidateMovesExactTokens`	Liquidation settles both legs exactly; repaid loan tokens in, seized collateral out. `loanToken != collateralToken, no callback token moves: loanBal[pool]' == loanBal[pool] + retRepaid AND loanBal[msg.sender]' == loanBal[msg.sender] - retRepaid AND collBal[pool]' == collBal[pool] - retSeized AND collBal[msg.sender]' == collBal[msg.sender] + retSeized AND (seizedAssets != 0 => retSeized == seizedAssets)`	✅	🎯
LL-UT-21	`utLlSettleMovesExactTokens`	Settle pays the receiver exactly the seized collateral out of the pool and moves no loan token. `receiver != pool, loanToken != collateralToken: collBal[receiver]' == collBal[receiver] + retSeized AND collBal[pool]' == collBal[pool] - retSeized AND loanBal[pool]' == loanBal[pool]`	✅	🎯
LL-UT-22	`utLlAccrueIdleMarketMovesOnlyClock`	On a market with no interest model, accruing interest moves only the clock. `irm == 0: accrueInterest(): totals AND fee-sink position unchanged AND lastUpdate' == block.timestamp`	✅	🎯
LL-UT-23	`utLlAccrueZeroDebtLeavesTotals`	With nothing borrowed, accruing interest leaves every total and position unchanged. `totalBorrowAssets == 0: accrueInterest(): totals AND fee-sink position unchanged`	✅	🎯

Accrue Interest

Identification	Name	Description	Properties	Mutations
LL-AI-01	`aiLlSupplyAccrualIdempotent`	An explicit accrueInterest right before supply leaves storage identical to supply alone. `storage(accrueInterest(mp); supply(args)) == storage(supply(args))`	✅	🎯
LL-AI-02	`aiLlWithdrawAccrualIdempotent`	An explicit accrueInterest right before withdraw leaves storage identical to withdraw alone. `storage(accrueInterest(mp); withdraw(args)) == storage(withdraw(args))`	✅	🎯
LL-AI-03	`aiLlBorrowAccrualIdempotent`	An explicit accrueInterest right before borrow leaves storage identical to borrow alone. `storage(accrueInterest(mp); borrow(args)) == storage(borrow(args))`	✅	🎯
LL-AI-04	`aiLlRepayAccrualIdempotent`	An explicit accrueInterest right before repay leaves storage identical to repay alone. `storage(accrueInterest(mp); repay(args)) == storage(repay(args))`	✅	🎯
LL-AI-05	`aiLlSetFeeAccrualIdempotent`	An explicit accrueInterest right before setFee leaves storage identical to setFee alone. `storage(accrueInterest(mp); setFee(mp, newFee)) == storage(setFee(mp, newFee))`	✅	🎯
LL-AI-06	`aiLlWithdrawCollateralAccrualIdempotent`	An explicit accrueInterest right before withdrawCollateral leaves storage identical to it alone. `storage(accrueInterest(mp); withdrawCollateral(args)) == storage(withdrawCollateral(args))`	✅	🎯
LL-AI-07	`aiLlSettleAccrualIdempotent`	An explicit accrueInterest right before settle leaves storage identical to settle alone. `storage(accrueInterest(mp); settle(args)) == storage(settle(args))`	✅	🎯
LL-AI-08	`aiLlLiquidateAccrualIdempotent`	An explicit accrueInterest right before liquidate leaves storage identical to liquidate alone. `storage(accrueInterest(mp); liquidate(args)) == storage(liquidate(args))`	✅	🎯
LL-AI-09	`aiLlAccrueTwiceIdempotent`	Accruing interest twice in one block is the same as accruing once. Killed by a clock stamped with the elapsed window instead of the current time, which makes a second accrual in the same block land somewhere else. `storage(accrueInterest(mp); accrueInterest(mp)) == storage(accrueInterest(mp))`	✅	🎯

Reachability

Identification	Name	Description	Properties	Mutations
LL-RC-01	`rcLlSetOwnerIsReachable`	The owner rotates ownership to a new owner. `setOwner(newOwner): satisfy(owner' == newOwner AND newOwner != owner)`	✅	🎯
LL-RC-02	`rcLlSetFeeRecipientIsReachable`	The owner installs a new fee recipient. `setFeeRecipient(r): satisfy(feeRecipient' == r AND r != feeRecipient)`	✅	🎯
LL-RC-03	`rcLlSetFeeIsReachable`	The owner moves a created market's fee. `setFee(newFee): satisfy(fee' == newFee AND fee' != fee)`	✅	🎯
LL-RC-04	`rcLlSupplyIsReachable`	A supplier's assets-anchored deposit mints positive shares. `supply(assets, 0, u): satisfy(retShares > 0 AND supplyShares[u]' > supplyShares[u])`	✅	🎯
LL-RC-05	`rcLlSupplySharesAnchoredIsReachable`	A shares-anchored supply mints exactly the requested shares. `supply(0, shares, u): satisfy(shares > 0 AND retAssets > 0 AND retShares == shares)`	✅	🎯

LL-RC-06	`rcLlSupplyBootstrapIsReachable`	The first supplier bootstraps an empty market's share supply. `totalSupplyShares == 0: supply(assets, 0, u): satisfy(retShares > 0 AND supplyShares[u]' == totalSupplyShares')`	✅	🎯
LL-RC-07	`rcLlSupplyControlledWithCallbackIsReachable`	Supply through the controller hook and the supply callback succeeds. `marketController != 0 AND data.length > 0: satisfy(retShares > 0 AND supplyShares[u]' > supplyShares[u])`	✅	🎯
LL-RC-08	`rcLlWithdrawIsReachable`	An assets-anchored withdraw pays out and burns supply shares. `u == msg.sender: withdraw(assets, 0, u): satisfy(retAssets > 0 AND supplyShares[u]' < supplyShares[u])`	✅	🎯
LL-RC-09	`rcLlWithdrawByDelegateIsReachable`	An authorized delegate withdraws on behalf of the owner. `u != msg.sender: withdraw(assets, 0, u): satisfy(retShares > 0 AND supplyShares[u]' < supplyShares[u])`	✅	🎯
LL-RC-10	`rcLlWithdrawAllLeavesAssetDustIsReachable`	The sole supplier's full exit leaves zero shares with asset dust. `shares == totalSupplyShares == supplyShares[u] > 0: satisfy(supplyShares[u]' == 0 AND totalSupplyShares' == 0 AND totalSupplyAssets' > 0)`	✅	🎯
LL-RC-11	`rcLlBorrowIsReachable`	An assets-anchored borrow mints debt shares against collateral. `u == msg.sender: borrow(assets, 0, u): satisfy(retShares > 0 AND borrowShares[u]' > borrowShares[u])`	✅	🎯
LL-RC-12	`rcLlRepayIsReachable`	An assets-anchored repay strictly shrinks the borrower's debt. `borrowShares[u] > 0: repay(assets, 0, u): satisfy(retShares > 0 AND borrowShares[u]' < borrowShares[u])`	✅	🎯
LL-RC-13	`rcLlRepayAllLeavesAssetResidueIsReachable`	A full repay leaves zero borrow shares with residual borrow assets. `shares == totalBorrowShares == borrowShares[u] > 0: satisfy(borrowShares[u]' == 0 AND totalBorrowShares' == 0 AND totalBorrowAssets' > 0)`	✅	🎯
LL-RC-14	`rcLlSupplyCollateralIsReachable`	A collateral deposit credits exactly the deposited assets. `supplyCollateral(assets, u): satisfy(assets > 0 AND collateral[u]' == collateral[u] + assets)`	✅	🎯
LL-RC-15	`rcLlWithdrawCollateralWithLiveDebtIsReachable`	A debtor withdraws excess collateral past the active health check. `u == msg.sender AND borrowShares[u] != 0: satisfy(collateral[u]' < collateral[u])`	✅	🎯
LL-RC-16	`rcLlWithdrawCollateralFullExitIsReachable`	A debt-free holder exits its full collateral down to zero. `borrowShares[u] == 0 AND assets == collateral[u] > 0: satisfy(collateral[u]' == 0)`	✅	🎯 🎯
LL-RC-17	`rcLlSettleSupplyAnchoredIsReachable`	A supply-anchored settle burns lender shares and seizes collateral. `supplyShares[u] > 0: settle(supplyShares, 0): satisfy(supplySharesBurned > 0 AND seizedAssets > 0 AND supplyShares[u]' < supplyShares[u])`	✅	🎯
LL-RC-18	`rcLlSettleBorrowAnchoredIsReachable`	A borrow-anchored settle cancels borrower debt shares. `settle(0, borrowShares): satisfy(borrowSharesCanceled > 0 AND borrowShares[b]' < borrowShares[b])`	✅	🎯
		A seize-anchored liquidation reduces collateral and		

LL-RC-19	`rcLlLiquidateSeizeAnchoredIsReachable`	debt. `liquidate(b, seizedAssets, 0): satisfy(seized > 0 AND collateral[b]' < collateral[b] AND borrowShares[b]' < borrowShares[b])`	✅	🎯
LL-RC-20	`rcLlLiquidateRepaidAnchoredIsReachable`	A repaid-shares-anchored liquidation seizes collateral and cuts debt. `collateral[b] > 0: liquidate(b, 0, repaidShares): satisfy(seized > 0 AND borrowShares[b]' < borrowShares[b])`	✅	🎯
LL-RC-21	`rcLlLiquidateBadDebtIsReachable`	A bad-debt liquidation zeroes the position and socializes the loss. `liquidate(b, seizedAssets, 0): satisfy(collateral[b]' == 0 AND borrowShares[b]' == 0 AND totalSupplyAssets' < totalSupplyAssets)`	✅	🎯
LL-RC-22	`rcLlFlashLoanIsReachable`	A nonzero flash loan runs and leaves LidoLend's token balance exactly where it started. `balance[token][LidoLend] >= assets > 0: flashLoan(token, assets): satisfy(balance[token][LidoLend]' == balance[token][LidoLend])`	✅	🎯
LL-RC-23	`rcLlSetAuthorizationGrantIsReachable`	A user grants authorization to a delegate. `setAuthorization(d, true): satisfy(isAuthorized[msg.sender][d]')`	✅	🎯
LL-RC-24	`rcLlAccrueInterestGrowsDebtIsReachable`	Interest accrual grows borrow and supply assets together. `irm != 0 AND totalBorrowAssets > 0 AND lastUpdate < now AND borrowRate > 0: satisfy(totalBorrowAssets' > totalBorrowAssets AND totalSupplyAssets' > totalSupplyAssets)`	✅	🎯
LL-RC-25	`rcLlAccrueInterestMintsFeeSharesIsReachable`	Accrual with a nonzero fee mints fee shares to the recipient. `irm != 0 AND totalBorrowAssets > 0 AND lastUpdate < now AND borrowRate > 0 AND fee > 0: satisfy(supplyShares[feeRecipient]' > supplyShares[feeRecipient])`	✅	🎯
LL-RC-26	`rcLlSupplyWithdrawRoundTripIsReachable`	A supplier enters and fully exits within one block. `u == msg.sender: (a1, s1) = supply(assets, 0, u); (a2, s2) = withdraw(0, s1, u): satisfy(s1 > 0 AND a2 > 0)`	✅	🎯
LL-RC-27	`rcLlSetAuthorizationRevokeIsReachable`	A user revokes a previously granted authorization. `isAuthorized[msg.sender][d]: setAuthorization(d, false): satisfy(!isAuthorized[msg.sender][d]')`	✅	🎯
LL-RC-28	`rcLlBorrowSharesAnchoredIsReachable`	A shares-anchored borrow mints exactly the requested debt shares. `u == msg.sender: borrow(0, shares, u): satisfy(shares > 0 AND retAssets > 0 AND retShares == shares)`	✅	🎯
LL-RC-29	`rcLlSetFeeToZeroIsReachable`	The owner clears a market's positive fee down to zero. `fee > 0: setFee(0): satisfy(fee' == 0)`	✅	🎯
LL-RC-30	`rcLlWithdrawAllWithLiveDebtIsReachable`	A lender can withdraw its whole position while the market still carries debt and another lender stays invested. `supplyShares[o] > 0 AND totalSupplyShares > supplyShares[u] AND totalBorrowAssets > 0 AND lastUpdate == now AND balance[loanToken][LidoLend] >= totalSupplyAssets: withdraw(0, supplyShares[u], u): satisfy(retAssets > 0 AND supplyShares[u]' == 0 AND totalBorrowAssets' > 0 AND supplyShares[o]' > 0)`	✅	🎯
LL-RC-31	`rcLlSettleOnHealthyBorrowerIsReachable`	Settle can cancel debt on a borrower still inside its liquidation threshold; LidoLend itself runs no health gate. `borrowed <= maxBorrow (healthy): settle(0, borrowShares): satisfy(borrowSharesCanceled > 0)`	✅	🎯

LL-RC-32	`rcLlSettleOnUnhealthyBorrowerIsReachable`	Settle cancels debt just the same on an under-water borrower; keeping bad debt out is the controller's job. `borrowed > maxBorrow (under water): settle(0, borrowShares): satisfy(borrowSharesCanceled > 0)`	✅	🎯
LL-RC-33	`rcLlRepayByUnauthorizedThirdPartyIsReachable`	Anyone can clear another account's debt; repay runs no authorization gate. `u != msg.sender AND !isAuthorized[u][msg.sender]: repay(assets, 0, u): satisfy(retShares > 0 AND borrowShares[u]' < borrowShares[u])`	✅	🎯
LL-RC-34	`rcLlWithdrawToExactZeroLiquidityIsReachable`	A withdraw may take a market with live debt down to exactly zero free liquidity; the liquidity check is at most, not strictly below. `lastUpdate == now AND totalBorrowAssets > 0 AND assets == totalSupplyAssets - totalBorrowAssets: withdraw(assets, 0, u): satisfy(retAssets > 0 AND totalSupplyAssets' == totalBorrowAssets' > 0)`	✅	🎯
LL-RC-35	`rcLlBorrowThenSettleUnlocksSettleableSupplyIsReachable`	Borrowing can open the settle door: a holding fully served by free liquidity becomes settleable once a borrow drains it. `ownedDown(onBehalf) <= totalSupplyAssets - totalBorrowAssets: borrow(...); settle(supplyShares, 0): satisfy(borrowAssetsCanceled > 0)`	✅	🎯

Reverts

Identification	Name	Description	Properties	Mutations
LL-RV-01	`rvLlSetOwnerRevertsIff`	Ownership changes hands exactly when the current owner names a different address. `REVERTS <=> (msg.sender != owner OR newOwner == owner)`	✅	🎯
LL-RV-02	`rvLlSetFeeRevertsWhenNotOwner`	Only the owner can change the market fee. `msg.sender != owner => REVERTS`	✅	🎯
LL-RV-03	`rvLlSetFeeRevertsWhenFeeUnchanged`	Setting the market fee to the value it already holds is rejected. `newFee == fee => REVERTS`	✅	🎯
LL-RV-04	`rvLlSetFeeRevertsWhenAboveMaxFee`	The market fee can never be raised above the contract's own cap. `newFee > MAX_FEE => REVERTS`	✅	🎯
LL-RV-05	`rvLlSetFeeSucceedsOnValidInput`	With interest already up to date, the owner can always set a changed fee that stays within the cap. `msg.sender == owner AND newFee != fee AND newFee <= MAX_FEE AND lastUpdate == block.timestamp => !REVERTS`	✅	🎯
LL-RV-06	`rvLlSetFeeRecipientRevertsIff`	The fee recipient changes exactly when the owner names a different one. `REVERTS <=> (msg.sender != owner OR newFeeRecipient == feeRecipient)`	✅	🎯
LL-RV-07	`rvLlSupplyRevertsWhenInconsistentInput`	A supply must be denominated in either assets or shares, never both and never neither. `(assets == 0) == (shares == 0) => REVERTS`	✅	🎯
LL-RV-08	`rvLlSupplyRevertsWhenZeroOnBehalf`	Nobody can supply on behalf of the zero address. `onBehalf == 0 => REVERTS`	✅	🎯
LL-RV-09	`rvLlWithdrawRevertsWhenInconsistentInput`	A withdrawal must be denominated in either assets or shares, never both and never neither. `(assets == 0) == (shares == 0) =>`	✅	🎯

		`REVERTS`		
LL-RV-10	`rvLlWithdrawRevertsWhenZeroReceiver`	A withdrawal can never pay out to the zero address. `receiver == 0 => REVERTS`	✅	🎯
LL-RV-11	`rvLlWithdrawRevertsWhenUnauthorized`	Only the position holder, or someone it authorized, can withdraw its supply. `msg.sender != onBehalf AND !isAuthorized[onBehalf][msg.sender] => REVERTS`	✅	🎯
LL-RV-12	`rvLlBorrowRevertsWhenInconsistentInput`	A borrow must be denominated in either assets or shares, never both and never neither. `(assets == 0) == (shares == 0) => REVERTS`	✅	🎯
LL-RV-13	`rvLlBorrowRevertsWhenZeroReceiver`	A borrow can never pay out to the zero address. `receiver == 0 => REVERTS`	✅	🎯
LL-RV-14	`rvLlBorrowRevertsWhenUnauthorized`	Only the position holder, or someone it authorized, can borrow against it. `msg.sender != onBehalf AND !isAuthorized[onBehalf][msg.sender] => REVERTS`	✅	🎯
LL-RV-15	`rvLlRepayRevertsWhenInconsistentInput`	A repayment must be denominated in either assets or shares, never both and never neither. `(assets == 0) == (shares == 0) => REVERTS`	✅	🎯
LL-RV-16	`rvLlRepayRevertsWhenZeroOnBehalf`	Nobody can repay on behalf of the zero address. `onBehalf == 0 => REVERTS`	✅	🎯
LL-RV-17	`rvLlSupplyCollateralRevertsWhenZeroAssets`	Posting zero collateral is rejected. `assets == 0 => REVERTS`	✅	🎯
LL-RV-18	`rvLlSupplyCollateralRevertsWhenZeroOnBehalf`	Nobody can post collateral on behalf of the zero address. `onBehalf == 0 => REVERTS`	✅	🎯
LL-RV-19	`rvLlWithdrawCollateralRevertsWhenZeroAssets`	Withdrawing zero collateral is rejected. `assets == 0 => REVERTS`	✅	🎯
LL-RV-20	`rvLlWithdrawCollateralRevertsWhenZeroReceiver`	A collateral withdrawal can never pay out to the zero address. `receiver == 0 => REVERTS`	✅	🎯
LL-RV-21	`rvLlWithdrawCollateralRevertsWhenUnauthorized`	Only the position holder, or someone it authorized, can pull its collateral out. `msg.sender != onBehalf AND !isAuthorized[onBehalf][msg.sender] => REVERTS`	✅	🎯
LL-RV-22	`rvLlSettleRevertsWhenSettlementDisabled`	A market that has not enabled settlement can never be settled. `!isSettlementEnabled => REVERTS`	✅	🎯
LL-RV-23	`rvLlSettleRevertsWhenInconsistentInput`	A settlement must be anchored to either supply shares or borrow shares, never both and never neither. `(supplySharesIn == 0) == (borrowSharesIn == 0) => REVERTS`	✅	🎯
LL-RV-24	`rvLlSettleRevertsWhenZeroReceiver`	A settlement can never pay out to the zero address. `receiver == 0 => REVERTS`	✅	🎯
LL-RV-25	`rvLlSettleRevertsWhenUnauthorized`	Only the position holder, or someone it authorized, can settle on its behalf. `msg.sender != onBehalf AND !isAuthorized[onBehalf][msg.sender] => REVERTS`	✅	🎯

LL-RV-26	`rvLlSettleRevertsOnBorrowAnchoredNoSupply`	Cancelling a borrower's debt requires the supplier absorbing the loss to actually hold a supply position. `onBehalf != feeRecipient AND supplySharesIn == 0 AND borrowSharesIn > 0 AND supplyShares[onBehalf] == 0 => REVERTS`	✅	🎯
LL-RV-27	`rvLlSettleRevertsWhenBorrowAnchoredExceedsDebt`	A borrow-anchored settlement can never cancel more debt than the borrower actually owes. `supplySharesIn == 0 AND borrowSharesIn > borrowShares[borrower] => REVERTS`	✅	🎯
LL-RV-28	`rvLlLiquidateRevertsWhenInconsistentInput`	A liquidation must be denominated in either seized collateral or repaid shares, never both and never neither. `(seizedAssets == 0) == (repaidShares == 0) => REVERTS`	✅	🎯
LL-RV-29	`rvLlFlashLoanRevertsIff`	A flash loan goes through exactly when it asks for a non-zero amount; token transfers themselves are assumed to succeed. `token transfers AND callback assumed to succeed: REVERTS <=> assets == 0`	✅	🎯
LL-RV-30	`rvLlSetAuthorizationRevertsIff`	Granting or revoking an authorization goes through exactly when it actually changes something. `REVERTS <=> newIsAuthorized == isAuthorized[msg.sender][authorized]`	✅	🎯
LL-RV-31	`rvLlAccrueInterestRevertsWhenHookInProgress`	The market cannot be re-entered while one of its controller hooks is still running; shown on accrueInterest, the same gate guards every hooked action. `hookInProgress => REVERTS`	✅	🎯
LL-RV-32	`rvLlSettleRevertsWhenIdleCoversTheSlice`	A settle stands down when the market's own free cash could have paid out the slice being cancelled. `lastUpdate == now, borrowSharesIn == 0, supplySharesIn > 0: sliceDown(supplySharesIn) <= free => REVERTS [free := totalSupplyAssets − totalBorrowAssets, sliceDown(s) := s * (totalSupplyAssets + 1) / (totalSupplyShares + 1e6)]`	❌ → ✖	Issue #42 (MEDIUM)

Access Control

Identification	Name	Description	Properties	Mutations
LL-AC-01	`acLlOnlyOwnerOrInitLatchMovesOwner`	The owner changes only by the owner or through the one-shot initializer. `owner' != owner => msg.sender == owner OR (!isInitialized AND isInitialized')`	✅	🎯
LL-AC-02	`acLlOnlyOwnerMovesFeeRecipient`	The fee recipient changes only when the caller was the owner. `feeRecipient' != feeRecipient => msg.sender == owner`	✅	🎯
LL-AC-03	`acLlOnlyOwnerMovesMarketFee`	A market's fee changes only when the caller was the owner. `market[id].fee' != market[id].fee => msg.sender == owner`	✅	🎯
LL-AC-04	`acLlUnauthorizedCannotReduceSupplyShares`	An unauthorized caller cannot reduce a user's supply shares. `msg.sender != user AND !isAuthorized[user][msg.sender] => supplyShares[user]' >= supplyShares[user]`	✅	🎯
LL-AC-05	`acLlUnauthorizedCannotIncreaseBorrowShares`	An unauthorized caller cannot increase a user's borrow shares. `msg.sender != user AND !isAuthorized[user][msg.sender] => borrowShares[user]' <= borrowShares[user]`	✅	🎯
LL-AC-06	`acLlAuthorizationRowMovesOnlyByAuthorizer`	An authorization is granted or revoked only by the user it belongs to, outside the signature-based grant path. `signature grant path out of scope: isAuthorized[authorizer][authorized]' != isAuthorized[authorizer][authorized] => msg.sender == authorizer`	✅	🎯
LL-AC-07	`acLlSeizureOnlyByLiquidateOrSettle`	A non-consenting user's collateral decreases only through liquidate or settle. `collateral[user]' < collateral[user] AND msg.sender != user AND !isAuthorized[user][msg.sender] => f in {liquidate, settle}`	✅	🎯
LL-AC-08	`acLlDebtFreeCollateralNeverSeized`	A debt-free user's collateral is never seized by an unauthorized caller. `msg.sender != user AND !isAuthorized[user][msg.sender] AND borrowShares[user] == 0 => collateral[user]' >= collateral[user]`	✅	🎯🎯

LidoLendERC4626

- The vault running its real bytecode; the core, the oracle, the interest model and the tokens linked as CVL models
- Core reads answered by a stub that does not accrue, so the pending-interest regime is uncovered
- Token ledgers holding up to 7 modeled accounts each, decimals drawn between 6 and 18
- Loops running up to 3 iterations
- The vault taken as deployed and initialized, the wiring step itself out of scene
- The vault's share supply capped at uint128 in the conversion, preview and never-revert laws, so supply above that band is uncovered

Valid State

Identification	Name	Description	Properties	Mutations
VA-VS-01	`assetIsLoanToken`	The ERC4626 asset is the market loan token. `asset() == params.loanToken`	⏱	🎯
VA-VS-02	`marketIdCanonical`	The vault's market id is exactly the one its stored parameters derive. `controller != 0 => marketId == id(loanToken, collateralToken, oracle, irm, lltv)`	⏱	🎯🎯
VA-VS-03	`offsetBounded`	The virtual-share offset never tops the loan token past the 18-decimal share standard. `loanDecimalsOffset <= 18`	⏱	🎯

Variable Transitions

Identification	Name	Description	Properties	Mutations
VA-VT-01	`vtVaMarketIdentityImmutable`	Once set at initialization, the vault's controller, market and token pair are locked for life. `marketId' == marketId AND controller' == controller AND loanToken' == loanToken AND collateralToken' == collateralToken`	⏱	🎯🎯
VA-VT-02	`vtVaMarketParamsImmutable`	The stored market parameters and the decimals offset never change. `oracle' == oracle AND irm' == irm AND lltv' == lltv AND loanDecimalsOffset' == loanDecimalsOffset`	⏱	🎯

State Transitions

Identification	Name	Description	Properties	Mutations
VA-ST-01	`stVaAllowanceSpendOnly`	Third parties only spend an allowance down. `allowance[o][s]' > allowance[o][s] => o == msg.sender`	⏱	🎯
VA-ST-02	`stVaThirdPartyDebitIsAuthorized`	Shares never leave a holder's hands unless the holder acts or granted the caller an allowance. `(balanceOf[a]' < balanceOf[a] AND a != msg.sender) => allowance[a][msg.sender] != 0`	⏱	🎯

High Level

Identification	Name	Description	Properties	Mutations
VA-HL-01	`hlVaDepositLeavesOtherBalances`	A deposit moves no share balance other than the receiver's. `other != receiver => balanceOf[other]' == balanceOf[other]`	✅	🎯
VA-HL-02	`hlVaRedeemLeavesOtherBalances`	A redeem moves no share balance other than the owner's. `other != owner => balanceOf[other]' == balanceOf[other]`	✅	🎯
VA-HL-03	`hlVaSettleSpendsAllowanceExactly`	Settle spends the owner's allowance by exactly the burned shares; infinite approval is untouched. `msg.sender != owner, a = allowance[owner][msg.sender]; a' == (a == max_uint256 ? max_uint256 : a - sharesBurned)`	✅	🎯
VA-HL-04	`hlVaDepositRedeemRoundTripNoProfit`	A deposit-redeem round trip never returns more assets than were deposited. `shares = deposit(assets, receiver); redeem(shares, receiver, receiver) <= assets`	⏱	🎯
VA-HL-05	`hlVaPreviewRoundingFavorsVault`	Exit rounding never undercuts entry; previewWithdraw and previewMint round toward the vault. `previewWithdraw(assets) >= previewDeposit(assets) AND previewMint(shares) >= previewRedeem(shares)`	✅	🎯
VA-HL-06	`hlVaWithdrawBurnsAtMostPreview`	A withdraw burns at most the pre-state previewWithdraw quote. `balanceOf[owner] - balanceOf[owner]' <= previewWithdraw(assets)`	✅	🎯🎯🎯
VA-HL-07	`hlVaMintRedeemNoProfit`	Cycling exact shares through mint then redeem never returns more assets than they cost. `assetsIn = mint(shares, receiver); redeem(shares, receiver, receiver) <= assetsIn`	⏱	🎯🔻
VA-HL-08	`hlVaSupplyMovesOnlyOnFlows`	Shares are created only by the deposit entry points and destroyed only by the exit ones. `totalSupply' > totalSupply => f in {deposit, mint}; totalSupply' < totalSupply => f in {withdraw, redeem, settle, fallbackExit}`	⏱	🎯
VA-HL-09	`hlVaSettleBurnsAtMostRequestedShares`	Settle burns at most the shares the caller asked for, and the whole burn lands on the owner. `(sharesBurned, ...) = settle(shares, borrowers, receiver, owner): sharesBurned <= shares AND balanceOf[owner]' == balanceOf[owner] - sharesBurned`	⏱	🎯
VA-HL-10	`hlVaSettleBurnsProRataOfConsumedBudget`	Settle burns the request scaled by the core supply-share budget the cuts consumed, rounded up. `budget := posSupplyShares[id][vault] * shares / totalSupply, consumed := posSupplyShares[id][vault] - posSupplyShares[id][vault]': budget != 0 AND consumed <= budget => sharesBurned == ceil(shares * consumed / budget)`	⏱	🎯
VA-HL-11	`hlVaFallbackExitBurnsProRataOfConsumedBudget`	With no free liquidity the fallback exit burns pro rata of the consumed core budget, rounded up. `maxWithdrawableAssets(vault) == 0 AND previewRedeem(shares) != 0, budget := posSupplyShares[id][vault] * shares / totalSupply, consumed := posSupplyShares[id][vault] - posSupplyShares[id][vault]': budget != 0 AND consumed <= budget => sharesBurned == ceil(shares * consumed / budget)`	⏱	🎯

VA-HL-12	`hlVaSettleDoesNotDiluteRemainingHolders`	Settle never dilutes the holders who stay; the core lending position backing each remaining share never falls. `budget := posSupplyShares[id][vault] * shares / totalSupply, consumed := posSupplyShares[id][vault] - posSupplyShares[id][vault]': budget != 0 AND consumed <= budget => posSupplyShares[id][vault]' * totalSupply >= posSupplyShares[id][vault] * totalSupply'`	⏱	🎯
VA-HL-13	`hlVaSettlePaysSeizedCollateralToReceiver`	Settle hands the seized collateral to the caller-named receiver and keeps none of it. `receiver != LIDO_LEND: collateralBal[receiver]' == collateralBal[receiver] + collateralSeized AND (receiver != vault => collateralBal[vault]' == collateralBal[vault])`	✅	🎯
VA-HL-14	`hlVaFallbackExitPaysSeizedCollateralToReceiver`	The fallback exit hands the seized collateral to the caller-named receiver and keeps none. `receiver != LIDO_LEND AND collateralToken != loanToken: collateralBal[receiver]' == collateralBal[receiver] + collateralSeized AND (receiver != vault => collateralBal[vault]' == collateralBal[vault])`	⏱	🎯🔻
VA-HL-15	`hlVaSettleTouchesOnlyNamedBorrowers`	Settle cuts only the borrowers the caller named. `borrowers.length == 2, other != borrowers[0] AND other != borrowers[1]: borrowShares[other]' == borrowShares[other] AND collateral[other]' == collateral[other]`	✅	🎯
VA-HL-16	`hlVaFallbackExitSettleLegRespectsIterationCap`	The fallback exit's settlement sweep stops at the caller's iteration cap; one iteration never cuts two borrowers. `maxIterations == 1, b1 != b2: (borrowShares[b1]' == borrowShares[b1] AND collateral[b1]' == collateral[b1]) OR (borrowShares[b2]' == borrowShares[b2] AND collateral[b2]' == collateral[b2])`	✅	🎯
VA-HL-17	`hlVaFallbackExitDoesNotDiluteRemainingHolders`	The fallback exit never dilutes the holders who stay; the core lending position backing each remaining share never falls. `maxWithdrawableAssets(vault) == 0 AND previewRedeem(shares) != 0, budget := posSupplyShares[id][vault] * shares / totalSupply, consumed := posSupplyShares[id][vault] - posSupplyShares[id][vault]': budget != 0 AND consumed <= budget => posSupplyShares[id][vault]' * totalSupply >= posSupplyShares[id][vault] * totalSupply'`	⏱	🎯🔻
VA-HL-18	`hlVaFallbackExitBurnsAgainstPreviewedBudget`	The exit burns the request scaled by the consumed slice of previewFallbackExit's quoted budget. `maxWithdrawableAssets(vault) == 0, (_, quotedSupplyShares) = previewFallbackExit(shares), consumed := posSupplyShares[id][vault] - posSupplyShares[id][vault]': quotedSupplyShares != 0 AND consumed <= quotedSupplyShares => sharesBurned == ceil(shares * consumed / quotedSupplyShares)`	⏱	🎯
VA-HL-19	`hlVaSettleCutsTheNamedBorrower`	The collateral settle reports as seized comes out of the named borrower's own position. `borrowers.length == 1: collateral[borrowers[0]]' == collateral[borrowers[0]] -`	✅	🎯

		`collateralSeized`		
VA-HL-20	`hlVaPreviewFallbackExitQuotesLiquidityAndProRataBudget`	Previewing a fallback exit quotes the shares' value capped by free liquidity, plus their pro-rata slice of the vault's core position. `previewFallbackExit(shares) == (min(convertToAssets(shares), maxWithdrawableAssets(vault)), position[id][vault].supplyShares * shares / totalSupply)`	✅	🎯🎯
VA-HL-21	`hlVaDepositMintsAtLeastPreview`	A deposit mints the receiver at least the pre-state previewDeposit quote. `balanceOf[receiver]' - balanceOf[receiver] >= previewDeposit(assets)`	✅	🎯
VA-HL-22	`hlVaMintChargesAtMostPreview`	A mint charges the caller at most the pre-state previewMint quote. `assetBal[caller] - assetBal[caller]' <= previewMint(shares)`	✅	🎯
VA-HL-23	`hlVaRedeemPaysAtLeastPreview`	A redeem pays the receiver at least the pre-state previewRedeem quote. `receiver not in {vault, LIDO_LEND} => assetBal[receiver]' - assetBal[receiver] >= previewRedeem(shares)`	✅	🎯
VA-HL-24	`hlVaMaxRedeemIsWholeBalanceWhenLiquidityCovers`	An exit that free liquidity fully covers is quoted as the owner's exact share balance. `bal := shareBal[owner]: previewRedeem(bal) <= maxWithdrawableAssets(vault) => maxRedeem(owner) == bal`	✅	🎯
VA-HL-25	`hlVaFallbackExitDeliversPreviewedLoanAssetsWhenLiquidCovers`	REGION A (previewRedeem(shares) <= freeLiquidity), where the liquid leg pays the whole exit:. `previewRedeem(shares) <= maxWithdrawableAssets(vault): fallbackExit: loanAssets == previewFallbackExit(shares).loanAssets`	⏱	🎯🔻
VA-HL-26	`hlVaFallbackExitDeliversPreviewedLoanAssetsWhenLiquidPartial`	REGION B (liquidity is positive but short), where the liquid leg pays exactly what is free. `0 < maxWithdrawableAssets(vault) < previewRedeem(shares): fallbackExit: loanAssets == maxWithdrawableAssets(vault)`	⏱	🎯🔻
VA-HL-27	`hlVaFallbackExitBurnsAtMostRequestedSharesWhenLiquidCovers`	REGION A, the burn bound: the liquid leg redeems the whole request and the settle leg does not run, so the burn is the request itself. `previewRedeem(shares) <= maxWithdrawableAssets(vault): fallbackExit: sharesBurned == shares AND shareBal[owner]' == shareBal[owner] - sharesBurned`	✅	🎯🔻
VA-HL-28	`hlVaFallbackExitBurnsAtMostRequestedSharesWhenLiquidPartial`	REGION B, the burn bound. Both legs run, so this is where a settled slice scaled by the whole request instead of the remainder would burn past what was asked. `0 < maxWithdrawableAssets(vault) < previewRedeem(shares): fallbackExit: sharesBurned <= shares AND shareBal[owner]' == shareBal[owner] - sharesBurned`	⏱	🎯🔻
VA-HL-29	`hlVaFallbackExitBurnsAtMostRequestedSharesWhenNoLiquidity`	REGION C, the burn bound. Only the settle leg runs, so the burn is its settled slice alone. `maxWithdrawableAssets(vault) == 0 < previewRedeem(shares): fallbackExit: sharesBurned <= shares AND shareBal[owner]' == shareBal[owner] - sharesBurned`	⏱	🎯🔻

ERC4626

Identification	Name	Description	Properties	Mutations

ERC4626-01	`assetReturnsConfiguredToken`	The vault always reports the configured underlying ERC-20 token as its asset. `asset() == underlyingAsset`	✅	🎯
ERC4626-02	`assetNeverReverts`	The asset getter never reverts. `asset() never REVERTS`	✅	🎯
ERC4626-03	`totalAssetsReturnsManaged`	The vault reports no managed assets when it holds no lending position, and never more than the market has supplied in total. `(supplyShares[vault] == 0 => totalAssets() == 0) && totalAssets() <= totalSupplyAssets`	✅	🎯
ERC4626-04	`totalAssetsNeverReverts`	The totalAssets getter never reverts. `totalAssets() never REVERTS`	✅	🎯🎯
ERC4626-05	`convertToSharesSameForAnyCaller`	Everyone gets the same share quote; convertToShares never depends on who asks. `convertToShares(e1, assets) == convertToShares(e2, assets) (same state, same timestamp)`	✅	🎯
ERC4626-06	`convertToSharesNeverReverts`	Share conversion never reverts on realistic amounts. `assets < 2^128 && totalSupply <= max_uint128 - 10^18 => convertToShares(assets) never REVERTS`	✅	🎯
ERC4626-07	`sharesToAssetsRoundTripNeverExceeds`	convertToShares rounds down, per EIP-4626: converting assets to shares and back never yields more than the original amount. `convertToAssets(convertToShares(assets)) <= assets (assets < max_uint128)`	⏱	🎯
ERC4626-08	`convertToAssetsSameForAnyCaller`	Everyone gets the same asset quote; convertToAssets never depends on who asks. `convertToAssets(e1, shares) == convertToAssets(e2, shares) (same state, same timestamp)`	✅	🎯
ERC4626-09	`convertToAssetsNeverReverts`	Asset conversion never reverts on realistic amounts. `shares < 2^128 => convertToAssets(shares) never REVERTS`	✅	🎯
ERC4626-10	`assetsToSharesRoundTripNeverExceeds`	convertToAssets rounds down: converting shares to assets and back never yields more than the original amount. `convertToShares(convertToAssets(shares)) <= shares (shares < max_uint128)`	✅	🎯
ERC4626-11	`depositAboveMaxLimitReverts`	maxDeposit is an honest ceiling, per EIP-4626: a deposit above the advertised limit cannot slip through; it reverts. `assets > maxDeposit(receiver) => deposit(assets, receiver) REVERTS`	✅	🎯
ERC4626-12	`maxDepositNeverReverts`	The maxDeposit getter never reverts. `maxDeposit(receiver) never REVERTS`	✅	🎯
ERC4626-13	`mintAboveMaxLimitReverts`	maxMint is an honest ceiling: a mint of more shares than advertised reverts. `shares > maxMint(receiver) => mint(shares, receiver) REVERTS`	✅	🎯
ERC4626-14	`maxMintNeverReverts`	The maxMint getter never reverts. `totalSupply <= max_uint128 - 10^18 => maxMint(receiver) never REVERTS`	✅	🎯🎯
ERC4626-15	`withdrawAboveMaxLimitReverts`	The advertised maxWithdraw is an honest ceiling; withdrawing more assets than it allows always reverts. `assets > maxWithdraw(owner) => withdraw(assets, receiver, owner) REVERTS`	✅	🎯
ERC4626-16	`maxWithdrawNeverReverts`	The maxWithdraw getter never reverts. `maxWithdraw(owner) never REVERTS`	✅	🎯
ERC4626-17	`maxRedeemNeverReverts`	The maxRedeem getter is total under the share-supply band: it answers for any owner without reverting. `maxRedeem(owner) never REVERTS (shareTotalSupply < max_uint128 - 10^18)`	✅	🎯
ERC4626-18	`redeemAboveMaxLimitReverts`	The advertised maxRedeem is an honest ceiling; redeeming more shares than it allows always reverts. `shares > maxRedeem(owner) => redeem(shares, receiver, owner) REVERTS`	✅	🎯
ERC4626-19	`previewDepositIncludesFees`	previewDeposit is fee-inclusive, per EIP-4626: it quotes at most the ideal convertToShares result, so fees are never hidden from the quote. `previewDeposit(assets) <= convertToShares(assets) (assets < max_uint128)`	✅	🎯

ERC4626-20	`previewMintSameForAnyCaller`	Everyone gets the same previewMint quote; it never depends on who asks. `previewMint(e1, shares) == previewMint(e2, shares) (same state, same timestamp)`	✅	🎯
ERC4626-21	`previewMintIncludesFees`	previewMint is fee-inclusive: it quotes at least the ideal convertToAssets result, so the cost of minting is never understated. `previewMint(shares) >= convertToAssets(shares) (shares < max_uint128)`	✅	🎯
ERC4626-22	`previewWithdrawSameForAnyCaller`	Everyone gets the same previewWithdraw quote; it never depends on who asks. `previewWithdraw(e1, assets) == previewWithdraw(e2, assets) (same state, same timestamp)`	✅	🎯
ERC4626-23	`previewRedeemIncludesFees`	previewRedeem is fee-inclusive: it quotes at most the ideal convertToAssets result, so a redemption payout is never overstated. `previewRedeem(shares) <= convertToAssets(shares) (shares < max_uint128)`	✅	🎯
ERC4626-24	`depositTransfersAndMintsCorrectly`	A deposit charges the caller exactly the assets it names and mints the receiver exactly the shares it returns. `caller != LIDO_LEND => assetBal[caller]' == assetBal[caller] - assets AND shareBal[receiver]' == shareBal[receiver] + shares AND shareTotalSupply' == shareTotalSupply + shares`	✅	🎯🎯
ERC4626-25	`mintTransfersAndMintsCorrectly`	A mint gives the receiver exactly the shares it names and charges the caller exactly the assets it returns. `caller != LIDO_LEND => assetBal[caller]' == assetBal[caller] - assetsCharged AND shareBal[receiver]' == shareBal[receiver] + shares AND shareTotalSupply' == shareTotalSupply + shares`	✅	🎯🎯
ERC4626-26	`mintDecreasesAllowanceByAssetsSpent`	A mint spends exactly the charged assets from the caller's asset allowance. `assetAllowance[caller][vault] != max_uint256 => assetAllowance[caller][vault]' == assetAllowance[caller][vault] - assetsCharged`	✅	🎯
ERC4626-27	`withdrawBurnsAndTransfersCorrectly`	A withdraw burns exactly the shares it returns from the owner and pays the receiver exactly the assets it names. `shareBal[owner]' == shareBal[owner] - sharesBurned AND (receiver not in {vault, LIDO_LEND} => assetBal[receiver]' == assetBal[receiver] + assets) AND shareTotalSupply' == shareTotalSupply - sharesBurned`	✅	🎯🎯
ERC4626-28	`redeemBurnsAndTransfersCorrectly`	A redeem burns exactly the shares it names from the owner and pays the receiver exactly the assets it returns. `shareBal[owner]' == shareBal[owner] - shares AND (receiver not in {vault, LIDO_LEND} => assetBal[receiver]' == assetBal[receiver] + assetsOut) AND shareTotalSupply' == shareTotalSupply - shares`	✅	🎯
ERC4626-29	`withdrawRevertsOnInsufficientShareAllowance`	A third-party withdraw MUST revert when the share allowance cannot cover the burn. `caller != owner && shareAllowance[owner][caller] != max_uint256 && shareAllowance[owner][caller] < sharesBurned => withdraw REVERTS`	✅	🎯
ERC4626-30	`redeemRevertsOnInsufficientShareAllowance`	A third-party redeem MUST revert when the share allowance is below the shares. `caller != owner && shareAllowance[owner][caller] != max_uint256 && shareAllowance[owner][caller] < shares => redeem REVERTS`	✅	🎯
ERC4626-31	`withdrawDecrementsShareAllowanceCorrectly`	A third-party withdraw spends a finite share allowance by exactly the shares burned, and never touches an infinite one. `(shareAllowance[owner][caller] != max_uint256 => shareAllowance[owner][caller]' == shareAllowance[owner][caller] - sharesBurned) && (shareAllowance[owner][caller] == max_uint256 => shareAllowance[owner][caller]' == max_uint256)`	✅	🎯
		A third-party redeem spends a finite share allowance by exactly the shares redeemed, and never touches an infinite one. `(shareAllowance[owner][caller] != max_uint256 =>`		

ERC4626-32	`redeemDecrementsShareAllowanceCorrectly`	`shareAllowance[owner][caller]' == shareAllowance[owner][caller] - shares) && (shareAllowance[owner][caller] == max_uint256 => shareAllowance[owner][caller]' == max_uint256)`	✅	🎯
ERC4626-33	`donationCannotMoveSharePrice`	Donating tokens straight to the vault cannot move the share price, so redemption quotes resist donation attacks. `assetBal[vault] += donation => previewRedeem(shares)' == previewRedeem(shares)`	✅	🎯
ERC4626-34	`depositAtAdvertisedMaxDoesNotRevert`	The advertised maxDeposit is itself depositable; a deposit of exactly the limit goes through. `assets == maxDeposit(receiver) < 2^128 => deposit(assets, receiver) does not REVERT`	✅	🎯🎯
ERC4626-35	`mintAtAdvertisedMaxDoesNotRevert`	The advertised maxMint is itself mintable; a mint of exactly the limit goes through. `shares == maxMint(receiver) < 2^128 => mint(shares, receiver) does not REVERT`	✅	🎯
ERC4626-36	`withdrawAtAdvertisedMaxDoesNotRevert`	The advertised maxWithdraw is itself withdrawable; a self-exit of exactly the limit goes through. `msg.sender == owner, assets == maxWithdraw(owner) => withdraw(assets, receiver, owner) does not REVERT`	⏱	🎯
ERC4626-37	`redeemAtAdvertisedMaxDoesNotRevert`	The advertised maxRedeem is itself redeemable; a self-exit of exactly the limit goes through. `msg.sender == owner, shares == maxRedeem(owner) => redeem(shares, receiver, owner) does not REVERT`	⏱	🎯

Reachability

Identification	Name	Description	Properties	Mutations
VA-RC-01	`rcVaDepositIsReachable`	A deposit mints positive shares and pulls the depositor's loan tokens. `satisfy(shares > 0 AND shareBal[receiver]' > shareBal[receiver] AND loanBal[sender]' < loanBal[sender])`	✅	🎯
VA-RC-02	`rcVaWithdrawIsReachable`	A withdraw burns the owner's shares and delivers loan tokens to the receiver. `satisfy(shares > 0 AND shareBal[owner]' < shareBal[owner] AND loanBal[receiver]' > loanBal[receiver])`	✅	🎯
VA-RC-03	`rcVaRedeemIsReachable`	A redeem returns positive assets while burning the owner's shares. `satisfy(assetsOut > 0 AND shareBal[owner]' < shareBal[owner] AND loanBal[receiver]' > loanBal[receiver])`	✅	🎯
VA-RC-04	`rcVaSettleIsReachable`	A targeted settle burns a positive amount of the owner's shares. `satisfy(sharesBurned > 0 AND shareBal[owner]' == shareBal[owner] - sharesBurned)`	✅	🎯
VA-RC-05	`rcVaFallbackExitIsReachable`	The fallback exit burns a positive amount of the owner's shares. `satisfy(sharesBurned > 0 AND shareBal[owner]' == shareBal[owner] - sharesBurned)`	⏱	🎯
VA-RC-06	`rcVaFirstDepositIsReachable`	The first deposit bootstraps the share supply from zero. `totalSupply == 0; satisfy(shares > 0 AND totalSupply' == shares)`	✅	🎯
VA-RC-07	`rcVaFirstMintIsReachable`	The first mint bootstraps the share supply through the ceil-pricing path. `totalSupply == 0; satisfy(assets > 0 AND shares > 0 AND totalSupply' == shares)`	✅	🎯
VA-RC-08	`rcVaDepositWithExistingSharesIsReachable`	A deposit mints shares to the receiver under existing-supply proportional pricing. `totalSupply > 0; satisfy(shares > 0 AND shareBal[receiver]' == shareBal[receiver] + shares)`	✅	🎯
		A one-wei deposit mints a positive share amount and charges		

VA-RC-09	`rcVaDepositOneWeiIsReachable`	the depositor its wei. `assets == 1; satisfy(shares > 0 AND loanBal[sender]' == loanBal[sender] - 1)`	✅	🎯
VA-RC-10	`rcVaWithdrawMaxIsReachable`	A withdraw of the full maxWithdraw amount succeeds. `room := maxWithdraw(owner), room > 0; satisfy(withdraw(room).shares > 0)`	✅	🎯
VA-RC-11	`rcVaRedeemFullBalanceIsReachable`	A full-balance redeem zeroes the owner's shares and pays positive assets. `bal := shareBal[owner], bal > 0; satisfy(redeem(bal).assetsOut > 0 AND shareBal[owner]' == 0)`	✅	🎯
VA-RC-12	`rcVaFallbackExitFullBalanceIsReachable`	A full-balance fallback exit burns the owner's entire position. `bal := balanceOf[owner]; satisfy(fallbackExit(bal).sharesBurned > 0 AND balanceOf[owner]' == 0)`	⏱️	🎯
VA-RC-13	`rcVaSettleSingleBorrowerIsReachable`	A settle against exactly one named borrower burns a positive share amount. `borrowers.length == 1; satisfy(sharesBurned > 0)`	✅	🎯
VA-RC-14	`rcVaSettleMultipleBorrowersIsReachable`	A settle sweeping several named borrowers burns a positive share amount. `borrowers.length == 2; satisfy(sharesBurned > 0)`	✅	🎯
VA-RC-15	`rcVaFallbackExitLiquidLegIsReachable`	The fallback exit resolves through the liquid leg, delivering loan tokens. `satisfy(sharesBurned > 0 AND loanAssets > 0 AND loanBal[receiver]' > loanBal[receiver])`	⏱️	🎯
VA-RC-16	`rcVaFallbackExitSingleIterationIsReachable`	A fallback exit capped at one settlement iteration completes a settle cut. `maxIterations == 1; satisfy(sharesBurned > 0 AND (collateralSeized > 0 OR debtCanceled > 0))`	✅	🎯
VA-RC-17	`rcVaDepositThenRedeemIsReachable`	A lender deposits for shares and redeems them back in one round trip. `shares := deposit(assets); satisfy(shares > 0 AND redeem(shares).assetsBack > 0)`	✅	🎯
VA-RC-18	`rcVaDepositThenSettleIsReachable`	Freshly deposited shares immediately fund a targeted settle cut. `shares := deposit(assets); satisfy(shares > 0 AND settle(shares).sharesBurned > 0)`	✅	🎯
VA-RC-19	`rcVaWithdrawOnBehalfIsReachable`	A third party with allowance withdraws on the owner's behalf, consuming the allowance. `sender != owner; satisfy(shares > 0 AND allowance[owner][sender]' < allowance[owner][sender])`	✅	🎯
VA-RC-20	`rcVaRedeemOnBehalfIsReachable`	A third party with allowance redeems on the owner's behalf, consuming the allowance. `sender != owner; satisfy(assetsOut > 0 AND allowance[owner][sender]' < allowance[owner][sender])`	✅	🎯
VA-RC-21	`rcVaSettleOnBehalfIsReachable`	A third party with allowance runs a settle cut on the owner's shares. `sender != owner; satisfy(sharesBurned > 0 AND allowance[owner][sender]' < allowance[owner][sender])`	✅	🎯
VA-RC-22	`rcVaPreviewFallbackExitMixedIsReachable`	The fallback-exit preview reports a mixed liquid/illiquid outcome. `satisfy(loanAssets > 0 AND morphoSupplyShares > 0 AND loanAssets < convertToAssets(shares))`	✅	🎯
VA-RC-23	`rcVaMintWithExistingSharesIsReachable`	A mint pulls assets at existing-supply ceil pricing and credits the receiver. `totalSupply > 0; satisfy(assets > 0 AND shares > 0 AND shareBal[receiver]' == shareBal[receiver] + shares)`	✅	🎯
VA-RC-24	`rcVaFallbackExitPartialFillIsReachable`	A fallback exit can partial-fill, burning fewer shares than requested rather than reverting.	⏱️	🎯🔻

		`satisfy(sharesBurned > 0 AND sharesBurned < shares)`		
VA-RC-25	`rcVaFallbackExitLiquidLegCapCanRevert`	The redeem leg's own maxRedeem gate can revert an exit the owner holds in full. `msg.sender == owner AND previewRedeem(shares) <= maxWithdrawableAssets(vault) AND shares > maxRedeem(owner); satisfy(REVERTS)`	⏱	🎯🔻
VA-RC-26	`rcVaFallbackExitDefaultIterationCapSettles`	A zero maxIterations runs the default settlement sweep rather than skipping it. `maxIterations == 0; satisfy(sharesBurned > 0 AND (collateralSeized > 0 OR debtCanceled > 0))`	⏱	🎯
VA-RC-27	`rcVaSettleZeroProgressSucceeds`	A settle can complete with zero progress: nothing burned, nothing seized, nothing canceled. `satisfy(sharesBurned == 0 AND collateralSeized == 0 AND debtCanceled == 0)`	✅	🎯
VA-RC-28	`rcVaFallbackExitZeroProgressSucceeds`	A fallback exit can complete with zero progress: no liquid leg, no settle cut, nothing burned. `satisfy(sharesBurned == 0 AND loanAssets == 0)`	✅	🎯
VA-RC-29	`rcVaDepositBelowThresholdMintsNothing`	A positive deposit can mint zero shares; the assets are supplied into the position and no shares come back. `satisfy(assets > 0 AND deposit(assets, receiver) == 0)`	✅	🎯

Reverts

Identification	Name	Description	Properties	Mutations
VA-RV-01	`rvVaSettleRevertsOnEmptyBorrowers`	Settling against an empty borrower list reverts. `settle: borrowers.length == 0 => REVERTS`	✅	🎯
VA-RV-02	`rvVaSettleRevertsOnZeroShares`	Settling zero shares reverts. `settle: shares == 0 => REVERTS`	✅	🎯
VA-RV-03	`rvVaSettleRevertsOnZeroReceiver`	Settling toward the zero receiver reverts. `settle: receiver == 0 => REVERTS`	✅	🎯
VA-RV-04	`rvVaSettleRevertsAboveOwnerBalance`	Settling more shares than the owner holds reverts. `settle: shares > balanceOf[owner] => REVERTS`	✅	🎯
VA-RV-05	`rvVaSettleRevertsOnZeroSettlingShares`	An exit slice whose pro-rata core budget floors to zero reverts. `settle: position[id][vault].supplyShares * shares < totalSupply => REVERTS`	✅	🎯
VA-RV-06	`rvVaSettleRevertsOnMissingAllowance`	A third-party settle with an insufficient finite allowance reverts. `settle: sender != owner AND allowance[owner][sender] != max_uint256 AND allowance[owner][sender] < sharesBurned => REVERTS`	✅	🎯
VA-RV-07	`rvVaFallbackExitRevertsOnZeroShares`	Fallback-exiting zero shares reverts. `fallbackExit: shares == 0 => REVERTS`	✅	🎯
VA-RV-08	`rvVaFallbackExitRevertsOnZeroReceiver`	Fallback-exiting toward the zero receiver reverts. `fallbackExit: receiver == 0 => REVERTS`	✅	🎯
VA-RV-09	`rvVaFallbackExitRevertsAboveOwnerBalance`	Fallback-exiting more shares than the owner holds reverts. `fallbackExit: shares > balanceOf[owner] => REVERTS`	✅	🎯
VA-RV-10	`rvVaFallbackExitRevertsOnMissingAllowance`	A third-party fallback exit with an insufficient finite allowance reverts. `fallbackExit: sender != owner AND allowance[owner][sender] != max_uint256 AND allowance[owner][sender] < sharesBurned => REVERTS`	✅	🎯
VA-RV-11	`rvVaWithdrawRevertsOnMissingAllowance`	A third-party withdraw with an insufficient finite allowance reverts. `withdraw: sender != owner AND allowance[owner][sender] != max_uint256 AND allowance[owner][sender] <`	✅	🎯

		`previewWithdraw(assets) => REVERTS`		
VA-RV-12	`rvVaRedeemRevertsAboveOwnerBalance`	Redeeming above the owner's balance reverts; one that goes through debits exactly those shares. `redeem: (shares > balanceOf[owner] => REVERTS) AND (!REVERTS => balanceOf[owner]' == balanceOf[owner] - shares)`	✅	🎯
VA-RV-13	`rvVaMarketParamsNeverReverts`	The market params getter never reverts. `marketParams() never REVERTS`	✅	🎯
VA-RV-14	`rvVaDecimalsNeverReverts`	Decimals never reverts. `decimals() never REVERTS`	✅	🎯
VA-RV-15	`rvVaPreviewFallbackExitNeverReverts`	Quoting a fallback exit never reverts for share amounts in the supported range. `shares < 2^128 => previewFallbackExit(shares) !REVERTS`	✅	🎯
VA-RV-16	`rvVaPreviewDepositNeverReverts`	Quoting a deposit never reverts for asset amounts in the supported range. `assets < 2^128 AND totalSupply < 2^128 => previewDeposit(assets) !REVERTS`	✅	🎯
VA-RV-17	`rvVaPreviewMintNeverReverts`	Quoting a mint never reverts for share amounts in the supported range. `shares < 2^128 => previewMint(shares) !REVERTS`	✅	🎯
VA-RV-18	`rvVaPreviewWithdrawNeverReverts`	Quoting a withdrawal never reverts for asset amounts in the supported range. `assets < 2^128 AND totalSupply < 2^128 => previewWithdraw(assets) !REVERTS`	✅	🎯
VA-RV-19	`rvVaPreviewRedeemNeverReverts`	Quoting a redemption never reverts for share amounts in the supported range. `shares < 2^128 => previewRedeem(shares) !REVERTS`	✅	🎯
VA-RV-20	`rvVaFallbackExitDoesNotRevertOnShortfall`	The fallback exit is best-effort; when the settlement sweep falls short it returns a partial fill instead of reverting. `(msg.sender == owner OR allowance[owner][msg.sender] >= shares) AND maxWithdrawableAssets(vault) == 0 AND previewRedeem(shares) != 0 AND position[id][vault].supplyShares * shares >= totalSupply => fallbackExit !REVERTS`	⏱	🎯
VA-RV-21	`rvVaFallbackExitRevertsOnZeroSettlingShares`	A fallback exit whose pro-rata core budget floors to zero reverts, once the liquid leg is out. `fallbackExit: maxWithdrawableAssets == 0 AND previewRedeem(shares) != 0 AND position[id][vault].supplyShares * shares < totalSupply => REVERTS`	✅	🎯
VA-RV-22	`rvVaFallbackExitZeroQuoteOnlyBurnsWorthlessPositions`	The zero-quote leg of the fallback exit only ever burns a position the vault itself values at nothing. `fallbackExit: maxWithdrawableAssets == 0 AND previewRedeem(shares) == 0 AND !REVERTS => previewRedeem(balanceOf(owner)) == 0`	✅	🎯

Access Control

Identification	Name	Description	Properties	Mutations
VA-AC-01	`acVaAllowanceMovesOnlyByOwnerOrSpender`	A share allowance changes only when the caller is its owner or its spender. `allowance[o][s]' != allowance[o][s] => msg.sender == o OR msg.sender == s`	⏱	🎯

LidoLend_LinkedController

- The core and the controller running their real bytecode, linked; GuardianSignals, the oracle, the interest model and the tokens linked as CVL models
- Settlement queue narrowed to 7 live nodes plus its sentinel
- Core positions holding up to 7 modeled accounts, an eighth holder deleted rather than analyzed
- The core's borrower callbacks summarized away, so reentrancy is unanalyzed
- Transient inflow counter zeroed at the start of every rule
- Loops running up to 3 iterations
- The protocol fee pinned to zero in the end to end withdrawal round trip (CL-HL-08), so the live fee regime is uncovered there

Valid State

Identification	Name	Description	Properties	Mutations
CL-VS-01	`heapKeyFreshCoreBacked`	Every queued borrower carries exactly its live risk key. `wiring: marketController[id] == controller, heap <= 7 nodes; heapSlot[b] != 0 => collateral[b] != 0 AND heapKey[heapSlot[b]] == min(borrowShares[b] * WAD / collateral[b], max_uint96)`	⏱️	🎯
CL-VS-02	`heapMembershipMatchesCoreLifecycle`	The queue tracks exactly the borrowers holding debt and collateral. `wiring: marketController[id] == controller, heap <= 7 nodes; heapSlot[b] != 0 <=> (borrowShares[b] != 0 AND collateral[b] != 0)`	✅	🎯
CL-VS-03	`activationBucketsBackedByCoreCollateral`	Un-matured collateral never exceeds the core's collateral. `wiring: marketController[id] == controller, heap <= 7 nodes; activating[b] + dust[b] <= collateral[b]`	✅	🎯

High Level

Identification	Name	Description	Properties	Mutations
CL-HL-01	`hlLwcSupplyCollateralCreditMatchesCoreDelta`	A collateral deposit never books more as inactive than the core credits, and how much stays inactive follows the caller's trust. `supplyCollateral(assets, b) with instant := instantActivationCallers[msg.sender], stalled := isFrozen OR activatingStart[b] == 0 OR now < activatingStart[b] + activatingDelay[b], d := (activating[b] + dust[b])' - (activating[b] + dust[b]): collateral[b]' - collateral[b] == assets AND d <= assets AND (!instant AND stalled => d == assets) AND (instant AND stalled => d == (assets + dust[b] >= minActivationAmount ? -dust[b] : assets))`	✅	🎯
CL-HL-02	`hlLwcLiquidateSeizureDrainsBucketsExactly`	A liquidation seizure drains the buckets by the seized amount, saturating at the inactive total. `liquidate(borrower): inactive := activating[borrower] + dust[borrower],`	✅	🎯

		`seized := collateral[borrower] - collateral[borrower]' => inactive' == max(inactive - seized, 0)`		
CL-HL-03	`hlLwcSettleSeizureDrainsBucketsExactly`	A settlement seizure drains the buckets by the seized amount, saturating at the inactive total. `settle(borrower): inactive := activating[borrower] + dust[borrower], seized := collateral[borrower] - collateral[borrower]' => inactive' == max(inactive - seized, 0)`	⏱	🎯🔻
CL-HL-04	`hlLwcBorrowLeavesDebtCoveredByActiveCollateral`	A core borrow leaves the debt covered by ACTIVE collateral at LLTV, up to one wei of round-up. `lastUpdate == now AND totalBorrowAssets + VIRTUAL_ASSETS <= totalBorrowShares + VIRTUAL_SHARES: borrow(assets, 0, u): active := collateral[u]' - activating[u]' - dust[u]'; debt(u) + assets <= maxBorrow(active) AND debt(u)' <= maxBorrow(active) + 1`	⏱	🎯
CL-HL-05	`hlLwcMaxDepositQuoteMatchesCoreHeadroom`	The deposit-headroom quote is the core's real supplied-assets headroom under the cap. `lastUpdate == now OR totalBorrowAssets == 0 OR irm == 0: paused := now < resumeSinceTimestamp: (paused => maxDeposit() == 0) AND (!paused AND supplyCap == 0 => maxDeposit() == max_uint256) AND (!paused AND supplyCap != 0 => maxDeposit() == (supplyCap > totalSupplyAssets ? supplyCap - totalSupplyAssets : 0))`	✅	🎯
CL-HL-06	`hlLwcMaxDepositIsSuppliableEndToEnd`	The core accepts a deposit of exactly the quoted amount; quote and cap gate read one total. `lastUpdate == now AND !hookInProgress AND data.length == 0 AND supplyCap <= max_uint128 AND 0 < q == maxDeposit() < max_uint256 AND uint128 share/balance-counter headroom AND balance[sender] >= q AND allowance[sender][core] >= q: supply(q, 0, onBehalf) never REVERTS`	✅	🎯
CL-HL-07	`hlLwcMaxWithdrawableQuoteMatchesCoreFormula`	The withdrawable quote is the supplier's own assets capped by idle liquidity, zero while paused. `lastUpdate == now OR totalBorrowAssets == 0 OR irm == 0, paused := now < resumeSinceTimestamp: (paused => maxWithdrawableAssets(supplier) == 0) AND (!paused => maxWithdrawableAssets(supplier) == min(toAssetsDown(supplyShares[supplier], totalSupplyAssets, totalSupplyShares), max(totalSupplyAssets - totalBorrowAssets, 0)))`	✅	🎯
CL-HL-08	`hlLwcMaxWithdrawableIsWithdrawableEndToEnd`	The core pays out a withdrawal of exactly the amount the controller quotes. `!hookInProgress AND supplier == msg.sender AND fee == 0 AND q := maxWithdrawableAssets(supplier) != 0 AND supplier, core AND receiver in the bounded account set: withdraw(q, 0, supplier, receiver) never REVERTS`	⏱	🎯
CL-HL-09	`hlLwcWithdrawCollateralDebitsCoreAndHoldsActiveFloor`	A withdrawal debits the core row by exactly the assets asked; debt it leaves stands on ACTIVE at or above the activation floor. `withdrawCollateral(assets, u) with active := collateral[u]' - activating[u]' - dust[u]': collateral[u] - collateral[u]' == assets AND (borrowShares[u]' != 0 => active >= minActivationAmount)`	✅	🎯
CL-HL-10	`hlLwcWithdrawCollateralLeavesDebtCoveredByActiveCollateral`	A collateral withdrawal leaves the debt covered by ACTIVE collateral at LLTV, with no slack. `withdrawCollateral(assets, u, r): active := collateral[u]' - activating[u]' - dust[u]'; debt(u)' <= maxBorrow(active)`	⏱	🎯

CL-HL-11	`hlLwcSettleStrandsAtMostSubWeiDebt`	A settle that empties the collateral strands under one loan asset of debt, none at all supply-anchored. `lastUpdate == now AND borrower == nodes[1].account AND ltvStepBps == 0 AND settlementLtv[borrower] == 0: settle(supplyShares, borrowShares, borrower) with left := borrowShares[borrower]': collateral[borrower]' == 0 => (left * (totalBorrowAssets + VIRTUAL_ASSETS) < totalBorrowShares + VIRTUAL_SHARES AND (supplyShares != 0 => left == 0))`	⏱	🎯🔻
CL-HL-12	`hlLwcMaxWithdrawableQuoteAccrualInvariant`	Off the fee row, the withdrawable quote already prices in the pending interest; a real accrual moves nothing the projection has not counted. `supplier != feeRecipient: accrueInterest(p): maxWithdrawableAssets(supplier)' == maxWithdrawableAssets(supplier)`	✅	🎯

Reachability

Identification	Name	Description	Properties	Mutations
CL-RC-01	`rcLwcSupplyCollateralReachesActivation`	A collateral deposit reaches the controller's activation pipeline. `supplyCollateral(assets, u): satisfy((activating[u] + dust[u])' > activating[u] + dust[u])`	✅	🎯
CL-RC-02	`rcLwcBorrowInsertsBorrowerIntoHeap`	A borrow enters the borrower into the controller's settlement queue. `empty queue (nodes.length == 1) AND lastUpdate == now AND oraclePrice == ORACLE_PRICE_SCALE: borrow(assets, 0, u) THEN satisfy(slot[u]' != 0)`	✅	🎯🎯
CL-RC-03	`rcLwcFullRepayRemovesBorrowerFromHeap`	A full repay drops the borrower out of the queue. `slot[u] != 0: repay(u) THEN satisfy(slot[u]' == 0 AND borrowShares[u]' == 0)`	✅	🎯
CL-RC-04	`rcLwcFullSeizureLiquidationClearsHeapAndDebt`	A liquidation can wipe out a borrower's collateral and debt together and drop them from the queue. `slot[b] != 0: liquidate(b) THEN satisfy(collateral[b]' == 0 AND borrowShares[b]' == 0 AND slot[b]' == 0)`	✅	🎯
CL-RC-05	`rcLwcSettleAtRootCutsDebt`	A settle aimed at the queue root clears the bad-debt gate and the slice cap, and cuts real debt. `borrower == nodes[1].account: settle(borrower) THEN satisfy(borrowShares[borrower]' < borrowShares[borrower])`	⏱	🎯
CL-RC-06	`rcLwcWithdrawCollateralReKeysHeap`	Pulling collateral re-keys the borrower's queue entry. `slot[u] != 0: withdrawCollateral(u) THEN satisfy(slot[u]' != 0 AND nodes[slot[u]'].key' != nodes[slot[u]].key)`	✅	🎯
CL-RC-07	`rcLwcSupplyUnderCapSucceeds`	A deposit under a live supply cap clears the controller's cap check. `supplyCap != 0: satisfy(supply(assets, shares, onBehalf) completes)`	✅	🎯🎯
CL-RC-08	`rcLwcSubSaturatedMemberFeasible`	A live queue entry can carry a risk key below its ceiling while the borrower still holds collateral. `borrow(assets, shares, onBehalf): satisfy(heapSlot[onBehalf]' != 0 AND heapKey[heapSlot[onBehalf]']' < max_uint96 AND collateral[onBehalf]' != 0)`	✅	🎯
CL-RC-09	`rcLwcBorrowAtHeapCap`	With the settlement queue at its cap, a borrower already in the queue can still borrow. `heapLen == MAX_HEAP_NODES + 1 AND heapSlot[onBehalf] != 0: satisfy(borrow(assets, shares, onBehalf) completes)`	✅	🎯
		A borrow that really mints debt is reachable, so the collateral-coverage law is not vacuous. `lastUpdate == now AND totalBorrowAssets +`		

CL-RC-10	`rcLwcBorrowMintsDebtFeasible`	`VIRTUAL_ASSETS <= totalBorrowShares + VIRTUAL_SHARES: borrow(assets, 0, onBehalf): satisfy(assets > 0 AND borrowShares[onBehalf]' > borrowShares[onBehalf])`	✅	🎯
CL-RC-11	`rcLwcSupplyMaxDepositFeasible`	A deposit of exactly the quoted headroom is reachable, so the suppliability law is not vacuous. `lastUpdate == now AND data.length == 0 AND supplyCap <= max_uint128 AND q == maxDeposit() AND 0 < q < max_uint256 AND uint128 share/balance-counter headroom AND balance[sender] >= q AND allowance[sender][core] >= q: satisfy(supply(q, 0, onBehalf) completes)`	✅	🎯
CL-RC-12	`rcLwcSaturatedKeyRegimeReachable`	A core action can drive a borrower's queue key all the way to its ceiling. `heapSlot[onBehalf] == 0 OR heapKey[heapSlot[onBehalf]] < max_uint96: satisfy(heapSlot[onBehalf]' != 0 AND heapKey[heapSlot[onBehalf]']' == max_uint96)`	✅	🎯
CL-RC-13	`rcLwcMaxWithdrawableQuoteLiveUnderProjection`	The withdrawable quote can be live in the projected-totals regime CL-HL-05 and CL-HL-07 exclude. `lastUpdate < now AND totalBorrowAssets != 0 AND irm != 0: satisfy(maxWithdrawableAssets(supplier) > 0)`	✅	🎯
CL-RC-14	`rcLwcWithdrawPaysOutThroughNoOpHook`	A lender withdrawal pays out through the controller's no-op hook arm, the controller resumed. `lastUpdate == now: withdraw(assets, shares, u, r) THEN satisfy(retAssets > 0 AND supplyShares[u]' < supplyShares[u] AND now >= resumeSinceTimestamp)`	✅	🎯
CL-RC-15	`rcLwcAccrueInterestStampsClockThroughNoOpHook`	A standalone accrual stamps the core clock through the controller's no-op hook arm. `lastUpdate < now: accrueInterest(p) THEN satisfy(lastUpdate' == now AND now >= resumeSinceTimestamp)`	✅	🎯
CL-RC-16	`rcLwcMaturationUnblocksCoreBorrow`	Once the wait is over and the quorum scan has passed it, the matured lot carries a real core borrow. `lastUpdate == now AND price == ORACLE_PRICE_SCALE AND activatingStart[u] != 0 AND activatingStart[u] + activatingDelay[u] <= quorumFrontier AND strikeCount[u] == 0 AND suspectCount[u] == 0 AND collateral[u] - activating[u] - dust[u] < minActivationAmount: borrow(assets, 0, u): satisfy(activating[u] > 0 AND activating[u]' == 0 AND borrowShares[u]' > borrowShares[u])`	✅	🎯
CL-RC-17	`rcLwcSettleCutsDebtWithIdleLiquidityPresent`	A settle cuts debt idle liquidity alone could have paid out: the settleable gate weighs the settler's whole position against the idle total, not the requested cut. `lastUpdate == now AND price == ORACLE_PRICE_SCALE AND borrower == nodes[1].account AND borrower != 0: settle(borrower) THEN satisfy(idle > 0 AND totalBorrowAssets - totalBorrowAssets' <= idle AND borrowShares[borrower]' < borrowShares[borrower])`	✅	🎯
CL-RC-18	`rcLwcAccrualLiftsFeeRecipientQuote`	For a supplier holding the fee row, a real accrual lifts the withdrawable quote: the credit the accrual books is one the projection does not count. `lastUpdate < now AND supplier == feeRecipient: accrueInterest(p) THEN satisfy(maxWithdrawableAssets(supplier)' > maxWithdrawableAssets(supplier))`	⏱	🎯

Reverts

Identification	Name	Description	Properties	Mutations
CL-RV-01	`rvLwcPausedBlocksAllHookedCoreMutators`	A paused controller blocks every hooked core mutator. `now < resumeSinceTimestamp => REVERTS, for f in {supply, withdraw, borrow, repay, supplyCollateral, withdrawCollateral, settle, liquidate, accrueInterest}`	✅	🎯
CL-RV-02	`rvLwcFrozenBlocksCoreBorrow`	A frozen market blocks a core borrow. `isFrozen => borrow(p, assets, shares, onBehalf, receiver) REVERTS`	✅	🎯
CL-RV-03	`rvLwcFrozenBlocksDebtBearingCoreWithdrawCollateral`	A freeze blocks a core collateral withdrawal by a borrower carrying debt. `isFrozen AND borrowShares[onBehalf] != 0 => withdrawCollateral(p, assets, onBehalf, receiver) REVERTS`	✅	🎯
CL-RV-04	`rvLwcCoreSupplyRevertsOverCap`	A core deposit breaching a live supply cap reverts. `lastUpdate == now AND supplyCap != 0 AND totalSupplyAssets + assets > supplyCap => supply(p, assets, shares, onBehalf, data) REVERTS`	✅	🎯
CL-RV-05	`rvLwcCoreSettleRevertsWhileRootBadDebt`	No core settle proceeds while the controller's queue root carries bad debt. `lastUpdate == now AND hasBadDebt() => settle(p, supplyShares, borrowShares, onBehalf, borrower, receiver) REVERTS`	✅	🎯
CL-RV-06	`rvLwcBannedCannotBorrowEndToEnd`	A ban on the caller, receiver or borrower blocks the core borrow. `isBanned(marketId, msg.sender) OR isBanned(marketId, receiver) OR isBanned(marketId, onBehalf) => borrow(p, assets, shares, onBehalf, receiver) REVERTS`	✅	🎯
CL-RV-07	`rvLwcCoreSettleRevertsForIneligibleTarget`	No core settle proceeds against a borrower the queue does not name and who has not opted into a settlement LTV. `lastUpdate == now AND settlementLtv[borrower] == 0 AND (nodes.length < 2 OR nodes[1].account != borrower) AND (borrower != 0 OR nodes.length >= 2) => settle(p, supplyShares, borrowShares, onBehalf, borrower, receiver) REVERTS`	✅	🎯
CL-RV-08	`rvLwcShareAnchoredBorrowRoundingToZeroReverts`	A share-anchored borrow whose assets round to zero is refused. `lastUpdate == now AND shares != 0 AND shares * (totalBorrowAssets + VIRTUAL_ASSETS) < totalBorrowShares + VIRTUAL_SHARES => borrow(p, 0, shares, onBehalf, receiver) REVERTS`	✅	🎯

LidoLendERC4626_LinkedCore

- The vault and the core running their real bytecode; the controller, oracle, interest model and tokens linked as CVL models
- One symbolic market, the per-market mirrors carrying no market key
- Core positions and vault share balances holding up to 7 modeled accounts each, plus the zero slot
- Fresh accrual pinned in the rate laws, the deposit delta law and the witnesses, so pending interest is uncovered there; the frame, token, liquidity and totalAssets laws run unpinned

- The core's borrower callbacks and EIP-712 grant path summarized away, so reentrancy is unanalyzed
- Loops running up to 3 iterations

High Level

Identification	Name	Description	Properties	Mutations
VC-HL-01	`hlVcVaultNeverAuthorizesInCore`	No vault entry point outside settle and fallbackExit moves the core's authorization row for the vault. `any vault entry point f (f not view, pure OR fallback, f not in {settle, fallbackExit}): isAuthorized[vault][a]' == isAuthorized[vault][a]`	✅	🎯
VC-HL-02	`hlVcDepositSuppliesExactAssetsToCore`	A vault deposit adds exactly its assets to the core market's supply side. `lastUpdate == now: deposit(assets, receiver): totalSupplyAssets' == totalSupplyAssets + assets`	✅	🎯🎯
VC-HL-03	`hlVcVaultMovesOnlyItsOwnCorePosition`	No vault entry point outside settle and fallbackExit moves a core supply position other than the vault's own and the fee row's. `a != vault AND a != feeRecipient: any vault entry point f (f not view, pure OR fallback, f not in {settle, fallbackExit}): supplyShares[a]' == supplyShares[a]`	✅	🎯🎯
VC-HL-04	`hlVcDepositCreditCappedByPreDepositRate`	A deposit is credited no more core shares than the pre-deposit rate prices its assets at; the floor half of the rate law is VC-HL-05. `lastUpdate == now, A := totalSupplyAssets, S := totalSupplyShares, c := supplyShares[vault]' - supplyShares[vault]: deposit(assets, receiver): c * (A + VIRTUAL_ASSETS) <= assets * (S + VIRTUAL_SHARES)`	✅	🎯
VC-HL-05	`hlVcDepositCreditReachesPreDepositRateFloor`	A deposit is credited no fewer core shares than the floor of the pre-deposit price; the cap half of the rate law is VC-HL-04. `lastUpdate == now, A := totalSupplyAssets, S := totalSupplyShares, c := supplyShares[vault]' - supplyShares[vault]: deposit(assets, receiver): assets * (S + VIRTUAL_SHARES) < (c + 1) * (A + VIRTUAL_ASSETS)`	✅	🎯
VC-HL-06	`hlVcWithdrawBurnCoversPreWithdrawRate`	A withdrawal burns no fewer core shares than the pre-withdraw rate prices its assets at, so the exit's cost never lands on the holders who stay; the ceiling half of the rate law is VC-HL-07. `lastUpdate == now, A := totalSupplyAssets, S := totalSupplyShares, b := supplyShares[vault] - supplyShares[vault]': withdraw(assets, receiver, owner):`	✅	🎯

		`assets * (S + VIRTUAL_SHARES) <= b * (A + VIRTUAL_ASSETS)`		
VC-HL-07	`hlVcWithdrawBurnCappedByPreWithdrawRateCeiling`	A withdrawal burns no more core shares than the ceiling of the pre-withdraw price, so the leaver is not overcharged; the covering half of the rate law is VC-HL-06. `lastUpdate == now, A := totalSupplyAssets, S := totalSupplyShares, b := supplyShares[vault] - supplyShares[vault]': withdraw(assets, receiver, owner): (b - 1) * (A + VIRTUAL_ASSETS) < assets * (S + VIRTUAL_SHARES)`	✅	🎯
VC-HL-08	`hlVcDepositPassesAssetsThroughToCore`	A deposit passes the loan token straight through the vault into the core; the vault keeps none of it. The caller is neither scene contract already: setupEnv pins the sender off the vault and setupLidoLendERC4626 pins it off the linked core. `t := loanToken: deposit(assets, receiver): bal[t][core]' == bal[t][core] + assets AND bal[t][msg.sender]' == bal[t][msg.sender] - assets AND bal[t][vault]' == bal[t][vault]`	✅	🎯
VC-HL-09	`hlVcTotalAssetsAccrualInvariant`	Off the fee row, the vault's totalAssets already prices in the pending interest; a real accrual reveals nothing the projection has not counted. `feeRecipient != vault: accrueInterest(p): totalAssets()' == totalAssets()`	✅	🎯
VC-HL-10	`hlVcTotalAssetsStableAcrossOwnFeeCredit`	With the vault named the market's fee recipient, its totalAssets does not jump when the accrual books the fee credit onto the vault's own core row. `feeRecipient == vault: accrueInterest(p): totalAssets()' == totalAssets()`	❌→✅	Issue #71 (MEDIUM)

Reachability

Identification	Name	Description	Properties	Mutations
VC-RC-01	`rcVcDepositReachesCorePosition`	A vault deposit lands in the core's own supply position. `lastUpdate == now: deposit(assets, receiver): satisfy(supplyShares[vault]' > supplyShares[vault])`	✅	🎯
VC-RC-02	`rcVcCoreSideDonationMovesSharePrice`	A third party supplying to the core on the vault's behalf lifts the share price; the half of ERC4626-33 a direct token donation cannot reach, since supply gates no onBehalf. The position conjunct names the beneficiary; without it rounding alone meets the witness. `lastUpdate == now AND data.length == 0: LidoLend.supply(p, amount, 0, vault, data): satisfy(supplyShares[vault]' > supplyShares[vault] AND previewRedeem(shares)' > previewRedeem(shares))`	✅	🎯🎯
VC-RC-03	`rcVcWithdrawDebitsCorePosition`	A vault withdrawal debits the core's supply position. It also keeps VC-RV-01 honest: a withdrawal that clears the core's real liquidity gate exists, so that rule's implication is not carried by an empty domain. `lastUpdate == now: withdraw(assets, receiver, owner): satisfy(supplyShares[vault]' < supplyShares[vault])`	✅	🎯
VC-RC-04	`rcVcVaultExitsWithoutCoreGrant`	The vault reaches its own core position while holding no authorization row in the core; the read side of the authorization seam, the half the solo suite's CVL core stub cannot state. `lastUpdate == now AND !isAuthorized[vault][a] for all a: withdraw(assets, receiver, owner): satisfy(supplyShares[vault]' < supplyShares[vault])`	✅	🎯
VC-RC-05	`rcVcAccrualLandsInterestAndFeeCredit`	The pending-interest regime is live in scene: a real accrual lands interest on the supply side and books the fee credit onto the fee row. `lastUpdate < now: accrueInterest(p): satisfy(totalSupplyAssets' > totalSupplyAssets AND supplyShares[feeRecipient]' > supplyShares[feeRecipient])`	✅	🎯

Reverts

Identification	Name	Description	Properties	Mutations
VC-RV-01	`rvVcWithdrawAboveCoreLiquidityReverts`	A withdrawal past free liquidity reverts; the solo vault suite only assumes this core gate. `withdraw(assets, receiver, owner): assets > totalSupplyAssets - totalBorrowAssets => REVERTS`	✅	🎯🎯🎯

Erc4626ExitBook_LinkedVaultCore

- The book running its real bytecode; the vault and the core compiled in beside it for their state, and the controller and the tokens linked as CVL models
- Every vault call the book makes resolving into a CVL ERC4626 model: previews and redeemed amounts are frozen draws, and a redemption moves the vault's asset over the shared token ledger
- One vault anchored to its real storage; any other allow-listed vault is a free model row
- Token ledgers and the vault's share ledger holding up to 7 accounts each
- Loops running up to 3 iterations

Valid State

Identification	Name	Description	Properties	Mutations
EE-VS-01	`eeServiceFeeCapped`	The protocol's cut of a filler's reward is hard-capped; the owner cannot confiscate the bulk. `serviceFee <= MAX_SERVICE_FEE (0.25e18)`	✅	🎯
EE-VS-02	`eeServiceFeeHasRecipient`	A nonzero cut always has a recipient; a fill never pays fees into the void. `serviceFee != 0 => feeRecipient != 0`	✅	🎯
EE-VS-03	`eeZeroIdNeverCreated`	Ids start at one, so id zero always reads as 'no such request' and can never alias a real entry. `requests[0].owner == 0`	✅	🎯
EE-VS-04	`eeForwardIdsNeverCreated`	No request lives beyond the id counter, so a fresh placement can never land on an occupied row. `id > nextId => requests[id].owner == 0`	✅	🎯
EE-VS-05	`eeIssuedIdsDense`	Every id the book has ever handed out still resolves to a request; the registry has no holes. `1 <= id AND id <= nextId => requests[id].owner != 0`	✅	🎯
EE-VS-06	`eeUncreatedAssetZero`	No payout token is on file before a request exists; an unknown id reads back as a blank record. `requests[id].owner == 0 => requests[id].asset == 0`	✅	🎯
EE-VS-07	`eeUncreatedSizeRemainingZero`	No shares wait to be filled before a request exists. `requests[id].owner == 0 => requests[id].sizeRemaining == 0`	✅	🎯
EE-VS-08	`eeUncreatedSizeTotalZero`	No posted exit size is on record before a request exists. `requests[id].owner == 0 => requests[id].sizeTotal == 0`	✅	🎯
EE-VS-09	`eeUncreatedPriceZero`	No payout price is quoted before a request exists. `requests[id].owner == 0 => requests[id].pricePerShare == 0`	✅	🎯
EE-VS-10	`eeUncreatedStartRateZero`	No opening fee rate is set before a request exists. `requests[id].owner == 0 => requests[id].startRate == 0`	✅	🎯
EE-VS-11	`eeUncreatedEndRateZero`	No fee ceiling is set before a request exists. `requests[id].owner == 0 =>`	✅	🎯

		`requests[id].endRate == 0`		
EE-VS-12	`eeUncreatedRampStartZero`	No fee climb has a scheduled start before a request exists. `requests[id].owner == 0 => requests[id].rampStart == 0`	✅	🎯
EE-VS-13	`eeUncreatedRampEndZero`	No fee climb has a scheduled end before a request exists. `requests[id].owner == 0 => requests[id].rampEnd == 0`	✅	🎯
EE-VS-14	`eeUncreatedCreatedAtZero`	No birth date is on record before a request exists. `requests[id].owner == 0 => requests[id].createdAt == 0`	✅	🎯
EE-VS-15	`eeUncreatedDeadlineZero`	No expiry is on record before a request exists. `requests[id].owner == 0 => requests[id].deadline == 0`	✅	🎯
EE-VS-16	`eeUncreatedNotFullFill`	No all-or-nothing policy is set before a request exists. `requests[id].owner == 0 => !requests[id].fullFillOnly`	✅	🎯
EE-VS-17	`eeUncreatedStatusOpen`	An unissued request shows the default Open status; nothing is pre-filled or pre-cancelled. `requests[id].owner == 0 => requests[id].status == ExitStatus.Open`	✅	🎯
EE-VS-18	`eeRemainingImpliesOpen`	Shares wait to exit only behind an open request; a closed one never has anything left to take. `requests[id].sizeRemaining > 0 => requests[id].status == ExitStatus.Open`	✅	🎯
EE-VS-19	`eeOpenImpliesRemaining`	An open request always has shares left to take; draining it closes it on the spot. `requests[id].owner != 0 AND requests[id].status == Open => requests[id].sizeRemaining > 0`	✅	🎯
EE-VS-20	`eeRemainingLeTotal`	Nobody is ever owed more than the lender originally posted. `requests[id].sizeRemaining <= requests[id].sizeTotal`	✅	🎯
EE-VS-21	`eeCreatedSizeTotalPositive`	No request is ever born empty; the book carries no entries with nothing behind them. `requests[id].owner != 0 => requests[id].sizeTotal != 0`	✅	🎯
EE-VS-22	`eeCreatedPricePositive`	Every request locks in a nonzero payout price, so a filler never takes the redemption for free. `requests[id].owner != 0 => requests[id].pricePerShare != 0`	✅	🎯
EE-VS-23	`eeStartRateLeEndRate`	The priority fee only climbs; a request never opens paying more than its advertised ceiling. `requests[id].startRate <= requests[id].endRate`	✅	🎯
EE-VS-24	`eeEndRateCapped`	No request promises a fee above the contract's cap; the lender always keeps the bulk.	✅	🎯

		`requests[id].endRate <= MAX_PRIORITY_FEE (0.1e18)`		
EE-VS-25	`eeRampStartLeRampEnd`	The fee ramp is a well-formed schedule; its climb cannot end before it begins. `requests[id].rampStart <= requests[id].rampEnd`	✅	🎯
EE-VS-26	`eeRampEndsByDeadline`	The fee finishes climbing while the request is still fillable; no ramp runs past expiry. `requests[id].owner != 0 => requests[id].createdAt + requests[id].rampEnd <= requests[id].deadline`	✅	🎯
EE-VS-27	`eeDeadlineAfterCreation`	Every request opens with a live fill window; none is born already expired. `requests[id].owner != 0 => requests[id].deadline > requests[id].createdAt`	✅	🎯
EE-VS-28	`eeDeadlineWithinLifetime`	No request can sit on the book forever; its expiry stays within the contract's lifetime cap. `requests[id].owner != 0 => requests[id].deadline <= requests[id].createdAt + MAX_REQUEST_LIFETIME`	✅	🎯
EE-VS-29	`eeCreatedAtNotInFuture`	A request's birth date never leads the chain, so the fee ramp's clock never runs backwards. `requests[id].createdAt <= block.timestamp`	✅	🎯
EE-VS-30	`eeZeroOwnerNoIds`	The empty address, the book's 'no such request' marker, never accumulates a listing of its own. `ownerRequestIds[0].length == 0`	✅	🎯
EE-VS-31	`eeOwnerIndexIssued`	A lender's history names only requests the book actually issued; it never points into thin air. `i < ownerRequestIds[o].length => 1 <= ownerRequestIds[o][i] AND ownerRequestIds[o][i] <= nextId`	✅	🎯
EE-VS-32	`eeOwnerIndexOwnership`	A lender's history lists only their own requests; nobody else's exits show up under their name. `i < ownerRequestIds[o].length => requests[ownerRequestIds[o][i]].owner == o`	✅	🎯
EE-VS-33	`eeOwnerIndexStrictlyIncreasing`	A lender's history reads in placement order and never lists the same request twice. `i < j AND j < ownerRequestIds[o].length => ownerRequestIds[o][i] < ownerRequestIds[o][j]`	✅	🎯
EE-VS-34	`eeZeroVaultNeverAllowed`	The empty vault can never make the allow-list. `!isVaultAllowed[0]`	✅	🎯
EE-VS-35	`eeCreatedIffVaultBound`	Every request names the vault that redeems it, and no vault binding floats free. `requests[id].owner != 0 <=> vaultOf[id] != 0`	✅	🎯

EE-VS-36	`eeVaultAssetBinding`	A request pays out in the underlying of the vault it is bound to, never quoting one asset and paying another. `requests[id].owner != 0 => requests[id].asset == vaultOf[id].asset()`	✅	🎯
EE-VS-37	`eeFullFillOnlyOpenUntouched`	An all-or-nothing request is never part-filled; while open it carries all the lender posted. `requests[id].fullFillOnly AND requests[id].status == ExitStatus.Open => requests[id].sizeRemaining == requests[id].sizeTotal`	✅	🎯
EE-VS-38	`eeOwnerIndexLenLeNextId`	No lender's history can list more requests than the book has ever issued. `ownerRequestIds[o].length <= nextId`	✅	🎯
EE-VS-39	`eeLinkedVaultAssetBinding`	The same binding at the vault compiled into the scene, where asset() is the mirrored one. `requests[id].owner != 0 AND vaultOf[id] == vault (the linked LidoLendERC4626) => requests[id].asset == vault.asset()`	✅	🎯

Variable Transitions

Identification	Name	Description	Properties	Mutations
EE-VT-01	`vtEeNextIdStepsByOne`	The id counter never decreases and steps by at most one. `nextId' == nextId OR nextId' == nextId + 1`	✅	🎯🎯
EE-VT-02	`vtEeReqOwnerFrozenOnceSet`	A request's owner is frozen once set. `requests[id].owner != 0 => requests[id].owner' == requests[id].owner`	✅	🎯
EE-VT-03	`vtEeClosedStatusTerminal`	A closed request's status never changes. `requests[id].owner != 0 AND requests[id].status != Open => requests[id].status' == requests[id].status`	✅	🎯
EE-VT-04	`vtEeSizeRemainingNeverGrows`	A created request's remaining size never grows. `requests[id].owner != 0 => requests[id].sizeRemaining' <= requests[id].sizeRemaining`	✅	🎯🎯
EE-VT-05	`vtEeTermsFrozenOnceCreated`	A created request's economic terms are frozen. `requests[id].owner != 0 => (requests[id].asset' == requests[id].asset AND requests[id].sizeTotal' == requests[id].sizeTotal AND requests[id].pricePerShare' == requests[id].pricePerShare AND requests[id].startRate' == requests[id].startRate AND requests[id].endRate' == requests[id].endRate)`	✅	🎯
		A created request's schedule and policy are frozen. `requests[id].owner != 0 =>`		

EE-VT-06	`vtEeScheduleFrozenOnceCreated`	`(requests[id].rampStart' == requests[id].rampStart AND requests[id].rampEnd' == requests[id].rampEnd AND requests[id].createdAt' == requests[id].createdAt AND requests[id].deadline' == requests[id].deadline AND requests[id].fullFillOnly' == requests[id].fullFillOnly)`	✅	🎯
EE-VT-07	`vtEeVaultFrozenOnceCreated`	A created request's bound vault is frozen. `requests[id].owner != 0 => vaultOf[id]' == vaultOf[id]`	✅	🎯🎯
EE-VT-08	`vtEeOwnerIndexAppendOnly`	An owner's index only appends. `(ownerRequestIds[o].length' == ownerRequestIds[o].length OR ownerRequestIds[o].length' == ownerRequestIds[o].length + 1) AND (i < ownerRequestIds[o].length => ownerRequestIds[o][i]' == ownerRequestIds[o][i])`	✅	🎯
EE-VT-09	`vtEeOwnerIndexAppendsFreshId`	Only a freshly placed request grows a listing, and it always lands in its placer's own listing. `(ownerRequestIds[o].length' == ownerRequestIds[o].length + 1 => (nextId' == nextId + 1 AND ownerRequestIds[o][ownerRequestIds[o].length]' == nextId')) AND ((nextId' == nextId + 1 AND requests[nextId'].owner' == o) => ownerRequestIds[o].length' == ownerRequestIds[o].length + 1)`	✅	🎯

State Transitions

Identification	Name	Description	Properties	Mutations
EE-ST-01	`stEeSizeMovesOnlyOpen`	Size moves only on an open request. `requests[id].sizeRemaining' != requests[id].sizeRemaining => requests[id].status == Open`	✅	🎯
EE-ST-02	`stEeFilledClosesAtZero`	A fill closes only at zero remaining. `requests[id].sizeRemaining' < requests[id].sizeRemaining AND requests[id].status' == Filled => requests[id].sizeRemaining' == 0`	✅	🎯
EE-ST-03	`stEeCancelZeroesSize`	A cancel zeroes the remaining size. `requests[id].status' != requests[id].status AND requests[id].status' == Cancelled => requests[id].sizeRemaining' == 0`	✅	🎯
EE-ST-04	`stEeMintOwnsNewId`	A mint hands the new id to the caller. `nextId' != nextId => requests[nextId'].owner' == msg.sender`	✅	🎯
EE-ST-05	`stEeMintIndexesOwner`	A mint appends one entry to the caller's index. `nextId' != nextId => ownerRequestIds[msg.sender].length' == ownerRequestIds[msg.sender].length + 1`	✅	🎯

High Level

Identification	Name	Description	Properties	Mutations
EE-HL-01	`hlEeFillLeavesOtherRequests`	A fill moves no other request's remaining size. `id2 != id: fillExitRequest(id, depositAssets, minAssetsOut): requests[id2].sizeRemaining' == requests[id2].sizeRemaining`	✅	🎯
EE-HL-02	`hlEePlaceLeavesExistingRequests`	A place moves no already-issued request's remaining size. `oldId <= nextId: placeExitRequest(vault, shares, ramp, deadline, fullFillOnly): requests[oldId].sizeRemaining' == requests[oldId].sizeRemaining`	✅	🎯
EE-HL-03	`hlEeCancelLeavesOtherRequests`	A cancel moves no other request's status. `id2 != id: cancelExitRequest(id): requests[id2].status' == requests[id2].status`	✅	🎯
EE-HL-04	`hlEeRampRateWithinBand`	The fee rate a fill would charge never leaves the request's own ramp band. `rate := _priorityFeeRate(requests[id]) at block.timestamp: requests[id].startRate <= rate AND rate <= requests[id].endRate AND rate <= MAX_PRIORITY_FEE (0.1e18)`	✅	🎯
EE-HL-05	`hlEeRampRateMonotoneInTime`	The fee rate never falls as a request ages; the auction only moves one way. `requests[id].createdAt <= t1 AND t1 <= t2 => _priorityFeeRate(requests[id]) at t1 <= _priorityFeeRate(requests[id]) at t2`	✅	🎯
EE-HL-06	`hlEeFillKeepsNoRemainder`	A fill hands out everything it redeemed; the book keeps no remainder. `forall i. requests[i].owner != Erc4626ExitBook AND vaultOf[i] != Erc4626ExitBook, feeRecipient != Erc4626ExitBook, msg.sender != Erc4626ExitBook: fillExitRequest(id, depositAssets, minAssetsOut): bal[requests[id].asset][Erc4626ExitBook]' == bal[requests[id].asset][Erc4626ExitBook]`	✅	🎯
EE-HL-07	`hlEeFillDebitsSizeByPreviewedShares`	A fill that carries no filler deposit draws the request down by exactly the shares its own preview quoted. `p := previewFill(id); fillExitRequest(id, 0, minAssetsOut): p.filledShares > 0 AND requests[id].sizeRemaining' == requests[id].sizeRemaining - p.filledShares AND (requests[id].status' == Filled <=> requests[id].sizeRemaining' == 0)`	✅	🎯
EE-HL-08	`hlEeFillTakesAssetsOutOfItsVault`	A fill pays the lender and the filler out of the vault it redeemed from. `a := requests[id].asset, o := requests[id].owner, f := feeRecipient, v := vaultOf[id], v not in {o, msg.sender, f}: fillExitRequest(id, 0, minAssetsOut): bal[a][v]' < bal[a][v]`	✅	🎯
EE-HL-09	`hlEePlacePricesAtItsOwnVault`	A placement binds the vault it named and prices a share at that vault's own quote. `id = placeExitRequest(vault, shares, ramp, deadline, fullFillOnly): vaultOf[id] == vault AND requests[id].pricePerShare == vault.convertToAssets(WAD)`	✅	🎯

EE-HL-10	`hlEePreviewQuotesAtItsOwnVault`	A preview quotes its assets out of the request's own vault, and quotes nothing when nothing is fillable. `v := vaultOf[id], requests[id].owner != 0: p = previewFill(id): (p.filledShares > 0 => p.assetsOut == v.previewRedeem(p.filledShares)) AND (p.filledShares == 0 => p.assetsOut == 0)`	✅	🎯

Unit Tests

Identification	Name	Description	Properties	Mutations
EE-UT-01	`utEeRampRateHoldsAtStartRate`	Through the opening hold the fee rate stays at the request's startRate. `block.timestamp - requests[id].createdAt < requests[id].rampStart => _priorityFeeRate(requests[id]) == requests[id].startRate`	✅	🎯
EE-UT-02	`utEeRampRateInterpolatesLinearly`	Across the climb the fee rate interpolates linearly from startRate to endRate. `elapsed := block.timestamp - requests[id].createdAt; requests[id].rampStart <= elapsed < requests[id].rampEnd => _priorityFeeRate(requests[id]) == requests[id].startRate + floor((requests[id].endRate - requests[id].startRate) * (elapsed - requests[id].rampStart) / (requests[id].rampEnd - requests[id].rampStart))`	✅	🎯
EE-UT-03	`utEeRampRatePlateausAtEndRate`	From rampEnd to the deadline the fee rate plateaus at endRate. `block.timestamp - requests[id].createdAt >= requests[id].rampEnd => _priorityFeeRate(requests[id]) == requests[id].endRate`	✅	🎯
EE-UT-04	`utEeFillPaysLenderOwedMinusPriorityFee`	A deposit-free fill pays the lender the price owed, capped at the assets redeemed, minus the ramp's fee. `o != msg.sender AND o != feeRecipient AND Erc4626ExitBook not in {o, v} AND v not in {o, msg.sender, feeRecipient}: fillExitRequest(id, 0, minAssetsOut): d(o) == owed - floor(owed * _priorityFeeRate(r) / WAD) [r := requests[id], a := r.asset, o := r.owner, v := vaultOf[id], d(x) := bal[a][x]' - bal[a][x], out := -d(v), owed := min(floor((r.sizeRemaining - r.sizeRemaining') * r.pricePerShare / WAD), out)]`	✅	🎯
EE-UT-05	`utEeFillSplitsFillerRewardAndServiceCut`	A deposit-free fill pays the filler the yield plus the ramp fee, net of the service cut the recipient gets. `o, msg.sender, f, v all distinct AND Erc4626ExitBook not in {o, f, v}: fillExitRequest(id, 0, minAssetsOut): d(msg.sender) == gross - cut AND d(f) == cut [r := requests[id], a := r.asset, o := r.owner, f := feeRecipient, v := vaultOf[id], d(x) := bal[a][x]' - bal[a][x], out := -d(v), owed := min(floor((r.sizeRemaining - r.sizeRemaining') * r.pricePerShare / WAD), out), gross := out - owed + floor(owed * _priorityFeeRate(r) / WAD), cut := floor(gross * serviceFee / WAD)]`	✅	🎯
EE-UT-06	`utEeOwnerRequestIdsReadsTheRequestedWindow`	An owner's page read returns the window it was asked for, starting where it was told. `1 <= count <= 3 AND offset < ownerRequestIds[o].length: page := getOwnerRequestIds(o, offset, count): page.length == min(count, ownerRequestIds[o].length - offset) AND page[0] == ownerRequestIds[o][offset]`	✅	🎯

Reachability

Identification	Name	Description	Properties	Mutations
		A fresh caller-owned exit request can be placed. `id = placeExitRequest(vault, shares, ramp,`		

EE-RC-01	`rcEePlaceExitRequestIsReachable`	`deadline, fullFillOnly): satisfy(requests[id].owner' == msg.sender AND requests[id].sizeRemaining' > 0)`	✅	🎯
EE-RC-02	`rcEePlaceFullFillOnlyIsReachable`	An all-or-nothing exit request can be placed. `id = placeExitRequest(vault, shares, ramp, deadline, true): satisfy(requests[id].fullFillOnly' AND requests[id].sizeRemaining' > 0)`	✅	🎯
EE-RC-03	`rcEePlaceOnLinkedVaultIsReachable`	A request can be placed against the in-scene vault, whose modelled asset() is pinned to the vault's own. `id = placeExitRequest(LidoLendERC4626, shares, ramp, deadline, fullFillOnly): satisfy(vaultOf[id]' == LidoLendERC4626 AND requests[id].sizeRemaining' > 0)`	✅	🎯
EE-RC-04	`rcEePlaceOnUnlinkedVaultIsReachable`	A request can be placed against a vault address other than the in-scene one. `vault != LidoLendERC4626: id = placeExitRequest(vault, shares, ramp, deadline, fullFillOnly): satisfy(vaultOf[id]' == vault AND requests[id].sizeRemaining' > 0)`	✅	🎯
EE-RC-05	`rcEePlaceOneShareIsReachable`	A one-share exit request can be placed. `id = placeExitRequest(vault, shares, ramp, deadline, fullFillOnly): satisfy(requests[id].sizeTotal' == 1)`	✅	🎯
EE-RC-06	`rcEePlaceMaxSharesIsReachable`	A request at the uint128 packed-field boundary can be placed. `id = placeExitRequest(vault, shares, ramp, deadline, fullFillOnly): satisfy(requests[id].sizeTotal' == max_uint128 (2^128 - 1))`	✅	🎯
EE-RC-07	`rcEePlaceZeroRampIsReachable`	A request with no priority-fee ramp can be placed. `id = placeExitRequest(vault, shares, ramp, deadline, fullFillOnly): satisfy(requests[id].startRate' == 0 AND requests[id].endRate' == 0 AND requests[id].sizeRemaining' > 0)`	✅	🎯
EE-RC-08	`rcEePlaceDeadlineAtLifetimeCapIsReachable`	A request whose deadline sits exactly at the lifetime cap can be placed. `id = placeExitRequest(vault, shares, ramp, deadline, fullFillOnly): satisfy(requests[id].deadline' == requests[id].createdAt' + MAX_REQUEST_LIFETIME AND requests[id].sizeRemaining' > 0)`	✅	🎯
EE-RC-09	`rcEeFillExitRequestIsReachable`	A fill can strictly shrink an open request, leaving it open. `fillExitRequest(id, depositAssets, minAssetsOut): satisfy(requests[id].sizeRemaining' < requests[id].sizeRemaining AND requests[id].sizeRemaining' > 0)`	✅	🎯
EE-RC-10	`rcEeFillFullFillOnlyIsReachable`	An all-or-nothing request can be closed by a single full fill. `fillExitRequest(id, depositAssets, minAssetsOut): satisfy(requests[id].fullFillOnly' AND requests[id].sizeRemaining > 0 AND requests[id].sizeRemaining' == 0 AND requests[id].status' == Filled)`	✅	🎯🎯
EE-RC-11	`rcEeFillOneShareStepIsReachable`	A fill can consume exactly one share. `fillExitRequest(id, depositAssets, minAssetsOut): satisfy(requests[id].sizeRemaining' == requests[id].sizeRemaining - 1)`	✅	🎯
EE-RC-12	`rcEeFillPaysRequestOwnerIsReachable`	A fill pays the exiting lender in the request asset. `owner := requests[id].owner, asset := requests[id].asset: fillExitRequest(id, depositAssets, minAssetsOut): satisfy(requests[id].sizeRemaining' < requests[id].sizeRemaining AND owner != msg.sender AND owner != feeRecipient AND owner != vaultOf[id] AND balance[asset][owner]' > balance[asset]`	✅	🎯🎯

		`[owner])`		
EE-RC-13	`rcEeFillPaysServiceFeeIsReachable`	A fill under a positive service fee pays the fee recipient. `recipient := feeRecipient, owner := requests[id].owner, asset := requests[id].asset: fillExitRequest(id, depositAssets, minAssetsOut): satisfy(serviceFee > 0 AND requests[id].sizeRemaining' < requests[id].sizeRemaining AND recipient != owner AND recipient != msg.sender AND recipient != vaultOf[id] AND balance[asset][recipient]' > balance[asset][recipient])`	✅	🎯
EE-RC-14	`rcEeCancelExitRequestIsReachable`	An owner can cancel an open request, zeroing its remaining size. `cancelExitRequest(id): satisfy(requests[id].sizeRemaining > 0 AND requests[id].status' == Cancelled AND requests[id].sizeRemaining' == 0)`	✅	🎯
EE-RC-15	`rcEeCancelPartiallyFilledIsReachable`	A partially filled request can still be cancelled. `cancelExitRequest(id): satisfy(0 < requests[id].sizeRemaining AND requests[id].sizeRemaining < requests[id].sizeTotal AND requests[id].status' == Cancelled)`	✅	🎯
EE-RC-16	`rcEeSetServiceFeeAtCapIsReachable`	The owner can set the service fee at its cap with a live recipient. `setServiceFee(recipient, fee): satisfy(fee == MAX_SERVICE_FEE (0.25e18) AND serviceFee' == fee AND feeRecipient' == recipient AND recipient != 0)`	✅	🎯
EE-RC-17	`rcEeSetServiceFeeDisableIsReachable`	The owner can disable a previously positive service fee. `setServiceFee(recipient, 0): satisfy(serviceFee > 0 AND serviceFee' == 0)`	✅	🎯
EE-RC-18	`rcEeSetVaultAllowedIsReachable`	The owner can allow-list a previously de-listed vault. `setVaultAllowed(vault, true): satisfy(!isVaultAllowed[vault] AND isVaultAllowed[vault]')`	✅	🎯
EE-RC-19	`rcEeSetVaultDelistIsReachable`	The owner can de-list a previously allowed vault. `setVaultAllowed(vault, false): satisfy(isVaultAllowed[vault] AND !isVaultAllowed[vault]')`	✅	🎯
EE-RC-20	`rcEePlaceThenCancelIsReachable`	A lender can place a request and immediately cancel it. `id := placeExitRequest(vault, shares, ramp, deadline, fullFillOnly); cancelExitRequest(id): satisfy(requests[id].status' == Cancelled AND requests[id].sizeRemaining' == 0)`	✅	🎯
EE-RC-21	`rcEePlaceThenFillIsReachable`	A freshly placed request can immediately be filled. `id := placeExitRequest(vault, shares, ramp, deadline, fullFillOnly); total := requests[id].sizeTotal; fillExitRequest(id, depositAssets, minAssetsOut): satisfy(requests[id].sizeRemaining' < total)`	✅	🎯
EE-RC-22	`rcEeAllowThenPlaceIsReachable`	A newly allow-listed vault can immediately back a fresh request. `setVaultAllowed(vault, true); id := placeExitRequest(vault, shares, ramp, deadline, fullFillOnly): satisfy(!isVaultAllowed[vault] AND vaultOf[id]' == vault AND requests[id].sizeRemaining' > 0)`	✅	🎯
EE-RC-23	`rcEeRecoverErc20IsReachable`	The owner can recover stray erc20 tokens to a recipient. `recoverERC20(token, recipient, amount): satisfy(amount > 0 AND balance[token][recipient]' == balance[token][recipient] + amount)`	✅	🎯
EE-RC-24	`rcEeRecoverEtherIsReachable`	The owner can sweep the book's full ether balance. `recoverEther(recipient): satisfy(address(this).balance > 0 AND`	✅	🎯

		`address(this).balance' == 0)`		
EE-RC-25	`rcEePreviewFillPositiveIsReachable`	An open request can preview a positive fillable amount and payout. `p := previewFill(id): satisfy(p.filledShares > 0 AND p.assetsOut > 0)`	✅	🎯
EE-RC-26	`rcEeLastRequestIdPositiveIsReachable`	The request counter can be positive, reading one higher after a placement. `before := lastRequestId(): placeExitRequest(vault, shares, ramp, deadline, fullFillOnly): satisfy(lastRequestId()' > 0 AND lastRequestId()' == before + 1)`	✅	🎯
EE-RC-27	`rcEeOwnerRequestIndexViewsIsReachable`	An owner's request index is observable through the count and window views. `count := getOwnerRequestCount(owner); ids := getOwnerRequestIds(owner, 0, count): satisfy(count > 0 AND ids.length > 0)`	✅	🎯
EE-RC-28	`rcEeFillWithDepositIsReachable`	A filler can seed deposit liquidity and fill in the same call. `fillExitRequest(id, depositAssets, minAssetsOut): satisfy(depositAssets > 0 AND requests[id].sizeRemaining' < requests[id].sizeRemaining)`	✅	🎯
EE-RC-29	`rcEePlaceRampedRatesIsReachable`	A strictly ramped priority-fee request can be placed. `id = placeExitRequest(vault, shares, ramp, deadline, fullFillOnly): satisfy(requests[id].startRate < requests[id].endRate AND requests[id].rampStart < requests[id].rampEnd AND requests[id].sizeRemaining > 0)`	✅	🎯
EE-RC-30	`rcEeFillFullCloseIsReachable`	A fill can consume the entire remaining size and close the request filled. `requests[id].sizeRemaining == 1: fillExitRequest(id, depositAssets, minAssetsOut): satisfy(requests[id].sizeRemaining' == 0 AND requests[id].status' == Filled)`	✅	🎯
EE-RC-31	`rcEeSetServiceFeeClearsRecipientIsReachable`	A fee update can retire the recipient on file and leave the fee at zero. `setServiceFee(r, fee): satisfy(feeRecipient != 0 AND feeRecipient' == 0 AND serviceFee' == 0)`	✅	🎯

Reverts

Identification	Name	Description	Properties	Mutations
EE-RV-01	`rvEeSetVaultAllowedRevertsIff`	Allow-listing reverts iff the caller is not the owner or the vault is zero. `setVaultAllowed(vault, allowed) REVERTS <=> (msg.sender != owner() OR vault == 0)`	✅	🎯
EE-RV-02	`rvEeSetServiceFeeRevertsIff`	The fee setter reverts iff a non-owner calls, the fee tops the cap, or a fee has no recipient. `setServiceFee(recipient, fee) REVERTS <=> (msg.sender != owner OR fee > MAX_SERVICE_FEE (0.25e18) OR (recipient == 0 AND fee != 0))`	✅	🎯
EE-RV-03	`rvEePlaceExitRequestRevertsIff`	A placement reverts iff a registry, price, backing, validation, or width guard trips. `placeExitRequest(...) REVERTS <=> !(isVaultAllowed[vault] AND 0 < vault.convertToAssets(WAD) <= max_uint128 AND 0 < shares <= min(vault.allowance(msg.sender, this), vault.balanceOf(msg.sender), max_uint128) AND ramp.startRate <= ramp.endRate <= MAX_PRIORITY_FEE AND ramp.rampStart <= ramp.rampEnd <= max_uint32 AND now + max(1, ramp.rampEnd) <= deadline <= min(now + MAX_REQUEST_LIFETIME, max_uint40) AND now + MAX_REQUEST_LIFETIME <= max_uint256 AND nextId < max_uint256)`	✅	🎯
		Filling an open request past its deadline reverts. `fillExitRequest(id, depositAssets, minAssetsOut):`		

EE-RV-04	`rvEeFillRevertsWhenExpired`	`requests[id].owner != 0 AND requests[id].status == Open AND block.timestamp > requests[id].deadline => REVERTS`	✅	🎯
EE-RV-05	`rvEeFillRevertsOnNoFillableShares`	A deposit-free fill with no fillable shares reverts. `fillExitRequest(id, depositAssets, minAssetsOut): depositAssets == 0 AND fillable(id) == 0 => REVERTS [available(id) := min(requests[id].sizeRemaining, vaultOf[id].maxRedeem(requests[id].owner), vaultOf[id].allowance(requests[id].owner, this)), fillable(id) := (requests[id].fullFillOnly AND available(id) < requests[id].sizeRemaining) ? 0 : available(id)]`	✅	🎯🎯
EE-RV-06	`rvEeFillRevertsOnUnmeetableFloor`	A slippage floor above the ledger cap can never be met. `fillExitRequest(id, depositAssets, minAssetsOut): minAssetsOut > max_uint128 => REVERTS`	✅	🎯
EE-RV-07	`rvEeCancelExitRequestRevertsIff`	A cancel reverts iff the caller is not the request owner or it is not open. `cancelExitRequest(id) REVERTS <=> (requests[id].owner != msg.sender OR requests[id].status != Open)`	✅	🎯
EE-RV-08	`rvEeRecoverEtherRevertsOnGuards`	Ether recovery reverts for a non-owner caller or a zero recipient. `recoverEther(recipient): (msg.sender != owner OR recipient == 0) => REVERTS`	✅	🎯
EE-RV-09	`rvEeRecoverErc20RevertsIff`	ERC-20 recovery reverts iff the caller is not the owner or the recipient is zero. `recoverERC20(token, recipient, amount) REVERTS <=> (msg.sender != owner OR recipient == 0)`	✅	🎯
EE-RV-10	`rvEeRecoverErc721RevertsIff`	ERC-721 recovery reverts iff the caller is not the owner or the recipient is zero. `recoverERC721(token, recipient, tokenId) REVERTS <=> (msg.sender != owner OR recipient == 0)`	✅	🎯
EE-RV-11	`rvEeRecoverErc1155RevertsIff`	ERC-1155 recovery reverts iff the caller is not the owner or the recipient is zero. `recoverERC1155(token, recipient, tokenId) REVERTS <=> (msg.sender != owner OR recipient == 0)`	✅	🎯
EE-RV-12	`rvEeFeeRecipientNeverReverts`	feeRecipient() never reverts and reports the recipient the owner just set. `feeRecipient() never REVERTS; after setServiceFee(recipient, fee) succeeds, feeRecipient() == recipient`	✅	🎯
EE-RV-13	`rvEeServiceFeeNeverReverts`	serviceFee() never reverts and reports the rate the owner just set. `serviceFee() never REVERTS; after setServiceFee(recipient, fee) succeeds, serviceFee() == fee`	✅	🎯
EE-RV-14	`rvEeLastRequestIdNeverReverts`	lastRequestId() never reverts. `lastRequestId() never REVERTS`	✅	🎯
EE-RV-15	`rvEeIsVaultAllowedNeverReverts`	isVaultAllowed(vault) never reverts and reports the flag the owner just set. `isVaultAllowed(vault) never REVERTS; after setVaultAllowed(vault, allowed) succeeds, isVaultAllowed(vault) == allowed`	✅	🎯
EE-RV-16	`rvEeVaultOfNeverReverts`	vaultOf(id) never reverts and reports the vault the placement bound. `vaultOf(id) never REVERTS; after fresh = placeExitRequest(vault, shares, ramp, deadline, fullFillOnly) succeeds, vaultOf(fresh) == vault`	✅	🎯
EE-RV-17	`rvEeGetOwnerRequestCountNeverReverts`	getOwnerRequestCount(account) never reverts. `getOwnerRequestCount(account) never REVERTS`	✅	🎯
EE-RV-18	`rvEeGetExitRequestNeverReverts`	getExitRequest(id) never reverts. `getExitRequest(id) never REVERTS`	✅	🎯
EE-RV-19	`rvEePreviewFillNeverReverts`	previewFill(id) never reverts. `previewFill(id) never REVERTS`	✅	🎯
EE-RV-20	`rvEeGetRequestsNeverReverts`	getRequests(ids) never reverts on a bounded batch. `getRequests(ids) never REVERTS (ids.length <= 3)`	✅	🎯

EE-RV-21	`rvEeGetOwnerRequestIdsNeverReverts`	getOwnerRequestIds(account, offset, count) never reverts on a bounded page. `getOwnerRequestIds(account, offset, count) never REVERTS (count <= 3)`	✅	🎯
EE-RV-22	`rvEeGetRequestPreviewsNeverReverts`	getRequestPreviews(ids) never reverts on a bounded batch. `getRequestPreviews(ids) never REVERTS (ids.length <= 3)`	✅	🎯

Access Control

Identification	Name	Description	Properties	Mutations
EE-AC-01	`acEeOnlyRequestOwnerCancels`	A request is moved to Cancelled only by its owner. `(requests[id].status != Cancelled AND requests[id].status' == Cancelled) => msg.sender == requests[id].owner`	✅	🎯
EE-AC-02	`acEeOwnerIndexMovesOnlyBySelf`	An owner's request-id index moves only on that owner's own call. `ownerRequestIds[o].length' != ownerRequestIds[o].length => o == msg.sender`	✅	🎯
EE-AC-03	`acEeOnlyOwnerSetsFeeConfig`	The service-fee configuration moves only through the owner gate. `(serviceFee' != serviceFee OR feeRecipient' != feeRecipient) => msg.sender == owner`	✅	🎯
EE-AC-04	`acEeOnlyOwnerSetsVaultAllowlist`	The vault allow-list moves only through the owner gate. `isVaultAllowed[v]' != isVaultAllowed[v] => msg.sender == owner`	✅	🎯
EE-AC-05	`acEeOnlyOwnerMovesOwnership`	Book ownership moves only when the current owner initiates it. `owner' != owner => msg.sender == owner`	✅	🎯🎯
EE-AC-06	`acEeOnlyOwnerSweepsEther`	The book's ether balance is swept only by the owner. `address(this).balance' < address(this).balance => msg.sender == owner`	✅	🎯

MorphoExitBook_LinkedCore

- The core running its real bytecode; the controller, oracle, interest model and tokens linked as CVL models
- Fresh accrual pinned in the pricing, preview and fill-success rules, so the pending-interest regime is uncovered there
- Token layer a CVL model whose transfers succeed, each token ledger holding up to 7 accounts
- Loops running up to 3 iterations

Valid State

Identification	Name	Description	Properties	Mutations
ME-VS-01	`meServiceFeeCapped`	The owner can never set the protocol's cut of a filler's reward above the hard 25% cap. `serviceFee <= MAX_SERVICE_FEE (0.25e18)`	✅	🎯
ME-VS-02	`meServiceFeeHasRecipient`	A nonzero service fee always has a recipient; the protocol's cut cannot vanish into the void. `serviceFee != 0 => feeRecipient != 0`	✅	🎯🎯

ME-VS-03	`meZeroIdNeverCreated`	Id zero stays the 'no request' sentinel; no real request is ever filed under it. `requests[0].owner == 0`	✅	🎯
ME-VS-04	`meForwardIdsNeverCreated`	No request exists beyond the id counter; every request in the book was actually placed. `id > nextId => requests[id].owner == 0`	✅	🎯
ME-VS-05	`meIssuedIdsDense`	The registry has no holes; every id the counter has passed resolves to a real request. `1 <= id AND id <= nextId => requests[id].owner != 0`	✅	🎯
ME-VS-06	`meUncreatedAssetZero`	An id that was never issued names no payout asset. `requests[id].owner == 0 => requests[id].asset == 0`	✅	🎯
ME-VS-07	`meUncreatedSizeRemainingZero`	An id that was never issued has nothing left to fill. `requests[id].owner == 0 => requests[id].sizeRemaining == 0`	✅	🎯
ME-VS-08	`meUncreatedSizeTotalZero`	An id that was never issued never asked to exit anything. `requests[id].owner == 0 => requests[id].sizeTotal == 0`	✅	🎯
ME-VS-09	`meUncreatedPriceZero`	An id that was never issued carries no exit price. `requests[id].owner == 0 => requests[id].pricePerShare == 0`	✅	🎯
ME-VS-10	`meUncreatedStartRateZero`	An id that was never issued offers no opening priority-fee rate. `requests[id].owner == 0 => requests[id].startRate == 0`	✅	🎯
ME-VS-11	`meUncreatedEndRateZero`	An id that was never issued offers no peak priority-fee rate. `requests[id].owner == 0 => requests[id].endRate == 0`	✅	🎯
ME-VS-12	`meUncreatedRampStartZero`	An id that was never issued has no fee-ramp start scheduled. `requests[id].owner == 0 => requests[id].rampStart == 0`	✅	🎯
ME-VS-13	`meUncreatedRampEndZero`	An id that was never issued has no fee-ramp end scheduled. `requests[id].owner == 0 => requests[id].rampEnd == 0`	✅	🎯
ME-VS-14	`meUncreatedCreatedAtZero`	An id that was never issued has no birth date. `requests[id].owner == 0 => requests[id].createdAt == 0`	✅	🎯
ME-VS-15	`meUncreatedDeadlineZero`	An id that was never issued has no expiry. `requests[id].owner == 0 => requests[id].deadline == 0`	✅	🎯
ME-VS-16	`meUncreatedNotFullFill`	An id that was never issued makes no all-or-nothing demand. `requests[id].owner == 0 => !requests[id].fullFillOnly`	✅	🎯
ME-VS-17	`meUncreatedStatusOpen`	An id that was never issued still shows the default Open status. `requests[id].owner == 0 => requests[id].status == ExitStatus.Open`	✅	🎯
ME-VS-18	`meRemainingImpliesOpen`	Only an Open request has shares left to fill; closing settles the size to zero. `requests[id].sizeRemaining > 0 => requests[id].status == ExitStatus.Open`	✅	🎯🎯🎯
ME-VS-19	`meOpenImpliesRemaining`	An open request always has something left to fill. `requests[id].owner != 0 AND requests[id].status == Open => requests[id].sizeRemaining > 0`	✅	🎯
ME-VS-20	`meRemainingLeTotal`	A request can never owe more shares than the lender originally asked to exit. `requests[id].sizeRemaining <= requests[id].sizeTotal`	✅	🎯
ME-VS-21	`meCreatedSizeTotalPositive`	No empty requests; everything placed in the book asks to exit at least one share. `requests[id].owner != 0 => requests[id].sizeTotal != 0`	✅	🎯
ME-VS-22	`meStartRateLeEndRate`	The priority-fee ramp only climbs; waiting never offers a filler less than the opening rate. `requests[id].startRate <= requests[id].endRate`	✅	🎯
ME-VS-23	`meEndRateCapped`	Even at the top of its ramp, no request's priority fee breaks the book-wide 10% cap. `requests[id].endRate <= MAX_PRIORITY_FEE (0.1e18)`	✅	🎯

ME-VS-24	`meRampStartLeRampEnd`	A fee schedule is always well-formed; the climb cannot end before it begins. `requests[id].rampStart <= requests[id].rampEnd`	✅	🎯
ME-VS-25	`meRampEndsByDeadline`	The fee ramp finishes climbing before the request's deadline. `requests[id].owner != 0 => requests[id].createdAt + requests[id].rampEnd <= requests[id].deadline`	✅	🎯
ME-VS-26	`meDeadlineAfterCreation`	No request is born expired; every one opens with a live window to be filled. `requests[id].owner != 0 => requests[id].deadline > requests[id].createdAt`	✅	🎯
ME-VS-27	`meDeadlineWithinLifetime`	A request's fixed-price offer cannot be kept open longer than the book's configured lifetime cap. `requests[id].owner != 0 => requests[id].deadline <= requests[id].createdAt + MAX_REQUEST_LIFETIME`	✅	🎯
ME-VS-28	`meCreatedAtNotInFuture`	No request claims a placement time ahead of the chain, so the fee-ramp clock never runs backward. `requests[id].createdAt <= block.timestamp`	✅	🎯
ME-VS-29	`meZeroOwnerNoIds`	The zero address, the 'never created' marker, never accumulates a request listing of its own. `ownerRequestIds[address(0)].length == 0`	✅	🎯
ME-VS-30	`meOwnerIndexIssued`	An owner's listing only ever points at requests that were actually minted. `i < ownerRequestIds[o].length => (1 <= ownerRequestIds[o][i] AND ownerRequestIds[o][i] <= nextId)`	✅	🎯
ME-VS-31	`meOwnerIndexOwnership`	An owner's listing never shows anyone else's request. `i < ownerRequestIds[o].length AND ownerRequestIds[o][i] == rid => requests[rid].owner == o`	✅	🎯
ME-VS-32	`meOwnerIndexStrictlyIncreasing`	An owner's listing keeps placement order and never lists the same request twice. `i < j AND j < ownerRequestIds[o].length => ownerRequestIds[o][i] < ownerRequestIds[o][j]`	✅	🎯
ME-VS-33	`meZeroMarketIdNeverAllowed`	The empty market id, the book's 'no market' sentinel, is never on the allow-list. `!isMarketAllowed[bytes32(0)]`	✅	🎯
ME-VS-34	`meCreatedIffMarketBound`	A request exists exactly when it is bound to a market. `requests[id].owner != 0 <=> _marketIdOf[id] != bytes32(0)`	✅	🎯
ME-VS-35	`meAllowedMarketHasParamsRow`	A market cannot be allowed before the book holds its full description. `isMarketAllowed[mid] => _marketParams[mid] is registered (its row was written by setMarketAllowed)`	✅	🎯
ME-VS-36	`meBoundMarketHasParamsRow`	Every placed request resolves to a registered market description, even one disallowed later. `requests[id].owner != 0 => _marketParams[_marketIdOf[id]] is registered (its row was written by setMarketAllowed)`	✅	🎯
ME-VS-37	`meCreatedAssetIsMarketLoanToken`	A request always pays out in its own market's loan token. `requests[id].owner != 0 => requests[id].asset == _marketParams[_marketIdOf[id]].loanToken`	✅	🎯
ME-VS-38	`meParamsRowMarketExistsOnCore`	Every market the book registers really exists on the Morpho core. `_marketParams[mid] has a stored row => LidoLend.market[mid].lastUpdate != 0`	✅	🎯
ME-VS-39	`meFullFillOnlyOpenUntouched`	An all-or-nothing request is never partially drawn; while open it keeps its full size. `requests[id].fullFillOnly AND requests[id].status == Open => requests[id].sizeRemaining == requests[id].sizeTotal`	✅	🎯

ME-VS-40	`meOwnerIndexLenLeNextId`	Nobody's listing can hold more requests than the book has ever minted. `ownerRequestIds[o].length <= nextId`	✅	🎯
ME-VS-41	`meParamsRowCanonical`	A registry row always describes the id it is filed under; one id, one market description. `_marketParams[mid] has a stored row => MarketParamsLib.id(_marketParams[mid]) == mid`	✅	🎯

Variable Transitions

Identification	Name	Description	Properties	Mutations
ME-VT-01	`vtMeNextIdIncrementsByOne`	The id counter never moves backward and mints one id at a time. `nextId' == nextId OR nextId' == nextId + 1`	✅	🎯🎯
ME-VT-02	`vtMeRequestOwnerImmutable`	A created request's owner is permanent. `requests[id].owner != 0 => requests[id].owner' == requests[id].owner`	✅	🎯🎯
ME-VT-03	`vtMeClosedStatusTerminal`	A closed request's status is terminal. `(requests[id].owner != 0 AND requests[id].status != Open) => requests[id].status' == requests[id].status`	✅	🎯
ME-VT-04	`vtMeSizeRemainingNonIncreasing`	A created request's remaining size only shrinks. `requests[id].owner != 0 => requests[id].sizeRemaining' <= requests[id].sizeRemaining`	✅	🎯🎯
ME-VT-05	`vtMeRequestTermsFrozen`	A created request's economic terms are frozen. `requests[id].owner != 0 => (requests[id].asset' == requests[id].asset AND requests[id].sizeTotal' == requests[id].sizeTotal AND requests[id].pricePerShare' == requests[id].pricePerShare AND requests[id].startRate' == requests[id].startRate AND requests[id].endRate' == requests[id].endRate)`	✅	🎯🎯🎯
ME-VT-06	`vtMeRequestScheduleFrozen`	A created request's schedule, policy flag and bound market are frozen. `requests[id].owner != 0 => (requests[id].rampStart' == requests[id].rampStart AND requests[id].rampEnd' == requests[id].rampEnd AND requests[id].createdAt' == requests[id].createdAt AND requests[id].deadline' == requests[id].deadline AND requests[id].fullFillOnly' == requests[id].fullFillOnly AND _marketIdOf[id]' == _marketIdOf[id])`	✅	🎯🎯🎯🎯
ME-VT-07	`vtMeOwnerIndexAppendOnly`	An owner's listing grows only at its end, one entry at a time; its prefix is never rewritten. `(ownerRequestIds[o].length' == ownerRequestIds[o].length OR ownerRequestIds[o].length' == ownerRequestIds[o].length + 1) AND (j < ownerRequestIds[o].length => ownerRequestIds[o][j]' == ownerRequestIds[o][j])`	✅	🎯
		Only a freshly placed request grows a listing, and it always lands in its placer's own listing. `(ownerRequestIds[o].length' ==`		

ME-VT-08	`vtMeOwnerIndexAppendsFreshId`	`ownerRequestIds[o].length + 1 => (nextId' == nextId + 1 AND ownerRequestIds[o][ownerRequestIds[o].length]' == nextId')) AND ((nextId' == nextId + 1 AND requests[nextId'].owner' == o) => ownerRequestIds[o].length' == ownerRequestIds[o].length + 1)`	✅	🎯
ME-VT-09	`vtMeParamsRowLatches`	Once a market is registered its record is permanent, never unwritten and never blanked. `(written(_marketParams[mid]) => written(_marketParams[mid])') AND ((written(_marketParams[mid]) AND _marketParams[mid].loanToken != 0) => _marketParams[mid].loanToken' != 0)`	✅	🎯
ME-VT-10	`vtMeParamsRowValuesFrozen`	A registered market's params row is frozen; one id always means one market. `written(_marketParams[mid]) => (_marketParams[mid].loanToken' == _marketParams[mid].loanToken AND _marketParams[mid].collateralToken' == _marketParams[mid].collateralToken AND _marketParams[mid].oracle' == _marketParams[mid].oracle AND _marketParams[mid].irm' == _marketParams[mid].irm AND _marketParams[mid].lltv' == _marketParams[mid].lltv)`	✅	🎯
ME-VT-11	`vtMeCoreMarketStaysCreated`	No book entry point makes the Morpho core lose a market's creation stamp. `MORPHO.market[mid].lastUpdate != 0 => MORPHO.market[mid].lastUpdate' != 0`	✅	🎯

State Transitions

Identification	Name	Description	Properties	Mutations
ME-ST-01	`stMeSizeMovesOnlyOpen`	Size moves only on an open request. `requests[id].sizeRemaining' != requests[id].sizeRemaining => requests[id].status == Open`	✅	🎯
ME-ST-02	`stMeMintOwnsNewId`	A mint hands the new id to the caller. `nextId' != nextId => requests[nextId'].owner' == msg.sender`	✅	🎯
ME-ST-03	`stMeMintIndexesOwner`	A mint appends one entry to the caller's index. `nextId' != nextId => ownerRequestIds[msg.sender].length' == ownerRequestIds[msg.sender].length + 1`	✅	🎯

High Level

Identification	Name	Description	Properties	Mutations
ME-HL-01	`hlMeFillLeavesOtherRequests`	A fill moves no other request's remaining size. `fillExitRequest(id, depositAssets, minAssetsOut): id2 != id => requests[id2].sizeRemaining' == requests[id2].sizeRemaining`	✅	🎯
ME-HL-02	`hlMePlaceLeavesExistingRequests`	A place moves no pre-existing request's remaining size. `placeExitRequest(marketId, shares, ramp, deadline, fullFillOnly): oldId <= nextId => requests[oldId].sizeRemaining' ==`	✅	🎯

		`requests[oldId].sizeRemaining`		
ME-HL-03	`hlMeCancelLeavesOtherRequests`	A cancel moves no other request's status. `cancelExitRequest(id): id2 != id => requests[id2].status' == requests[id2].status`	✅	🎯
ME-HL-04	`hlMeRampRateWithinBand`	The fee rate a fill would charge never leaves the request's own ramp band. `rate = _priorityFeeRate(requests[id]): requests[id].startRate <= rate AND rate <= requests[id].endRate AND rate <= MAX_PRIORITY_FEE (0.1e18)`	✅	🎯
ME-HL-05	`hlMeRampRateMonotoneInTime`	The fee rate never falls as a request ages; the auction only moves one way. `requests[id].createdAt <= t1 AND t1 <= t2 => _priorityFeeRate(requests[id]) at t1 <= _priorityFeeRate(requests[id]) at t2`	✅	🎯
ME-HL-06	`hlMeFillKeepsNoRemainder`	A fill hands out everything it withdrew; the book keeps no remainder. `forall i. requests[i].owner != MorphoExitBook, feeRecipient != MorphoExitBook, msg.sender != MorphoExitBook, asset == requests[id].asset: fillExitRequest(id, depositAssets, minAssetsOut): bal[asset][MorphoExitBook]' == bal[asset][MorphoExitBook]`	✅	🎯
ME-HL-07	`hlMeFillDebitsSizeByPreviewedShares`	A fill that carries no filler deposit draws the request down by exactly the shares its own preview quoted. `p = previewFill(id); fillExitRequest(id, 0, minAssetsOut): requests[id].sizeRemaining' == requests[id].sizeRemaining - p.filledShares AND p.filledShares > 0 AND (requests[id].status' == Filled <=> requests[id].sizeRemaining' == 0)`	✅	🎯
ME-HL-08	`hlMeFillTouchesOnlyItsOwnMarket`	A fill touches exactly one market; no other market's core row or positions move. `other != marketIdOf[id]: fillExitRequest(id, depositAssets, minAssetsOut): LidoLend.market[other]' == LidoLend.market[other] (all six fields, lastUpdate included) AND LidoLend.position[other][who]' == LidoLend.position[other][who] (supplyShares, borrowShares, collateral)`	✅	🎯
ME-HL-09	`hlMeFillBurnsOwnMarketByTheDebitedShares`	The shares a fill burns in the request's own market equal the shares it debits in the book. `own == marketIdOf[id], owner == requests[id].owner, LidoLend.market[own].lastUpdate == block.timestamp: fillExitRequest(id, 0, minAssetsOut): LidoLend.position[own][owner].supplyShares - LidoLend.position[own][owner].supplyShares' == requests[id].sizeRemaining - requests[id].sizeRemaining'`	✅	🎯
ME-HL-10	`hlMePlacePricesAtItsOwnMarket`	On an accrued market, a placement binds the id it named and prices a share at its supply price. `LidoLend.market[marketId].lastUpdate == block.timestamp OR LidoLend.market[marketId].totalBorrowAssets == 0 OR _marketParams[marketId].irm == 0: id = placeExitRequest(marketId, shares, ramp, deadline, fullFillOnly): marketIdOf[id] == marketId AND requests[id].pricePerShare == WAD * (LidoLend.market[marketId].totalSupplyAssets + 1) / (LidoLend.market[marketId].totalSupplyShares + 1e6)`	✅	🎯
ME-HL-11	`hlMePreviewQuotesAtItsOwnMarket`	On an accrued market, a preview quotes its assets out of the request's own market. `own == marketIdOf[id], requests[id].owner != 0 AND (LidoLend.market[own].lastUpdate == block.timestamp OR LidoLend.market[own].totalBorrowAssets == 0 OR _marketParams[own].irm == 0): p = previewFill(id): p.assetsOut == p.filledShares * (LidoLend.market[own].totalSupplyAssets + 1) / (LidoLend.market[own].totalSupplyShares + 1e6)`	✅	🎯
ME-HL-12	`hlMeFillTakesAssetsOutOfItsMarket`	A fill pays the lender and the filler out of the market it drew from. `a := requests[id].asset, o := requests[id].owner, f := feeRecipient, LidoLend not in {o, msg.sender, f}: fillExitRequest(id, 0, minAssetsOut): bal[a][LidoLend]' < bal[a][LidoLend]`	✅	🎯

Unit Tests

Identification	Name	Description	Properties	Mutations
ME-UT-01	`utMeRampRateHoldsAtStartRate`	Through the opening hold the fee rate stays at the request's startRate. `block.timestamp - requests[id].createdAt < requests[id].rampStart => _priorityFeeRate(requests[id]) == requests[id].startRate`	✅	🎯
ME-UT-02	`utMeRampRateInterpolatesLinearly`	Across the climb the fee rate interpolates linearly from startRate to endRate. `elapsed == block.timestamp - requests[id].createdAt, requests[id].rampStart <= elapsed < requests[id].rampEnd => _priorityFeeRate(requests[id]) == startRate + floor((endRate - startRate) * (elapsed - rampStart) / (rampEnd - rampStart))`	✅	🎯
ME-UT-03	`utMeRampRatePlateausAtEndRate`	From rampEnd to the deadline the fee rate plateaus at endRate. `block.timestamp - requests[id].createdAt >= requests[id].rampEnd => _priorityFeeRate(requests[id]) == requests[id].endRate`	✅	🎯
ME-UT-04	`utMeFillPaysLenderOwedMinusPriorityFee`	A deposit-free fill pays the lender the price owed, capped at the assets withdrawn, minus the ramp's fee. `o != msg.sender AND o != feeRecipient AND o != MorphoExitBook AND LidoLend not in {o, msg.sender, feeRecipient}: fillExitRequest(id, 0, minAssetsOut): d(o) == owed - floor(owed * _priorityFeeRate(r) / WAD) [r := requests[id], a := r.asset, o := r.owner, d(x) := bal[a][x]' - bal[a][x], out := -d(LidoLend), owed := min(floor((r.sizeRemaining - r.sizeRemaining') * r.pricePerShare / WAD), out)]`	✅	🎯
ME-UT-05	`utMeFillSplitsFillerRewardAndServiceCut`	A deposit-free fill pays the filler the yield plus the fee net of the service cut, and the recipient the cut. `o, msg.sender, f, LidoLend all distinct AND MorphoExitBook not in {o, f}: fillExitRequest(id, 0, minAssetsOut): d(msg.sender) == gross - cut AND d(f) == cut [r := requests[id], a := r.asset, o := r.owner, f := feeRecipient, d(x) := bal[a][x]' - bal[a][x], out := -d(LidoLend), owed := min(floor((r.sizeRemaining - r.sizeRemaining') * r.pricePerShare / WAD), out), gross := out - owed + floor(owed * _priorityFeeRate(r) / WAD), cut := floor(gross * serviceFee / WAD)]`	✅	🎯
ME-UT-06	`utMeFillLeavesTheBookWithoutACorePosition`	A fill opens no supply position for the book; a fronted deposit is booked elsewhere. `own := _marketIdOf[id], LidoLend.feeRecipient != MorphoExitBook: fillExitRequest(id, depositAssets, minAssetsOut): LidoLend.position[own][MorphoExitBook].supplyShares' == LidoLend.position[own][MorphoExitBook].supplyShares`	✅	🎯
ME-UT-07	`utMeOwnerRequestIdsReadsTheRequestedWindow`	An owner's page read returns the window it was asked for, starting where it was told. `1 <= count <= 3 AND offset < ownerRequestIds[o].length: page := getOwnerRequestIds(o, offset, count): page.length == min(count, ownerRequestIds[o].length - offset) AND page[0]`	✅	🎯

		`== ownerRequestIds[o][offset]`		
ME-UT-08	`utMeAvailableSharesIsBackingCappedByIdle`	The shares a request can fill right now are its live backing, capped by what the market's idle assets buy back. `own := _marketIdOf[id], m := market[own], o := requests[id].owner, isAuthorized(o, this) AND (m.lastUpdate == block.timestamp OR m.totalBorrowAssets == 0 OR _marketParams[own].irm == 0): idle := m.totalSupplyAssets > m.totalBorrowAssets ? m.totalSupplyAssets - m.totalBorrowAssets : 0, backed := min(requests[id].sizeRemaining, position[own][o].supplyShares): _availableShares(id) == (backed * (m.totalSupplyAssets + 1) / (m.totalSupplyShares + 1e6) <= idle ? backed : idle * (m.totalSupplyShares + 1e6) / (m.totalSupplyAssets + 1))`	✅	🎯

Reachability

Identification	Name	Description	Properties	Mutations
ME-RC-01	`rcMePlaceExitRequestIsReachable`	A lender can open a fresh caller-owned request bound to the allow-listed market it names. `id = placeExitRequest(marketId, shares, ramp, deadline, fullFillOnly): satisfy(requests[id].owner == msg.sender AND requests[id].sizeRemaining > 0 AND marketIdOf[id] == marketId AND isMarketAllowed[marketId])`	✅	🎯
ME-RC-02	`rcMePlaceFullFillOnlyIsReachable`	An all-or-nothing request can be placed. `id = placeExitRequest(marketId, shares, ramp, deadline, true): satisfy(requests[id].fullFillOnly AND requests[id].sizeRemaining > 0)`	✅	🎯
ME-RC-03	`rcMePlaceOneWeiShareIsReachable`	The smallest non-zero request can be placed. `id = placeExitRequest(marketId, shares, ramp, deadline, fullFillOnly): satisfy(requests[id].sizeTotal == 1 AND requests[id].owner == msg.sender)`	✅	🎯
ME-RC-04	`rcMePlaceMaxUint128SharesIsReachable`	A request at the packed uint128 size boundary can be placed. `id = placeExitRequest(marketId, shares, ramp, deadline, fullFillOnly): satisfy(requests[id].sizeTotal == max_uint128 (2^128 - 1))`	✅	🎯
ME-RC-05	`rcMePlaceRampedRatesIsReachable`	A strictly ramped priority-fee request can be placed. `id = placeExitRequest(marketId, shares, ramp, deadline, fullFillOnly): satisfy(requests[id].startRate < requests[id].endRate AND requests[id].rampStart < requests[id].rampEnd AND requests[id].sizeRemaining > 0)`	✅	🎯
ME-RC-06	`rcMePlaceZeroRampIsReachable`	A zero-incentive request with no priority-fee ramp can be placed. `id = placeExitRequest(marketId, shares, ramp, deadline, fullFillOnly): satisfy(requests[id].endRate == 0 AND requests[id].sizeRemaining > 0)`	✅	🎯
ME-RC-07	`rcMePlaceDeadlineAtLifetimeCapIsReachable`	A deadline can land exactly on the max-lifetime cap. `id = placeExitRequest(marketId, shares, ramp, deadline, fullFillOnly): satisfy(requests[id].deadline' == requests[id].createdAt' + MAX_REQUEST_LIFETIME)`	✅	🎯
ME-RC-08	`rcMeFillPartialIsReachable`	A partial fill can shrink an open request and leave it open. `fillExitRequest(id, depositAssets, minAssetsOut): satisfy(requests[id].sizeRemaining' > 0 AND requests[id].sizeRemaining' <`	✅	🎯

		`requests[id].sizeRemaining AND requests[id].status' == Open)`		
ME-RC-09	`rcMeFillFullCloseIsReachable`	A fill can consume the entire remaining size and close the request filled. `requests[id].sizeRemaining == 1: fillExitRequest(id, depositAssets, minAssetsOut): satisfy(requests[id].sizeRemaining' == 0 AND requests[id].status' == Filled)`	✅	🎯
ME-RC-10	`rcMeFillPaysRequestOwnerIsReachable`	A third-party fill can pay exit proceeds to the request owner. `owner := requests[id].owner, asset := requests[id].asset: fillExitRequest(id, depositAssets, minAssetsOut): satisfy(requests[id].sizeRemaining' < requests[id].sizeRemaining AND balance[asset][owner]' > balance[asset][owner] AND msg.sender != owner AND owner != feeRecipient AND owner != LidoLend AND owner != MorphoExitBook)`	✅	🎯
ME-RC-11	`rcMeFillPaysServiceFeeIsReachable`	ME-UT-05's fee-leg witness; a fill under a positive service fee can credit the fee recipient. `serviceFee > 0, owner := requests[id].owner, asset := requests[id].asset: fillExitRequest(id, depositAssets, minAssetsOut): satisfy(requests[id].sizeRemaining' < requests[id].sizeRemaining AND balance[asset][feeRecipient]' > balance[asset][feeRecipient] AND feeRecipient != msg.sender AND feeRecipient != owner AND feeRecipient != LidoLend AND feeRecipient != MorphoExitBook)`	✅	🎯
ME-RC-12	`rcMeCancelExitRequestIsReachable`	An owner can cancel a live open request, zeroing its remaining size. `cancelExitRequest(id): satisfy(requests[id].sizeRemaining > 0 AND requests[id].status' == Cancelled AND requests[id].sizeRemaining' == 0)`	✅	🎯
ME-RC-13	`rcMeCancelLastWeiIsReachable`	A nearly-exhausted partially-filled request can still be cancelled. `cancelExitRequest(id): satisfy(requests[id].sizeRemaining == 1 AND requests[id].sizeTotal' > 1 AND requests[id].status' == Cancelled)`	✅	🎯
ME-RC-14	`rcMeSetServiceFeeIsReachable`	The owner can configure a positive service fee with a recipient. `setServiceFee(recipient, fee): satisfy(fee > 0 AND recipient != 0 AND serviceFee' == fee AND feeRecipient' == recipient)`	✅	🎯
ME-RC-15	`rcMeSetServiceFeeAtCapIsReachable`	The service fee can land exactly on its cap. `setServiceFee(recipient, fee): satisfy(serviceFee' == MAX_SERVICE_FEE (0.25e18))`	⏱️	🎯
ME-RC-16	`rcMeSetServiceFeeDisableIsReachable`	A previously positive service fee can be switched off. `setServiceFee(recipient, fee): satisfy(serviceFee > 0 AND serviceFee' == 0)`	✅	🎯
ME-RC-17	`rcMeSetMarketAllowedIsReachable`	The owner can allow-list a market and register its params row. `mid := id(params): setMarketAllowed(params, true): satisfy(!isMarketAllowed[mid] AND isMarketAllowed[mid]' AND _marketParams[mid].loanToken' == params.loanToken)`	✅	🎯
ME-RC-18	`rcMeSetMarketDelistIsReachable`	The owner can de-list a previously allowed market. `mid := id(params): setMarketAllowed(params, false): satisfy(isMarketAllowed[mid] AND !isMarketAllowed[mid]')`	✅	🎯
ME-RC-19	`rcMeRecoverERC20IsReachable`	The owner can recover stray erc20 tokens to a recipient. `recoverERC20(token, recipient, amount): satisfy(amount > 0 AND recipient != MorphoExitBook AND balance[token][recipient]' == balance[token][recipient] + amount)`	✅	🎯
		The owner can drain the book's full ether balance. `recoverEther(recipient):`		

ME-RC-20	`rcMeRecoverEtherIsReachable`	`satisfy(address(MorphoExitBook).balance > 0 AND address(MorphoExitBook).balance' == 0)`	✅	🎯
ME-RC-21	`rcMePlaceThenCancelIsReachable`	A freshly placed request can be cancelled in the same block. `id = placeExitRequest(marketId, shares, ramp, deadline, fullFillOnly); cancelExitRequest(id): satisfy(requests[id].status'' == Cancelled AND requests[id].sizeRemaining'' == 0 AND requests[id].sizeTotal'' > 0)`	✅	🎯
ME-RC-22	`rcMeAllowThenPlaceIsReachable`	A newly allow-listed market can immediately back a fresh request. `mid := id(params): setMarketAllowed(params, true); id = placeExitRequest(mid, shares, ramp, deadline, fullFillOnly): satisfy(_marketIdOf[id]'' == mid AND requests[id].sizeRemaining'' > 0)`	✅	🎯
ME-RC-23	`rcMePlaceThenPreviewIsReachable`	A freshly placed request can immediately preview a positive fillable amount. `id = placeExitRequest(marketId, shares, ramp, deadline, fullFillOnly); preview = previewFill(id): satisfy(preview.filledShares > 0)`	✅	🎯
ME-RC-24	`rcMeTwoOwnersCoexistIsReachable`	Requests of two distinct owners can coexist in the book. `id = placeExitRequest(marketId, shares, ramp, deadline, fullFillOnly): satisfy(requests[id].owner' == msg.sender AND requests[otherId].owner' != 0 AND requests[otherId].owner' != msg.sender)`	✅	🎯
ME-RC-25	`rcMePreviewFillPositiveIsReachable`	A standing request can preview positive fillable shares and positive assets out. `p = previewFill(id): satisfy(p.filledShares > 0 AND p.assetsOut > 0)`	✅	🎯
ME-RC-26	`rcMeLastRequestIdPositiveIsReachable`	The request counter can be positive and report the minted high-water mark. `lastRequestId(): satisfy(result == nextId AND result > 0)`	✅	🎯
ME-RC-27	`rcMeOwnerRequestCountPositiveIsReachable`	An active owner's request count can be positive and match the index length. `getOwnerRequestCount(owner): satisfy(result > 0 AND result == ownerRequestIds[owner].length)`	✅	🎯
ME-RC-28	`rcMeMarketParamsOfResolvesIsReachable`	A freshly placed request can resolve to the registered params of the market it named. `requestId = placeExitRequest(marketId, shares, ramp, deadline, fullFillOnly); p = marketParamsOf(requestId): satisfy(requests[requestId].owner' != 0 AND p.loanToken == requests[requestId].asset' AND p.loanToken != 0 AND _marketIdOf[requestId]' == marketId)`	✅	🎯
ME-RC-29	`rcMeFillWithDepositIsReachable`	A fill that fronts deposit liquidity into the core can shrink an open request. `fillExitRequest(id, depositAssets, minAssetsOut): satisfy(depositAssets > 0 AND requests[id].sizeRemaining' < requests[id].sizeRemaining)`	✅	🎯
ME-RC-30	`rcMeFillFullFillOnlyIsReachable`	An all-or-nothing request can be filled whole in one shot. `requests[id].sizeRemaining == 1: fillExitRequest(id, depositAssets, minAssetsOut): satisfy(requests[id].fullFillOnly' AND requests[id].sizeRemaining' == 0 AND requests[id].status' == Filled)`	✅	🎯
ME-RC-31	`rcMeFillLeavesTheOtherMarketIsReachable`	ME-HL-08's witness; a fill can execute while a second registered market stands untouched. `m := MORPHO.market(other); other != _marketIdOf[id] AND _marketParams[other] registered: fillExitRequest(id, depositAssets, minAssetsOut): satisfy(requests[id].sizeRemaining' < requests[id].sizeRemaining AND m.lastUpdate != 0`	✅	🎯

		`AND m.totalSupplyShares > 0 AND m.{totalSupplyAssets, totalSupplyShares, lastUpdate}' == m.{totalSupplyAssets, totalSupplyShares, lastUpdate})`		
ME-RC-32	`rcMeSetServiceFeeClearsRecipientIsReachable`	A fee update can retire the recipient on file and leave the fee at zero. `setServiceFee(r, fee): satisfy(feeRecipient != 0 AND feeRecipient' == 0 AND serviceFee' == 0)`	✅	🎯
ME-RC-33	`rcMeAvailableSharesIdleCapIsReachable`	A market with debt outstanding can cap a request below its own backing. `own := _marketIdOf[id], o := requests[id].owner, backed := min(requests[id].sizeRemaining, position[own][o].supplyShares): satisfy(market[own].totalBorrowAssets > 0 AND backed > 0 AND _availableShares(id) < backed)`	✅	🎯
ME-RC-34	`rcMeZeroPricePerShareIsReachable`	A placement on a market whose supply price rounds to zero still goes through. `id = placeExitRequest(marketId, shares, ramp, deadline, fullFillOnly): satisfy(requests[id].pricePerShare == 0 AND requests[id].sizeRemaining > 0)`	✅	🎯
ME-RC-35	`rcMeFillBelowFrozenPriceIsReachable`	A fill can land while the redeem under-produces the frozen-price entitlement; the frozen price is a ceiling on the lender's payout, never a floor. `o != msg.sender AND o != feeRecipient AND LidoLend not in {o, msg.sender, feeRecipient}: fillExitRequest(id, 0, minAssetsOut): satisfy(filled > 0 AND filled * pps / WAD > out AND d(o) < filled * pps / WAD) [filled := rem - rem', out := -d(LidoLend), pps := requests[id].pricePerShare]`	✅	🎯
ME-RC-36	`rcMeFillLeavesUnusedDepositInTheSource`	A fill can consume less than the filler funded; the remainder stays supplied under the filler's own core row, not refunded. `LidoLend not in {o, msg.sender, feeRecipient} AND LidoLend.feeRecipient != msg.sender: fillExitRequest(id, depositAssets, minAssetsOut): satisfy(depositAssets > 0 AND rem' < rem AND bal[asset][LidoLend]' > bal[asset][LidoLend] AND supplyShares[own][msg.sender]' > supplyShares[own][msg.sender])`	✅	🎯

Reverts

Identification	Name	Description	Properties	Mutations
ME-RV-01	`rvMeSetMarketAllowedRevertsIff`	setMarketAllowed rejects exactly a non-owner caller or an uncreated core market. `setMarketAllowed(params, allowed) REVERTS <=> (msg.sender != _owner OR MORPHO.market(params.id()).lastUpdate == 0)`	✅	🎯
ME-RV-02	`rvMeSetServiceFeeRevertsIff`	setServiceFee rejects exactly a non-owner, an over-cap fee, or a recipientless fee. `setServiceFee(recipient, fee) REVERTS <=> (msg.sender != _owner OR fee > MAX_SERVICE_FEE (0.25e18) OR (recipient == 0 AND fee != 0))`	✅	🎯
ME-RV-03	`rvMePlaceExitRequestRevertsWhenMarketNotAllowed`	Placing against a market off the allow-list reverts. `!isMarketAllowed[marketId] => placeExitRequest(marketId, shares, ramp, deadline, fullFillOnly) REVERTS`	✅	🎯
ME-RV-04	`rvMePlaceExitRequestRevertsWhenNotAuthorizedOnCore`	Placing without authorizing the book on the core reverts. `!MORPHO.isAuthorized(msg.sender, address(this)) => placeExitRequest(marketId, shares, ramp, deadline, fullFillOnly) REVERTS`	✅	🎯
ME-RV-05	`rvMePlaceExitRequestRevertsOnInsufficientBacking`	Placing more shares than the caller's core supply position reverts. `MORPHO.position(marketId, msg.sender).supplyShares < shares =>`	✅	🎯

		`placeExitRequest(marketId, shares, ramp, deadline, fullFillOnly) REVERTS`		
ME-RV-06	`rvMePlaceExitRequestRevertsOnZeroShares`	Placing a zero-share request reverts. `shares == 0 => placeExitRequest(marketId, shares, ramp, deadline, fullFillOnly) REVERTS`	✅	🎯
ME-RV-07	`rvMePlaceExitRequestRevertsOnHighPriorityFee`	Placing with an end priority-fee rate above the cap reverts. `ramp.endRate > MAX_PRIORITY_FEE (0.1e18) => placeExitRequest(marketId, shares, ramp, deadline, fullFillOnly) REVERTS`	✅	🎯
ME-RV-08	`rvMePlaceExitRequestRevertsOnStartAboveEnd`	Placing with a start rate above the end rate reverts. `ramp.startRate > ramp.endRate => placeExitRequest(marketId, shares, ramp, deadline, fullFillOnly) REVERTS`	✅	🎯
ME-RV-09	`rvMePlaceExitRequestRevertsOnRampStartAfterEnd`	Placing with a ramp start after the ramp end reverts. `ramp.rampStart > ramp.rampEnd => placeExitRequest(marketId, shares, ramp, deadline, fullFillOnly) REVERTS`	✅	🎯
ME-RV-10	`rvMePlaceExitRequestRevertsOnDeadlineInPast`	Placing with a deadline not in the future reverts. `deadline <= block.timestamp => placeExitRequest(marketId, shares, ramp, deadline, fullFillOnly) REVERTS`	✅	🎯
ME-RV-11	`rvMePlaceExitRequestRevertsOnDeadlineTooFar`	Placing with a deadline beyond the lifetime cap reverts. `deadline > block.timestamp + MAX_REQUEST_LIFETIME => placeExitRequest(marketId, shares, ramp, deadline, fullFillOnly) REVERTS`	✅	🎯
ME-RV-12	`rvMePlaceExitRequestRevertsOnRampPastDeadline`	Placing with a ramp that ends after the deadline reverts. `placeExitRequest(marketId, shares, ramp, deadline, fullFillOnly): block.timestamp + ramp.rampEnd > deadline => REVERTS`	✅	🎯
ME-RV-13	`rvMePlaceExitRequestRevertsOnSharesCastOverflow`	Placing a size above the uint128 record width reverts (snapshot cast). `placeExitRequest(marketId, shares, ramp, deadline, fullFillOnly): shares > max_uint128 => REVERTS`	✅	🎯
ME-RV-14	`rvMePlaceExitRequestRevertsOnDeadlineCastOverflow`	Placing a deadline above the uint40 record width reverts (snapshot cast). `placeExitRequest(marketId, shares, ramp, deadline, fullFillOnly): deadline > max_uint40 => REVERTS`	✅	🎯
ME-RV-15	`rvMePlaceExitRequestRevertsOnRampCastOverflow`	Placing ramp offsets above the uint32 record width reverts (snapshot cast). `placeExitRequest(marketId, shares, ramp, deadline, fullFillOnly): (ramp.rampStart > max_uint32 OR ramp.rampEnd > max_uint32) => REVERTS`	✅	🎯
ME-RV-16	`rvMePlaceValidRequestSucceeds`	A fully valid place on a freshly-accrued market succeeds; completeness twin of ME-RV-03..15. `placeExitRequest(marketId, shares, ramp, deadline, fullFillOnly): no ME-RV-03..15 revert cause holds (allow-list, core authorization, supply backing, nonzero size, ramp caps AND order, deadline window, record widths), m.lastUpdate == now (a freshly accrued market), WAD * (m.totalSupplyAssets + 1) / (m.totalSupplyShares + 1e6) <= max_uint128, now + MAX_REQUEST_LIFETIME <= max_uint256, nextId < max_uint256 => !REVERTS [m := market[marketId]]`	✅	🎯
ME-RV-17	`rvMeFillExitRequestRevertsWhenExpired`	Filling past the deadline reverts. `fillExitRequest(id, depositAssets, minAssetsOut): block.timestamp > requests[id].deadline => REVERTS`	✅	🎯
ME-RV-18	`rvMeFillExitRequestRevertsWhenAuthorizationRevoked`	Filling a request whose owner revoked the book's core authorization reverts. `fillExitRequest(id, depositAssets, minAssetsOut): !isAuthorized[requests[id].owner][address(this)] => REVERTS`	✅	🎯

ME-RV-19	`rvMeFillExitRequestRevertsOnZeroBacking`	Filling a request with no live supply backing reverts (pure-redeem leg). `fillExitRequest(id, depositAssets, minAssetsOut): depositAssets == 0 AND position[_marketIdOf[id]][requests[id].owner].supplyShares == 0 => REVERTS`	✅	🎯
ME-RV-20	`rvMeFillValidRequestSucceeds`	A fully backed, unexpired, no-floor pure-redeem fill succeeds on a fresh, debt-free market. `fillExitRequest(id, 0, 0): r.owner != 0 AND r.status == Open AND block.timestamp <= r.deadline AND isAuthorized[r.owner][address(this)] AND m.lastUpdate == block.timestamp AND m.totalBorrowAssets == 0 AND r.sizeRemaining <= position[i][r.owner].supplyShares <= m.totalSupplyShares AND r.sizeRemaining * (m.totalSupplyAssets + 1) >= m.totalSupplyShares + 1e6 AND !hookInProgress => !REVERTS [r := requests[id], i := _marketIdOf[id], m := market[i]]`	✅	🎯
ME-RV-21	`rvMeCancelExitRequestRevertsIff`	Cancel rejects exactly a non-owner caller or an already-closed request. `cancelExitRequest(id) REVERTS <=> (requests[id].owner != msg.sender OR requests[id].status != Open)`	✅	🎯
ME-RV-22	`rvMeRecoverEtherRevertsOnNotOwnerOrZeroRecipient`	Ether recovery rejects a non-owner caller or a zero recipient. `recoverEther(recipient): (msg.sender != owner() OR recipient == 0) => REVERTS`	✅	🎯
ME-RV-23	`rvMeRecoverTokenRevertsIff`	Erc-20 recovery rejects exactly a non-owner caller or a zero recipient. `token transfer modelled as succeeding: recoverERC20(token, recipient, amount) REVERTS <=> (msg.sender != owner() OR recipient == 0)`	✅	🎯
ME-RV-24	`rvMeRecoverNftRevertsIff`	Erc-721 recovery rejects exactly a non-owner caller or a zero recipient. `token transfer modelled as succeeding: recoverERC721(token, recipient, tokenId) REVERTS <=> (msg.sender != owner() OR recipient == 0)`	✅	🎯
ME-RV-25	`rvMeRecoverMultiTokenRevertsIff`	Erc-1155 recovery rejects exactly a non-owner caller or a zero recipient. `token transfer modelled as succeeding: recoverERC1155(token, recipient, tokenId) REVERTS <=> (msg.sender != owner() OR recipient == 0)`	✅	🎯
ME-RV-26	`rvMeMarketParamsOfNeverReverts`	marketParamsOf never reverts, an unregistered id included. `marketParamsOf(requestId): never REVERTS`	✅	🎯
ME-RV-27	`rvMeGetExitRequestNeverReverts`	getExitRequest never reverts, an unknown id included. `getExitRequest(id): never REVERTS`	✅	🎯
ME-RV-28	`rvMeGetRequestsNeverReverts`	The request batch read never reverts within the loop-unroll window. `ids.length <= 3: getRequests(ids) never REVERTS`	✅	🎯
ME-RV-29	`rvMeLastRequestIdNeverReverts`	lastRequestId never reverts. `lastRequestId(): never REVERTS`	✅	🎯
ME-RV-30	`rvMeGetOwnerRequestCountNeverReverts`	getOwnerRequestCount never reverts. `getOwnerRequestCount(owner) never REVERTS`	✅	🎯
ME-RV-31	`rvMeGetOwnerRequestIdsNeverReverts`	The owner-index page read never reverts, an out-of-range offset included. `getOwnerRequestIds(owner, offset, count) never REVERTS (count <= 3)`	✅	🎯
ME-RV-32	`rvMePreviewFillNeverReverts`	previewFill never reverts on a freshly-accrued market. `mid = _marketIdOf[id]: (LidoLend.market[mid].lastUpdate == 0 OR LidoLend.market[mid].lastUpdate == block.timestamp) AND LidoLend.market[mid].totalSupplyShares + 10^6 <= max_uint128 => previewFill(id) never REVERTS`	✅	🎯

ME-RV-33	`rvMeGetRequestPreviewsNeverReverts`	The preview batch read never reverts on a freshly-accrued market. `ids.length == 1, mid = _marketIdOf[ids[0]]: (LidoLend.market[mid].lastUpdate == 0 OR LidoLend.market[mid].lastUpdate == block.timestamp) AND LidoLend.market[mid].totalSupplyShares + 10^6 <= max_uint128 => getRequestPreviews(ids) never REVERTS`	✅	🎯
ME-RV-34	`rvMeFillExitRequestRevertsOnUnmeetableFloor`	A slippage floor above the token ledger's own ceiling can never be met. `fillExitRequest(id, depositAssets, minAssetsOut): minAssetsOut > max_uint128 => REVERTS`	✅	🎯

Access Control

Identification	Name	Description	Properties	Mutations
ME-AC-01	`acMeOnlyRequestOwnerCancels`	A request is moved to Cancelled only by its owner. `_book.requests[id].status != Cancelled AND _book.requests[id].status' == Cancelled => msg.sender == _book.requests[id].owner`	✅	🎯
ME-AC-02	`acMeOwnerIndexMovesOnlyForCaller`	An owner's request-id index moves only when that owner is the caller. `_book.ownerRequestIds[o].length' != _book.ownerRequestIds[o].length => o == msg.sender`	✅	🎯
ME-AC-03	`acMeOnlyOwnerSetsFeeConfig`	The service-fee configuration moves only for the owner. `(feeRecipient' != feeRecipient OR serviceFee' != serviceFee) => msg.sender == owner`	✅	🎯
ME-AC-04	`acMeOnlyOwnerMovesMarketAllowlist`	The market allow-list moves only for the owner. `isMarketAllowed[mid]' != isMarketAllowed[mid] => msg.sender == owner`	✅	🎯
ME-AC-05	`acMeOnlyOwnerWritesParamsRow`	A market-params registry row is written only by the owner. `(_marketParams[mid].loanToken' != _marketParams[mid].loanToken OR _marketParams[mid].collateralToken' != _marketParams[mid].collateralToken OR _marketParams[mid].oracle' != _marketParams[mid].oracle OR _marketParams[mid].irm' != _marketParams[mid].irm OR _marketParams[mid].lltv' != _marketParams[mid].lltv OR the call stores _marketParams[mid].loanToken while the row was never written before) => msg.sender == owner`	✅	🎯
ME-AC-06	`acMeOnlyOwnerMovesOwnership`	Ownership moves only when the current owner initiates it. `owner' != owner => msg.sender == owner`	✅	🎯
ME-AC-07	`acMeOnlyOwnerSweepsEther`	The book's ether balance is swept only by the owner. `address(this).balance' < address(this).balance => msg.sender == owner`	✅	🎯

Real Issues Found

Findings the review surfaced, from the manual pass and from verification itself. One rule reproduces each on the audited code; where the client committed to a fix, another states what the fix must bring.

[MEDIUM] `LidoLend::settle` closes a slice the market's idle liquidity could have paid out in cash (#42)

`settle` is the exit of last resort: a lender burns supply shares to cancel a borrower's debt and seize its collateral at par. The protocol states the intent in words: a lender may settle only the portion of its supply that an ordinary withdraw cannot serve, and the free-liquidity portion must be withdrawn instead (`lido-lend/SPECIFICATION.md`, the settle gate). The interface documents the cap that enforces it as cumulative: the canceled debt is capped at the part of `onBehalf`'s supply that the market's available liquidity cannot serve (`lido-lend/src/interfaces/ILidoLend.sol:35-36`), and the implementation matches that wording exactly (`lido-lend/src/LidoLend.sol:487`, with `_settleableSupply` at `lido-lend/src/LidoLend.sol:770-777`). The defect is the gap between the two: a cumulative cap constrains the total a lender may ever settle, and nothing constrains the single call. A lender whose holding outruns idle liquidity therefore clears the gate on any slice inside that headroom, including a slice the idle cash would have paid out in full, which is the case the specification says must be withdrawn instead. The rule's counterexample settles a slice worth one wei while 10034 sits idle, and the call does not revert. `LidoLendERC4626` is such a concentrated supplier, so the vault's own fallback path is built on the belief that the core refuses this (`lido-lend-markets/src/LidoLendERC4626.sol:184` and `:237-238`); that consequence is reasoned from the source rather than proved here, since the vault is outside this scene.

Reproduced on the audited code by LL-RV-32.

❌ Violated:

```
rule rvLlSettleRevertsWhenIdleCoversTheSlice(
    env e, LidoLend.MarketParams mp, uint256 supplyShares, uint256 borrowShares,
    address onBehalf, address borrower, address receiver
) {
    setupLidoLendValidState(e);
    require(ghostLlLastUpdate128 == to_mathint(e.block.timestamp),
        "SCOPE: frozen accrual, so the totals read here are the ones the guard prices the slice at");
    require(borrowShares == 0 && supplyShares > 0,
        "SCOPE: the supply-anchored branch, the one where the caller names the slice it settles");
    mathint freeBefore = ghostLlTotalSupplyAssets128 - ghostLlTotalBorrowAssets128;
    mathint sliceAssets = to_mathint(supplyShares)
        * (ghostLlTotalSupplyAssets128 + VIRTUAL_ASSETS_LL_CVL())
        / (ghostLlTotalSupplyShares128 + VIRTUAL_SHARES_LL_CVL());
    bool idleCoversSlice = sliceAssets <= freeBefore;

    settle@withrevert(e, mp, supplyShares, borrowShares, onBehalf, borrower, receiver);
    bool reverted = lastReverted;

    assert(idleCoversSlice => reverted,
        "settle must stand down when free liquidity alone could have paid the settled slice out in cash");
}
```

✖ Removed: the fix in fe636f7 deleted the supply anchored `settle` branch the rule exercises, so the rule cannot be stated on the mitigation commit.

[MEDIUM] `LidoLendERC4626::totalAssets` understates backing when the wrapper is the fee recipient (#71)

`LidoLendERC4626::totalAssets` prices the vault's raw stored core position against the projected market totals: the numerator is `LIDO_LEND.position(marketId, address(this)).supplyShares` as of `lastUpdate` (`lido-lend-markets/src/LidoLendERC4626.sol:123-128, 361-363`), while the denominator comes from `MorphoBalancesLib::expectedMarketBalances`, which already adds the pending `feeShares` to `totalSupplyShares` (`lido-lend/src/libraries/periphery/MorphoBalancesLib.sol:50-57`). When the vault itself is the core's `feeRecipient`, the accrual will also credit those `feeShares` to the vault's own position (`lido-lend/src/LidoLend.sol:716`), a credit the projection never adds to the numerator, so `totalAssets` understates the vault's backing by roughly the whole pending fee amount until anyone touches the market. Every OZ preview and conversion routes through the same `totalAssets`, so deposits are overpaid in shares and exits underpaid in assets during the stale window.

Reproduced on the audited code by VC-HL-10.

❌ Violated:

```
rule hlVcTotalAssetsStableAcrossOwnFeeCredit(env e, LidoLend.MarketParams p) {

    setupLidoLendERC4626_LinkedCore(e);
    require(ghostLlFeeRecipient == _Vault,
        "SCOPE: the owner named the vault the market's fee recipient");

    mathint viewBefore = totalAssets(e);

    _Core.accrueInterest(e, p);

    assert(to_mathint(totalAssets(e)) == viewBefore,
        "the price per share does not move when the accrual credits the vault's own row");
}
```

✅ Fixed in fb2c88d: passes on the mitigation commit.

[MEDIUM] `_handleSettle` blocks all settlement when a solvent queue root rounds to zero value (#73)

`_isBadDebt` compares a debt rounded up against a collateral value rounded down, so a position that owes less than it holds in exact arithmetic can still be called bad debt. The same rounding stops `allowedSlice` before any policy floor applies, so the queue root prices to nothing and `fallbackExit` halts at it. Targeting another borrower does not help: every settlement passes `_handleSettle`, which reads the root's verdict and reverts with `SettlementBadDebt`. MC-HL-45 states the verdict, MC-HL-46 states the cut, and both are Violated.

Reproduced on the audited code by MC-HL-45, MC-HL-46.

❌ Violated:

```
rule hlMcBadDebtVerdictIsNotRounding(env e) {

    setupLidoLendMarketControllerValidState(e);
    address root; uint256 sNext; uint256 bsNext; uint256 rNext;
    root, sNext, bsNext, rNext = nextSettlementBorrower(e);

    mathint shares = to_mathint(ghostLlCvlPosBorrowShares[root]);
    mathint collateral = to_mathint(ghostLlCvlPosCollateral[root]);
    require(root != 0 && shares != 0 && collateral != 0,
        "SAFE: only a position with debt and collateral is queued, and an empty queue has no root");

    require(to_mathint(ghostOraclePrice256[ghostLlCvlOracle]) >= ORACLE_PRICE_SCALE_MC_CVL() /
10000000000000
            && to_mathint(ghostOraclePrice256[ghostLlCvlOracle]) <= ORACLE_PRICE_SCALE_MC_CVL() * 10,
        "SCOPE: a price a deployed adapter reports, parity scaled by the pair's decimal gap, with a decade
of slack either side: UsdyStableOracle about 1.05e24, WstEthRateOracle about 1.21e36");

    require(to_mathint(ghostLlCvlTotalBorrowShares) >= to_mathint(ghostLlCvlTotalBorrowAssets)
            && to_mathint(ghostLlCvlTotalBorrowShares)
                <= to_mathint(ghostLlCvlTotalBorrowAssets) * VIRTUAL_SHARES_MC_CVL(),
        "SCOPE: a borrow index inside the band a market runs in. A first borrow mints VIRTUAL_SHARES per
asset, which is the upper edge, and only interest moves the ratio down toward the lower one. The residue a
zeroFloorSub can leave at the very bottom (no borrow assets against live shares) is a degenerate market,
not the one this rule is about");

    require(shares * (to_mathint(ghostLlCvlTotalBorrowAssets) + 1)
            * ORACLE_PRICE_SCALE_MC_CVL()
        <= collateral * to_mathint(ghostOraclePrice256[ghostLlCvlOracle])
```

```
            * (to_mathint(ghostLlCvlTotalBorrowShares) + VIRTUAL_SHARES_MC_CVL()),
        "the root owes no more than its collateral is worth, cross-multiplied so neither side rounds; the
+1 is the virtual asset toAssetsUp adds");

    assert(!hasBadDebt(e), "the gate reads bad debt off the position, not off its own rounding");
}
```

✖ Removed: accepted as design, restated on the mitigation commit as `hlMcBadDebtVerdictMatchesTheRoundedPredicate` and `hlMcZeroValuedRootClearsByExactShareRepay`.

[LOW] Permissionless `prune` lifts a count-derived ban and bypasses the restorer role (#8)

`_isBanned` calls an account banned when the latched bit is set or when its live flag count meets the current quorum (`lido-lend-markets/src/GuardianSignals.sol:300-302`), and only `_flag` writes that latch, so lowering the quorum under a standing flag count bans an account without it. Anyone can then call the permissionless `prune`, which drops stale flaggers and takes the count back under the quorum, while ending a latched ban needs `ACCESS_RESTORER_ROLE` (`lido-lend-markets/src/GuardianSignals.sol:124-127`).

Reproduced on the audited code by GS-HL-35, GS-AC-17.

❌ Violated:

```
rule hlGsPruneKeepsEffectiveBan(env e, GuardianSignals.Id id, address a) {

    setupGuardianSignalsValidState(e);
    require(ghostGsSuspectedByLen[id][a] == 0,
        "SCOPE: an empty suspect set isolates the flag channel");
    bool effectiveBefore = isBanned(e, id, a);

    prune(e, id, a);

    assert(effectiveBefore => isBanned(e, id, a),
        "the permissionless prune never lifts the effective ban");
}

rule acGsNoPermissionlessEffectiveUnban(env e, method f, calldataarg args,
    GuardianSignals.Id id, address a)
    filtered { f -> !EXCLUDED_FUNCTION_GS(f) } {

    setupGuardianSignals(e);
    require(!ghostGsHasRole[_Signals.ACCESS_RESTORER_ROLE()][e.msg.sender]
        && !ghostGsHasRole[_Signals.ACCESS_LIMITER_ROLE()][e.msg.sender]
        && !ghostGsHasRole[_Signals.MANAGER_ROLE()][e.msg.sender]
        && ghostGsGuardianPositions[to_bytes32(e.msg.sender)] == 0,
        "SCOPE: an unprivileged caller");
    bool effectiveBefore = isBanned(e, id, a);

    f(e, args);

    assert(effectiveBefore => isBanned(e, id, a),
        "an unprivileged caller cannot lift an effective ban");
}
```

✅ Fixed in d1200a4: passes on the mitigation commit as `hlGsPruneNeverLiftsLatchedBan` and `acGsPermissionlessUnbanOnlyByPrune`, the claim restated for the latch the fix adds.

[LOW] `CollateralActivationLib::debit` leaves market inflow charged when a remainder is demoted to dust (#11)

`debit` reverses the inflow the drained collateral had added, then demotes a sub-threshold activating remainder into freely withdrawable dust (`lido-lend-markets/src/libs/CollateralActivationLib.sol:203-224`). The decrement is computed before that demotion, from the drained amount alone, so the demoted remainder stays charged to the throttle and lengthens the activation delay of every later deposit.

Reproduced on the audited code by MC-HL-43.

❌ Violated:

```
rule hlMcSameTxDemotingWithdrawUndoesInflow(env e, LidoLendMarketController.Id id,
    ILidoLendMarketController.SupplyCollateralHookData ds,
    ILidoLendMarketController.WithdrawCollateralHookData dw) {

    setupLidoLendMarketControllerValidState(e);
    require(ds.onBehalf == dw.onBehalf, "SCOPE: one borrower supplies and withdraws");
    require(!ghostMcInstantActivationCallers[ds.caller],
        "SCOPE: an ungranted caller, the only one the throttle meters");
    require(ghostMcPosActivatingStart40[ds.onBehalf] == 0 && ghostMcPosDust128[ds.onBehalf] == 0,
        "SCOPE: a borrower with no standing lot and no dust");
    require(to_mathint(ds.assets) >= ghostMcCfgMinActivationAmount128,
        "SCOPE: supra-threshold, so the deposit is metered at all");
    require(ghostMcCfgActivatingLockAmount128 == 0,
        "SCOPE: no lock floor, so the demote branch is open");
    require(ghostLlCvlPosBorrowShares[ds.onBehalf] == 0,
        "SCOPE: a debt-free borrower, so the exit meets no capacity gate");
    mathint r = to_mathint(ds.assets) - to_mathint(dw.assets);
    require(r > 0 && r < ghostMcCfgMinActivationAmount128,
        "SCOPE: the withdrawal leaves a sub-threshold remainder");
    mathint inflowStart = currentInflow(e);
    require(inflowStart + to_mathint(ds.assets) <= ghostMcCfgMaxDelayInflow128,
        "SCOPE: headroom under the cap, so the deposit is metered in full");

    bytes supplyData = _HookEncoder.encodeSupplyCollateral(e, ds);
    on(e, id, ILidoLendMarketController.HookEvent.SupplyCollateral, supplyData);
    bytes withdrawData = _HookEncoder.encodeWithdrawCollateral(e, dw);
    on(e, id, ILidoLendMarketController.HookEvent.WithdrawCollateral, withdrawData);

    mathint inflowAfter = ghostMcActInflow128;
    mathint transientAfter = ghostMcTransientInflow[ds.onBehalf];

    assert(inflowAfter == inflowStart, "the demoted remainder leaves the inflow meter with the drain");
    assert(transientAfter == 0, "and the same-tx counter nets back to zero");
}
```

✅ Fixed in 0a41992: passes on the mitigation commit.

[LOW] `LidoLend::setFeeRecipient` leaves the interest window open, so the incoming recipient is paid the outgoing one's fees (#12)

`_accrueInterest` mints a market's fee to whichever address the global `feeRecipient` slot holds when accrual runs, and accrual is lazy, so the whole window since `lastUpdate` is paid on that market's next touch (`lido-lend/src/LidoLend.sol:699-725`). `setFeeRecipient` overwrites the address without accruing first, unlike its sibling `setFee` (`lido-lend/src/LidoLend.sol:131-137`, `lido-lend/src/LidoLend.sol:120-121`), so a window earned under the outgoing recipient is paid to the incoming one, and the slot being global redirects every fee-bearing market at once.

Reproduced on the audited code by LL-HL-52.

❌ Violated:

```
rule hlLlFeeRecipientRotationKeepsClosedWindow(
```

```
    env e, LidoLend.MarketParams mp, address newFeeRecipient
) {
    setupLidoLendValidState(e);
    require(mp.irm != 0, "SCOPE: interest-bearing market with a nonzero irm");
    require(ghostLlTotalBorrowAssets128 > 0, "SCOPE: outstanding debt to accrue on");
    require(ghostLlIrmBorrowRate[mp.irm] > 0, "SCOPE: a positive borrow rate applies");
    require(ghostLlFee128 > 0, "SCOPE: the market charges a fee");
    require(ghostLlLastUpdate128 < to_mathint(e.block.timestamp),
        "SCOPE: a window left open under the outgoing recipient");
    require(ERC20_ACCOUNT_BOUNDS(ghostLlLoanToken, newFeeRecipient),
        "SAFE: the model's account set is the narrowing erc20.spec declares; this re-pin lands the incoming
recipient's credit in a slot the supply-share ledger reads");
    mathint incomingBefore = ghostLlSupplyShares256[newFeeRecipient];

    setFeeRecipient(e, newFeeRecipient);
    accrueInterest(e, mp);

    assert(ghostLlSupplyShares256[newFeeRecipient] == incomingBefore,
        "the incoming recipient takes no share of a window that closed before the rotation");
}
```

✖ Removed: accepted as design, restated on the mitigation commit as `hlLlFeeWindowSettlesToRecipientOfRecord`.

[LOW] `_unsuspect` can extend the suspect surcharge window instead of shrinking it (#54)

`GuardianSignals::_unsuspect` removes the caller's report and then unconditionally calls `_snapshotSuspects`, which restamps `suspectReportedAt` to `block.timestamp` whenever any report survives (`lido-lend-markets/src/GuardianSignals.sol:409-423`). The surcharge window a borrower's activation delay is scored by reads that anchor plus `suspectWindow` (`lido-lend-markets/src/libs/CollateralActivationLib.sol:371-376`), so a guardian retracting their own suspicion extends the surviving reports' effect instead of shrinking it: the retraction pushes the account anchor forward and the borrower waits longer than before the retraction. The client first answered this as the documented snapshot model and later committed to a fix that does not move the snapshot forward on removal.

Reproduced on the audited code by GS-HL-32, GS-RC-40.

❌ Violated:

```
rule hlGsRetractionNeverExtendsExpiry(env e, GuardianSignals.Id id, address a, address g,
    address[] accounts, GuardianSignals.BlockInfoWithHash blk) {

    setupGuardianSignalsValidState(e);
    require(accounts.length == 1 && accounts[0] == a, "SCOPE: single target");
    require(ghostGsSuspectedByLen[id][a] == 2
        && ghostGsSuspectedByPositions[id][a][to_bytes32(g)] != 0
        && ghostGsSuspectedByPositions[id][a][to_bytes32(e.msg.sender)] != 0,
        "SCOPE: g and the caller hold the only two reports");
    require(g != e.msg.sender, "SCOPE: distinct reporters");
    require(ghostGsSigSuspectedAt40[id][a][g] + ghostGsSuspectWindow40 >= to_mathint(e.block.timestamp)
        && ghostGsSigSuspectedAt40[id][a][e.msg.sender] + ghostGsSuspectWindow40
            >= to_mathint(e.block.timestamp),
        "SCOPE: both reports still live");
    mathint anchorBefore = ghostGsSigSuspectReportedAt40[id][a];
    require(anchorBefore < to_mathint(e.block.timestamp),
        "SCOPE: the account was last touched strictly earlier");

    unsuspect(e, id, accounts, blk);

    assert(ghostGsSigSuspectReportedAt40[id][a] <= anchorBefore,
        "a retraction never pushes the account anchor forward");
```

```
}
```

✅ Fixed in 8329de6: passes on the mitigation commit.

[LOW] A seized remainder below the activation floor keeps its timer instead of folding into dust (#68)

A voluntary withdrawal folds a sub-threshold activating remainder into freely withdrawable dust (`lido-lend-markets/src/libs/CollateralActivationLib.sol:218-223`). A forced seizure drains with no lock floor and skips that fold, so whatever the liquidation leaves behind waits out its original activation timer, however small it is.

Reproduced on the audited code by MC-HL-44.

❌ Violated:

```
rule hlMcLiquidateSeizeFoldsSubThresholdRemainder(env e, LidoLendMarketController.Id id,
    ILidoLendMarketController.LiquidateHookData d) {

    setupLidoLendMarketControllerValidState(e);

    bytes data = _HookEncoder.encodeLiquidate(e, d);
    on(e, id, ILidoLendMarketController.HookEvent.Liquidate, data);

    mathint actAfter = ghostMcPosActivating128[d.borrower];

    assert(actAfter != 0 => actAfter >= ghostMcCfgMinActivationAmount128,
        "a seizure folds a sub-threshold remainder into dust, as the voluntary path does");
}
```

✅ Fixed in 3df6901: passes on the mitigation commit.

Mutation Testing

A mutant is one deliberately broken line under `mutations/`, and a rule that flips to Violated on it is proven to catch that break. `./certora/mutations/run_mutation.sh <target> <id>` plants it, runs the rule and restores the source. An id documents one killer rule, so a break two rules catch is planted twice.

Every entry names its fault, shows the diff and carries that command; the Mutations column of each property table links to the entry for that rule's mutant.

Scene	Mutants	Rules that catch them	Entries
GuardianSignals	204	207	`mutations/GuardianSignals/README.md`
LidoLendMarketController	226	223	`mutations/LidoLendMarketController/README.md`
LidoLend	207	208	`mutations/LidoLend/README.md`
LidoLendERC4626	115	133	`mutations/LidoLendERC4626/README.md`
Erc4626ExitBook	109	128	`mutations/Erc4626ExitBook/README.md`
MorphoExitBook	129	152	`mutations/MorphoExitBook/README.md`

Reproducing the Results

The Certora Prover can be run either remotely (using Certora's cloud infrastructure) or locally (building from source). Both modes share the same initial setup steps.

Prerequisites

The instructions below are for Ubuntu 24.04. For step-by-step installation details refer to this setup tutorial.

1. Install Java (tested with JDK 21)

```
sudo apt update
sudo apt install default-jre
java -version
```

2. Install pipx: installs Python CLI tools in isolated environments, avoiding dependency conflicts

```
sudo apt install pipx
pipx ensurepath
```

3. Install Certora CLI. To match a specific prover version, pin it explicitly (e.g. `certora-cli==8.16.2`)

```
pipx install certora-cli
```

4. Install solc-select and the Solidity compiler version required by the project

```
pipx install solc-select
solc-select install 0.8.35
solc-select use 0.8.35
```

5. Create a versioned solc symlink for Certora. Configuration files reference the compiler as `solc0.8.35` (without dashes), but solc-select only creates a generic `solc` binary. Create the symlink so Certora can find it:

```
mkdir -p ~/.local/bin
ln -sf ~/.solc-select/artifacts/solc-0.8.35/solc-0.8.35 ~/.local/bin/solc0.8.35
```

Verify `~/.local/bin` is in your `PATH`. If not, add it:

```
echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.bashrc
source ~/.bashrc
```

Finally, from the repository root, pull the library dependencies the build imports:

```
git submodule update --init --recursive
```

Remote Execution

1. Set up Certora key. You can get a free key through the Certora Discord or on their website. Once you have it, export it:

```
echo "export CERTORAKEY=<your_certora_api_key>" >> ~/.bashrc
```

> **Note:** If a local prover is installed (see below), it takes priority. To force remote execution, add the `--server production` flag:
>
> ```
> certoraRun certora/confs/GuardianSignals/valid_state.conf --server production
> ```

Local Execution

Follow the full build instructions in the CertoraProver repository (v8.16.2). Once the local prover is installed it takes priority over the remote cloud by default. Tested on Ubuntu 24.04.

1. Install prerequisites

```
# JDK 19+
sudo apt install openjdk-21-jdk

# SMT solvers (z3 and cvc5 are required, others are optional)
# Download binaries and place them in PATH:
#   z3:   https://github.com/Z3Prover/z3/releases
#   cvc5: https://github.com/cvc5/cvc5/releases

# LLVM tools
sudo apt install llvm

# Rust 1.81.0+
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
cargo install rustfilt

# Graphviz (optional, for visual reports)
sudo apt install graphviz
```

2. Set up build output directory

```
export CERTORA="$HOME/CertoraProver/target/installed/"
mkdir -p "$CERTORA"
export PATH="$CERTORA:$PATH"
```

3. Clone and build

```
git clone --recurse-submodules https://github.com/Certora/CertoraProver.git
cd CertoraProver
git checkout tags/8.16.2
./gradlew assemble
```

4. Verify installation with test example

```
certoraRun.py -h
cd Public/TestEVM/Counter
certoraRun counter.conf
```

Running the Suites

Run from the repository root, the directory that holds `src/` and `certora/`. The confs share one solver profile through `certora/confs/_base.conf`: a 48 GB heap, up to 16 concurrent rules, and an SMT budget of two hours per query, so a workstation run wants `java_args` and `max_concurrent_rules` lowered first. One conf differs: `settlement_slice`, whose budget is half an hour.

The flags below appear in the commands that follow, and each answers one question:

Flag	What it does	When to reach for it
`--rule <name>`	runs one property instead of the conf	a property that did not decide, so it gets the budget rather than sharing it
`--exclude_rule <names>`	runs the conf without those properties	the rest of a conf whose one heavy property is being run separately
`--method <name>`	runs one case of a parametric property	an invariant that decided every case but one. The bare name is enough while it is unique in the scene; an overloaded name needs the full ABI signature
`--exclude_method <name>`	runs every other case	the second half of that split. `--method` runs that one case and nothing else, not even the base case, so without this half the rest of the property stays unproven
`--java_args '-Xmx<N>g'`	sets the JVM heap the scene is held in	a joint scene on a machine with less memory than the confs assume
`--max_concurrent_rules <n>`	caps properties in flight	a run that dies with no verdict: the solver processes live outside the heap and follow the core count

GuardianSignals

Every rule in these decided, so each runs whole:

```
certoraRun certora/confs/GuardianSignals/access_control.conf
certoraRun certora/confs/GuardianSignals/access_control_roles.conf
certoraRun certora/confs/GuardianSignals/frontier.conf
certoraRun certora/confs/GuardianSignals/high_level.conf
certoraRun certora/confs/GuardianSignals/reachability.conf
certoraRun certora/confs/GuardianSignals/reverts.conf
certoraRun certora/confs/GuardianSignals/reverts_heartbeat.conf
certoraRun certora/confs/GuardianSignals/transitions.conf
certoraRun certora/confs/GuardianSignals/unit_tests.conf
certoraRun certora/confs/GuardianSignals/valid_state.conf
```

LidoLendMarketController

Every rule in these decided, so each runs whole:

```
certoraRun certora/confs/LidoLendMarketController/access_control.conf
certoraRun certora/confs/LidoLendMarketController/access_control_roles.conf
certoraRun certora/confs/LidoLendMarketController/reachability.conf
certoraRun certora/confs/LidoLendMarketController/settlement_slice.conf
certoraRun certora/confs/LidoLendMarketController/transitions.conf
certoraRun certora/confs/LidoLendMarketController/unit_tests.conf
certoraRun certora/confs/LidoLendMarketController/valid_state.conf
```

`high_level.conf` holds 3 rules that did not decide, so the conf runs without them and they run one at a time:

```
certoraRun certora/confs/LidoLendMarketController/high_level.conf \
  --exclude_rule hlMcActivationDelayMonotoneBanded hlMcInflowDecayLaws hlMcRootSliceHonorsPolicyFloors

certoraRun certora/confs/LidoLendMarketController/high_level.conf --rule hlMcActivationDelayMonotoneBanded
certoraRun certora/confs/LidoLendMarketController/high_level.conf --rule hlMcInflowDecayLaws
certoraRun certora/confs/LidoLendMarketController/high_level.conf --rule hlMcRootSliceHonorsPolicyFloors
```

`reverts.conf` holds 6 rules that did not decide, so the conf runs without them and they run one at a time:

```
certoraRun certora/confs/LidoLendMarketController/reverts.conf \
  --exclude_rule rvMcNextSettlementBorrowerNeverReverts rvMcOnBorrowRevertsAboveActiveCapacity
rvMcOnBorrowRevertsIff rvMcOnWithdrawCollateralRevertsIff
rvMcOnWithdrawRevertsWhenRemainingActiveInsufficient rvMcSettlementForNeverReverts

certoraRun certora/confs/LidoLendMarketController/reverts.conf --rule
rvMcNextSettlementBorrowerNeverReverts
certoraRun certora/confs/LidoLendMarketController/reverts.conf --rule
rvMcOnBorrowRevertsAboveActiveCapacity
certoraRun certora/confs/LidoLendMarketController/reverts.conf --rule rvMcOnBorrowRevertsIff
certoraRun certora/confs/LidoLendMarketController/reverts.conf --rule rvMcOnWithdrawCollateralRevertsIff
certoraRun certora/confs/LidoLendMarketController/reverts.conf --rule
rvMcOnWithdrawRevertsWhenRemainingActiveInsufficient
certoraRun certora/confs/LidoLendMarketController/reverts.conf --rule rvMcSettlementForNeverReverts
```

LidoLend

Every rule in these decided, so each runs whole:

```
certoraRun certora/confs/LidoLend/access_control.conf
certoraRun certora/confs/LidoLend/accrue_interest.conf
certoraRun certora/confs/LidoLend/reachability.conf
certoraRun certora/confs/LidoLend/reverts.conf
certoraRun certora/confs/LidoLend/unit_tests.conf
certoraRun certora/confs/LidoLend/valid_state.conf
certoraRun certora/confs/LidoLend/variable_transitions.conf
```

`high_level.conf` holds 15 rules that did not decide, so the conf runs without them and they run one at a time:

```
certoraRun certora/confs/LidoLend/high_level.conf \
  --exclude_rule hlLlBorrowDebtValueBounded hlLlBorrowDebtValueBoundedAssetsMode hlLlBorrowRepayNoProfit
hlLlLastBorrowLiquidationLeavesOthersHealthy hlLlLiquidateLeavesOthersHealthy
hlLlLiquidateNeverRaisesDebtSharePrice hlLlLiquidateRevertsOnHealthyBorrower
hlLlLiquidateSeizureBoundedByIncentive hlLlLiquidateSocializesAtMostBorrowerDebt hlLlOthersStayHealthy
hlLlRepayCostBounded hlLlRepayNeverRaisesDebtSharePrice hlLlSettleBorrowAnchoredLeavesOthersHealthy
hlLlSupplyBookValueBounded hlLlSupplyWithdrawAnyModeNoProfit

certoraRun certora/confs/LidoLend/high_level.conf --rule hlLlBorrowDebtValueBounded
certoraRun certora/confs/LidoLend/high_level.conf --rule hlLlBorrowDebtValueBoundedAssetsMode
certoraRun certora/confs/LidoLend/high_level.conf --rule hlLlBorrowRepayNoProfit
certoraRun certora/confs/LidoLend/high_level.conf --rule hlLlLastBorrowLiquidationLeavesOthersHealthy
certoraRun certora/confs/LidoLend/high_level.conf --rule hlLlLiquidateLeavesOthersHealthy
certoraRun certora/confs/LidoLend/high_level.conf --rule hlLlLiquidateNeverRaisesDebtSharePrice
certoraRun certora/confs/LidoLend/high_level.conf --rule hlLlLiquidateRevertsOnHealthyBorrower
certoraRun certora/confs/LidoLend/high_level.conf --rule hlLlLiquidateSeizureBoundedByIncentive
certoraRun certora/confs/LidoLend/high_level.conf --rule hlLlLiquidateSocializesAtMostBorrowerDebt
certoraRun certora/confs/LidoLend/high_level.conf --rule hlLlOthersStayHealthy
certoraRun certora/confs/LidoLend/high_level.conf --rule hlLlRepayCostBounded
certoraRun certora/confs/LidoLend/high_level.conf --rule hlLlRepayNeverRaisesDebtSharePrice
certoraRun certora/confs/LidoLend/high_level.conf --rule hlLlSettleBorrowAnchoredLeavesOthersHealthy
```

```
certoraRun certora/confs/LidoLend/high_level.conf --rule hlLlSupplyBookValueBounded
certoraRun certora/confs/LidoLend/high_level.conf --rule hlLlSupplyWithdrawAnyModeNoProfit
```

`state_transitions.conf` holds a rule that did not decide, so the conf runs without it and it runs one at a time:

```
certoraRun certora/confs/LidoLend/state_transitions.conf \
  --exclude_rule stLlSupplyShrinkIsAccounted

certoraRun certora/confs/LidoLend/state_transitions.conf --rule stLlSupplyShrinkIsAccounted
```

LidoLendERC4626

`erc4626.conf` holds 3 rules that did not decide, so the conf runs without them and they run one at a time:

```
certoraRun certora/confs/LidoLendERC4626/erc4626.conf \
  --exclude_rule redeemAtAdvertisedMaxDoesNotRevert sharesToAssetsRoundTripNeverExceeds
withdrawAtAdvertisedMaxDoesNotRevert

certoraRun certora/confs/LidoLendERC4626/erc4626.conf --rule redeemAtAdvertisedMaxDoesNotRevert
certoraRun certora/confs/LidoLendERC4626/erc4626.conf --rule sharesToAssetsRoundTripNeverExceeds
certoraRun certora/confs/LidoLendERC4626/erc4626.conf --rule withdrawAtAdvertisedMaxDoesNotRevert
```

`fast_convert.conf` holds a rule that did not decide, so the conf runs without it and it runs one at a time:

```
certoraRun certora/confs/LidoLendERC4626/fast_convert.conf \
  --exclude_rule hlVaMintRedeemNoProfit

certoraRun certora/confs/LidoLendERC4626/fast_convert.conf --rule hlVaMintRedeemNoProfit
```

`fast_exit.conf` holds 6 rules that did not decide, so the conf runs without them and they run one at a time:

```
certoraRun certora/confs/LidoLendERC4626/fast_exit.conf \
  --exclude_rule hlVaFallbackExitBurnsAtMostRequestedSharesWhenLiquidPartial
hlVaFallbackExitBurnsAtMostRequestedSharesWhenNoLiquidity
hlVaFallbackExitDeliversPreviewedLoanAssetsWhenLiquidCovers
hlVaFallbackExitDeliversPreviewedLoanAssetsWhenLiquidPartial rcVaFallbackExitLiquidLegCapCanRevert
rcVaFallbackExitPartialFillIsReachable

certoraRun certora/confs/LidoLendERC4626/fast_exit.conf --rule
hlVaFallbackExitBurnsAtMostRequestedSharesWhenLiquidPartial
certoraRun certora/confs/LidoLendERC4626/fast_exit.conf --rule
hlVaFallbackExitBurnsAtMostRequestedSharesWhenNoLiquidity
certoraRun certora/confs/LidoLendERC4626/fast_exit.conf --rule
hlVaFallbackExitDeliversPreviewedLoanAssetsWhenLiquidCovers
certoraRun certora/confs/LidoLendERC4626/fast_exit.conf --rule
hlVaFallbackExitDeliversPreviewedLoanAssetsWhenLiquidPartial
certoraRun certora/confs/LidoLendERC4626/fast_exit.conf --rule rcVaFallbackExitLiquidLegCapCanRevert
certoraRun certora/confs/LidoLendERC4626/fast_exit.conf --rule rcVaFallbackExitPartialFillIsReachable
```

`fast_settle.conf` holds 2 rules that did not decide, so the conf runs without them and they run one at a time:

```
certoraRun certora/confs/LidoLendERC4626/fast_settle.conf \
  --exclude_rule hlVaFallbackExitDoesNotDiluteRemainingHolders
hlVaFallbackExitPaysSeizedCollateralToReceiver

certoraRun certora/confs/LidoLendERC4626/fast_settle.conf --rule
hlVaFallbackExitDoesNotDiluteRemainingHolders
certoraRun certora/confs/LidoLendERC4626/fast_settle.conf --rule
hlVaFallbackExitPaysSeizedCollateralToReceiver
```

`high_level.conf` holds 15 rules that did not decide, so the conf runs without them and they run one at a time:

```
certoraRun certora/confs/LidoLendERC4626/high_level.conf \
  --exclude_rule acVaAllowanceMovesOnlyByOwnerOrSpender hlVaDepositRedeemRoundTripNoProfit
hlVaFallbackExitBurnsAgainstPreviewedBudget hlVaFallbackExitBurnsAtMostRequestedSharesWhenLiquidPartial
hlVaFallbackExitBurnsAtMostRequestedSharesWhenNoLiquidity hlVaFallbackExitBurnsProRataOfConsumedBudget
hlVaFallbackExitDeliversPreviewedLoanAssetsWhenLiquidCovers
hlVaFallbackExitDeliversPreviewedLoanAssetsWhenLiquidPartial hlVaFallbackExitDoesNotDiluteRemainingHolders
hlVaFallbackExitPaysSeizedCollateralToReceiver hlVaMintRedeemNoProfit hlVaSettleBurnsAtMostRequestedShares
hlVaSettleBurnsProRataOfConsumedBudget hlVaSettleDoesNotDiluteRemainingHolders hlVaSupplyMovesOnlyOnFlows

certoraRun certora/confs/LidoLendERC4626/high_level.conf --rule acVaAllowanceMovesOnlyByOwnerOrSpender
certoraRun certora/confs/LidoLendERC4626/high_level.conf --rule hlVaDepositRedeemRoundTripNoProfit
certoraRun certora/confs/LidoLendERC4626/high_level.conf --rule hlVaFallbackExitBurnsAgainstPreviewedBudget
certoraRun certora/confs/LidoLendERC4626/high_level.conf --rule
hlVaFallbackExitBurnsAtMostRequestedSharesWhenLiquidPartial
certoraRun certora/confs/LidoLendERC4626/high_level.conf --rule
hlVaFallbackExitBurnsAtMostRequestedSharesWhenNoLiquidity
certoraRun certora/confs/LidoLendERC4626/high_level.conf --rule
hlVaFallbackExitBurnsProRataOfConsumedBudget
certoraRun certora/confs/LidoLendERC4626/high_level.conf --rule
hlVaFallbackExitDeliversPreviewedLoanAssetsWhenLiquidCovers
certoraRun certora/confs/LidoLendERC4626/high_level.conf --rule
hlVaFallbackExitDeliversPreviewedLoanAssetsWhenLiquidPartial
certoraRun certora/confs/LidoLendERC4626/high_level.conf --rule
hlVaFallbackExitDoesNotDiluteRemainingHolders
certoraRun certora/confs/LidoLendERC4626/high_level.conf --rule
hlVaFallbackExitPaysSeizedCollateralToReceiver
certoraRun certora/confs/LidoLendERC4626/high_level.conf --rule hlVaMintRedeemNoProfit
certoraRun certora/confs/LidoLendERC4626/high_level.conf --rule hlVaSettleBurnsAtMostRequestedShares
certoraRun certora/confs/LidoLendERC4626/high_level.conf --rule hlVaSettleBurnsProRataOfConsumedBudget
certoraRun certora/confs/LidoLendERC4626/high_level.conf --rule hlVaSettleDoesNotDiluteRemainingHolders
certoraRun certora/confs/LidoLendERC4626/high_level.conf --rule hlVaSupplyMovesOnlyOnFlows
```

`reachability.conf` holds 6 rules that did not decide, so the conf runs without them and they run one at a time:

```
certoraRun certora/confs/LidoLendERC4626/reachability.conf \
  --exclude_rule rcVaFallbackExitDefaultIterationCapSettles rcVaFallbackExitFullBalanceIsReachable
rcVaFallbackExitIsReachable rcVaFallbackExitLiquidLegCapCanRevert rcVaFallbackExitLiquidLegIsReachable
rcVaFallbackExitPartialFillIsReachable

certoraRun certora/confs/LidoLendERC4626/reachability.conf --rule
rcVaFallbackExitDefaultIterationCapSettles
certoraRun certora/confs/LidoLendERC4626/reachability.conf --rule rcVaFallbackExitFullBalanceIsReachable
certoraRun certora/confs/LidoLendERC4626/reachability.conf --rule rcVaFallbackExitIsReachable
certoraRun certora/confs/LidoLendERC4626/reachability.conf --rule rcVaFallbackExitLiquidLegCapCanRevert
certoraRun certora/confs/LidoLendERC4626/reachability.conf --rule rcVaFallbackExitLiquidLegIsReachable
certoraRun certora/confs/LidoLendERC4626/reachability.conf --rule rcVaFallbackExitPartialFillIsReachable
```

`reverts.conf` holds a rule that did not decide, so the conf runs without it and it runs one at a time:

```
certoraRun certora/confs/LidoLendERC4626/reverts.conf \
  --exclude_rule rvVaFallbackExitDoesNotRevertOnShortfall

certoraRun certora/confs/LidoLendERC4626/reverts.conf --rule rvVaFallbackExitDoesNotRevertOnShortfall
```

`transitions.conf` holds 4 rules that did not decide, so the conf runs without them and they run one at a time:

```
certoraRun certora/confs/LidoLendERC4626/transitions.conf \
  --exclude_rule stVaAllowanceSpendOnly stVaThirdPartyDebitIsAuthorized vtVaMarketIdentityImmutable
vtVaMarketParamsImmutable

certoraRun certora/confs/LidoLendERC4626/transitions.conf --rule stVaAllowanceSpendOnly
certoraRun certora/confs/LidoLendERC4626/transitions.conf --rule stVaThirdPartyDebitIsAuthorized
certoraRun certora/confs/LidoLendERC4626/transitions.conf --rule vtVaMarketIdentityImmutable
certoraRun certora/confs/LidoLendERC4626/transitions.conf --rule vtVaMarketParamsImmutable
```

`valid_state.conf` holds 3 rules that did not decide, so the conf runs without them and they run one at a time:

```
certoraRun certora/confs/LidoLendERC4626/valid_state.conf \
  --exclude_rule assetIsLoanToken marketIdCanonical offsetBounded

certoraRun certora/confs/LidoLendERC4626/valid_state.conf --rule assetIsLoanToken
certoraRun certora/confs/LidoLendERC4626/valid_state.conf --rule marketIdCanonical
certoraRun certora/confs/LidoLendERC4626/valid_state.conf --rule offsetBounded
```

LidoLend_LinkedController

`fast_hooks.conf` holds 2 rules that did not decide, so the conf runs without them and they run one at a time:

```
certoraRun certora/confs/LidoLend_LinkedController/fast_hooks.conf \
  --exclude_rule hlLwcSettleSeizureDrainsBucketsExactly hlLwcSettleStrandsAtMostSubWeiDebt

certoraRun certora/confs/LidoLend_LinkedController/fast_hooks.conf --rule
hlLwcSettleSeizureDrainsBucketsExactly
certoraRun certora/confs/LidoLend_LinkedController/fast_hooks.conf --rule
hlLwcSettleStrandsAtMostSubWeiDebt
```

`properties.conf` holds 7 rules that did not decide, so the conf runs without them and they run one at a time:

```
certoraRun certora/confs/LidoLend_LinkedController/properties.conf \
  --exclude_rule hlLwcBorrowLeavesDebtCoveredByActiveCollateral hlLwcMaxWithdrawableIsWithdrawableEndToEnd
hlLwcSettleSeizureDrainsBucketsExactly hlLwcSettleStrandsAtMostSubWeiDebt
hlLwcWithdrawCollateralLeavesDebtCoveredByActiveCollateral rcLwcAccrualLiftsFeeRecipientQuote
rcLwcSettleAtRootCutsDebt

certoraRun certora/confs/LidoLend_LinkedController/properties.conf --rule
hlLwcBorrowLeavesDebtCoveredByActiveCollateral
certoraRun certora/confs/LidoLend_LinkedController/properties.conf --rule
hlLwcMaxWithdrawableIsWithdrawableEndToEnd
certoraRun certora/confs/LidoLend_LinkedController/properties.conf --rule
hlLwcSettleSeizureDrainsBucketsExactly
certoraRun certora/confs/LidoLend_LinkedController/properties.conf --rule
hlLwcSettleStrandsAtMostSubWeiDebt
certoraRun certora/confs/LidoLend_LinkedController/properties.conf --rule
hlLwcWithdrawCollateralLeavesDebtCoveredByActiveCollateral
certoraRun certora/confs/LidoLend_LinkedController/properties.conf --rule
rcLwcAccrualLiftsFeeRecipientQuote
certoraRun certora/confs/LidoLend_LinkedController/properties.conf --rule rcLwcSettleAtRootCutsDebt
```

`valid_state.conf` holds a rule that did not decide, so the conf runs without it and it runs one at a time:

```
certoraRun certora/confs/LidoLend_LinkedController/valid_state.conf \
  --exclude_rule heapKeyFreshCoreBacked

certoraRun certora/confs/LidoLend_LinkedController/valid_state.conf --rule heapKeyFreshCoreBacked
```

LidoLendERC4626_LinkedCore

Every rule in these decided, so each runs whole:

```
certoraRun certora/confs/LidoLendERC4626_LinkedCore/properties.conf
```

Erc4626ExitBook_LinkedVaultCore

Every rule in these decided, so each runs whole:

```
certoraRun certora/confs/Erc4626ExitBook_LinkedVaultCore/properties.conf
certoraRun certora/confs/Erc4626ExitBook_LinkedVaultCore/valid_state.conf
```

MorphoExitBook_LinkedCore

Every rule in these decided, so each runs whole:

```
certoraRun certora/confs/MorphoExitBook_LinkedCore/valid_state.conf
```

`properties.conf` holds a rule that did not decide, so the conf runs without it and it runs one at a time:

```
certoraRun certora/confs/MorphoExitBook_LinkedCore/properties.conf \
  --exclude_rule rcMeSetServiceFeeAtCapIsReachable

certoraRun certora/confs/MorphoExitBook_LinkedCore/properties.conf --rule rcMeSetServiceFeeAtCapIsReachable
```

Splitting a parametric rule

An invariant proves a base case, a transient-storage case and one case per external method, all sharing one budget. The two commands below are one split: over the same signature they partition the rule, so a rule counts as decided when both come back.

```
certoraRun certora/confs/LidoLend_LinkedController/valid_state.conf \
  --rule heapKeyFreshCoreBacked --method settle

certoraRun certora/confs/LidoLend_LinkedController/valid_state.conf \
  --rule heapKeyFreshCoreBacked --exclude_method settle
```

Running on a smaller machine

```
certoraRun certora/confs/LidoLend_LinkedController/properties.conf \
  --java_args '-Xmx96g' --max_concurrent_rules 2
```

The solver processes live outside the JVM heap and their number follows the core count, so a run that dies without a verdict wants fewer concurrent rules rather than a larger heap.

Resources

- Certora Tutorials: official Certora documentation and guided tutorials
- AlexZoid personal site and FV resources: write-ups on verifying real protocols, alongside a curated collection of specs, examples and references
- Updraft Assembly & Formal Verification Course: comprehensive video course covering assembly and formal verification from the ground up
- Find Highs Using Certora Formal Verification: practical guide with a companion repo of simplified examples based on real code and bugs from private audits
- RareSkills Certora Book: structured tutorial covering CVL syntax, patterns, and common pitfalls